Lecture Notes on Theory of Automata and Formal Languages

Computer Science Department
University of Malaga

February 17, 2026

Contents

1 Preamble to discrete mathematics **9**

1.1 Reminder 10

1.2 Finite and Infinite Sets 19

1.3 Practices of chapter 1 25

2 Languages and grammars **27**

2.1 Languages 28

2.1.1 Concept of language 28

2.1.2 Language representation 35

2.2 Grammatical representation systems 36

2.2.1 Definition and functioning of grammars 36

2.2.2 Classification of rules 40

2.2.3 Classification of grammars 42

2.2.4 Classification of languages 43

2.3 Questions about languages 44

2.4 Operations and closures of languages 45

2.5 Closures for the types of languages 48

2.6 Practices of chapter 2 51

3 Regular expressions **53**
3.1 Definition of the regular expressions 54
3.2 Some properties of regular expressions 61
3.3 Practices of chapter 3 64

4 Finite automata **65**
4.1 Deterministic finite automata 66
4.2 Nondeterministic finite automata 73
4.3 Minimum deterministic finite automaton 80
4.4 Equivalences 80
4.5 Practices of chapter 4 81

5 Regularity conditions **83**
5.1 Myhill-Nerode 84
5.2 Pumping lemma 88

6 Context-free languages **91**
6.1 Derivations and Ambiguity 92
6.2 Recursion 97
6.3 CFG Simplification 99
6.4 Normal forms 101
6.5 Closure properties 104

6.6 Non-Deterministic Pushdown Automata 105
6.7 Pumping 114

7 The Turing machine **115**
7.1 Formal definition 116
7.2 Computability, decidability, enumerability 131

8 Recursive functions **135**
8.1 Formal definition 136
8.2 Enumerability, decidability, computability 145

9 The WHILE language **149**
9.1 Formal definition 150
9.2 Computability, decidability, enumerability 160

10 Turing completeness **163**
10.1 Extended While language 164
10.2 Recursive ⇒ While-computable 166

11 Universality **171**
11.1 Program numbering 172
11.1.1 Encoding of While programs 177

11.1.2 Decoding While programs 181
11.2 Universal function 186
11.3 Universal program 188

12 Theoretical limits of computing **195**
12.1 Halting problem 196
12.2 The busy beaver function 199

Appendices **203**
.1 Proposed solutions to the practices 205
.2 Proof of closure properties for regular languages 209
.3 Examples of closure properties with NDFA 221
.4 $L \in \mathscr{L}.2 \Rightarrow L \in \mathscr{L}(\text{NPA})$ 236
.5 F(WHILE) to REC proof 236
.6 Functioning of the Universal While program 246
References 251

Foreword

Writing just another textbook in theory of automata, formal languages or computability looks like pointless: there exist so many now, that we can hardly do better. However, every lecturer knows what is to be taught, and the lectures benefit from a particular nomenclature in a given subject, or a specific proof. In the end, you miss that textbook that satisfies your needs, that fits your communication skills or your expertise in the area. Such is the motivation behind compiling these lectures notes in the subject of Theory of automata and formal languages, in the traditional sense, but also, to be an ever-evolving document, that integrates what others have expressed before, the examples and exercises that were designed by other authors in distant parts of the planet, unsubscribing from a particular terminology, and unifying different forms of representing the same concepts. It is the sign of the XXI century: collaboration, contribution to common initiatives. The resulting manuscript has the structure of [Ramos and Morales, 2011], which is the reference book in Spanish for the subject *Teoría de autómatas y lenguajes formales* of our Computer Science program. Much of the

writing in LaTeX has been done by students taking the subject, and curated by myself, as professor of Computer Science and Artificial Intelligence at the University of Malaga.

> List of contributors: J. Alcázar Pérez, D. Arroyo Torres, A. Burgueño Romero, N. Carrasco González, E. Carreira Villalta, P. Fazio Arrabal, R. Ferreira Garcés, M. Guerrero Ramírez, P. Gutierrez Ruiz, S. López Molina, M. López Reviriego, Y. Martín Benzaquen, M. Mejía Jiménez, D. Mérida Granados, J. Morales Joya, R. Moret Galán, J. Pérez Bravo, E. Pérez Haro, A. Racero Serrano, A. Román Navas, A. Ruiz Vallecillo and J. Sarmiento.

Aditionaly, a library of open-source software that implements the main concepts in this manuscript is also offered for the programming language GNU/Octave, and it has been made public under CC0 license. This is just version 2.0, coming years will see it develop, enrich and accommodate new knowledge, teaching expertise and students' feedback.

Francisco Vico
November 2018
updated February 2023

Notation

$\mathbb{N}$	the natural/whole numbers set, i.e. $\{1, 2, \ldots\}$ (U+2115 *\mathbb{N}*)
$\mathbb{N}_0$	the natural numbers set with zero , i.e. $\{0, 1, 2, \ldots\}$
$\mathbb{R}$	the real numbers set (U+211d *\mathbb{R}*)
$\mathbb{Z}$	the integer numbers set (U+2124 *\mathbb{Z}*)
$\mathbb{Q}$	the rational numbers set (U+211a *\mathbb{Q}*)
$\emptyset$	the empty set (U+2205 *\emptyset*)
$\in$	belongs to (U + 2208 *\in*)
$\notin$	does not belong to (U + 2209 *\notin*)
$\forall$	universal quantification, for all; for any; for each (U+2200 *\forall*)
$\exists$	existential quantification, there exists (U+2203 *\exists*)
$\exists!$	uniqueness quantification, there exists exactly one
$\subseteq$	the (infix) subset relation between sets (U+2286 *\subseteq*)
$\subset$	the (infix) proper subset relation between sets (U+228a *\subset*)

$\cup$	the infix union operation on sets (U+222a *\cup*)
$\cap$	the infix intersection operation on sets (U+2229 *\cap*)
$\wedge$	logical conjunction (U+2227 *\land*)
$\vee$	logical (inclusive) disjunction (U+2228 *\lor*)
$-$	the infix set difference operation on sets (U+2212)
$\times$	the infix cartesian product of sets (U+00d7 *\times*)
A^n	the postfix n-fold cartesian product of A, i.e. $A \times \overset{n}{\cdots} \times A$
$\mathscr{P}(A)$	the power set of A (also represented by 2^A) (U+1D4AB *\mathcal{P}*)
$\vdash$	turnstile symbol to denote transition in automata (U+22A2 *\vdash*)
ε	empty string (U+03B5 *\varepsilon*)
$\mathscr{L}$	set of languages (U+2112 *\mathcal{L}*)
$\boldsymbol{\mathcal{L}}$	language generated by a metastring (U+1D4DB *\pazocal{L}*)
Σ	an alphabet Sigma (U+03A3 *\Sigma*)
Γ	an alphabet Gamma (U+0393 *\Gamma*)
δ	a transition function delta (U+03B4 *\delta*)
Δ	a transition relation Delta (U+0394 *\Delta*)
$:=$	assignment in While language (U+2254 *\coloneqq*)

The HTML/XML numeric character reference format is **&#x*hhhh*;**, where *hhhh* is a Unicode hexadecimal number. For example, $\mathscr{L}$ writes as **ℒ** in HTML/XML.

Chapter 1

Preamble to discrete mathematics

1.1 Reminder 10
1.2 Finite and Infinite Sets 19
1.3 Practices of chapter 1 25

No one shall expel us from the paradise that Cantor has created.
David Hilbert

1.1 Reminder

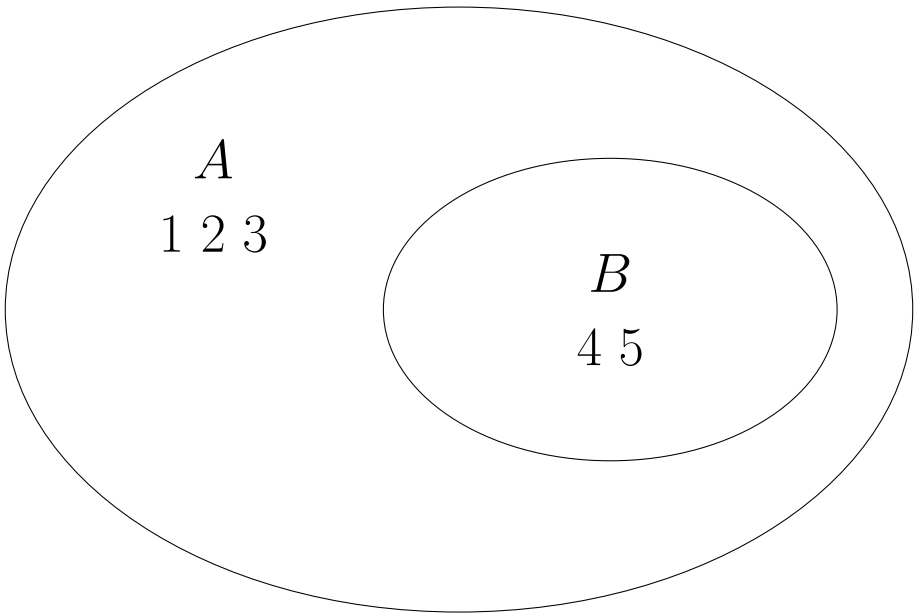
Definition 1.1.1. Power set (*conjunto de las partes de un conjunto*)

$$\mathscr{P}(A) = \{B \,:\, B \subseteq A\}$$

Remark. $\emptyset, A \in \mathscr{P}(A)$

Definition 1.1.2. Proper subset (*subconjunto propio*)
B is a proper subset of A iff

$$B \subset A \wedge B \neq \emptyset$$



Definition 1.1.3. Partition (*partición*)
$\Pi = \{A_1, A_2, \dots\} \subseteq \mathscr{P}(A)$ is a partition of A iff

- $\bigcup_{A_i \in \Pi} A_i = A$
- $A_i \cap A_j = \emptyset, \forall A_i \neq A_j$
- $A_i \neq \emptyset, \forall A_i \in \Pi$

Definition 1.1.4. Cartesian product (*producto cartesiano*)

$$A \times B = \{(a, b) \,:\, a \in A, b \in B\}$$

Remark. (a, b) is an ordered pair (*par ordenado*).

Definition 1.1.5. Relation (*relación*)
$R \subseteq A \times B$ is a relation from A to B.

Definition 1.1.6. Binary relation (*relación binaria*)
$R \subseteq A \times A$ is a binary relation on A.

Definition 1.1.7. Properties of binary relations

- R is Reflexive iff $(a, a) \in R \;\; \forall a \in A$
- R is Symmetric iff $(a, b) \in R \Rightarrow (b, a) \in R$
- R is Antisymmetric iff $(a, b) \in R \wedge a \neq b \Rightarrow (b, a) \notin R$
- R is Transitive iff $(a, b) \in R \wedge (b, c) \in R \Rightarrow (a, c) \in R$

Definition 1.1.8. Equivalence relation (*relación de equivalencia*)
R is an equivalence relation iff it is reflexive, symmetric and transitive.

Definition 1.1.9. Equivalence class (*clase de equivalencia*)
Being A a set, R an equivalence relation on A and $a \in A$, $[a]$ is an equivalence class defined as

$$[a] = \{b \in A \,:\, (a, b) \in R\}$$

Remark. An equivalence relation of a set defines a partition for that set (quotient set) where each element of the partition is an equivalence class.

[AI] Example of binary set and equivalence relation.

Definition 1.1.10. Identity relation I (*relación identidad*)

$$I = \{(a, a) \,:\, a \in A\}.$$

Definition 1.1.11. Inverse relation R^{-1} (*relación inversa*)

$$R^{-1} = \{(a, b) \,:\, (b, a) \in R\}$$

Definition 1.1.12. Power of a relation R^n (*potencia de una relación*)
Given $R \subseteq A \times A$,

$$R^n = \begin{cases} R & n = 1 \\ \big\{(a, b) \,:\, \exists x \in A, (a, x) \in R^{n-1} \wedge (x, b) \in R\big\} & n > 1 \end{cases}$$

Definition 1.1.13. Closure of relations (*cierre de relaciones*)
The closure of a relation under a certain property is the smallest relation which contains the first one and satisfies that property.

— Reflexive closure of R is $R \cup I$

— Symmetric closure of R is $R \cup R^{-1}$

— Transitive closure of R is $R^\infty = \bigcup_{n=1}^{\infty} R^n$

[**AI**] Example of types of relations.

Example 1.1.1. Given $A = \{a, b, c\}$, and the binary relation $R = \{(b, c), (c, a), (a, c)\}$, these are the powers 2, 3 and its transitive closure.

```
octave> powerrelation(['b','c'], ['c','a'], ['a','c'], 2);
{(b,c), (c,a), (a,c)}^2 = {(a,a), (b,a), (c,c)}

octave> powerrelation(['b','c'], ['c','a'], ['a','c'], 3);
{(b,c), (c,a), (a,c)}^3 = {(a,c), (b,c), (c,a)}

octave> powerrelation(['b','c'], ['c','a'], ['a','c']);
{(b,c), (c,a), (a,c)}^2 = {(a,a), (b,a), (c,c)}
{(b,c), (c,a), (a,c)} ∪ {(a,a), (b,a), (c,c)} = {(a,a), (a,c), (b,a),
(b,c), (c,a), (c,c)}

{(b,c), (c,a), (a,c)}^3 = {(a,c), (b,c), (c,a)}
{(a,a), (a,c), (b,a), (b,c), (c,a), (c,c)} ∪ {(a,c), (b,c), (c,a)} =
{(a,a), (a,c), (b,a), (b,c), (c,a), (c,c)}

{(b,c), (c,a), (a,c)}^∞= {(a,a), (a,c), (b,a), (b,c), (c,a), (c,c)}
```

Definition 1.1.14. Function (*función*)

$$f : A \to B \text{ is a function iff } \forall a \in A \ \exists! \ (a, b) \in f.$$

Remark. "!" denotes "one and only one".

Definition 1.1.15. Domain and range of a function (*dominio y rango*)
Given $f : A \to B$,

$$Dom(f) = \{a \in A \ : \ f(a) = b, b \in B\}$$
$$Rg(f) = \{f(a) \in B \ : \ a \in A\}$$

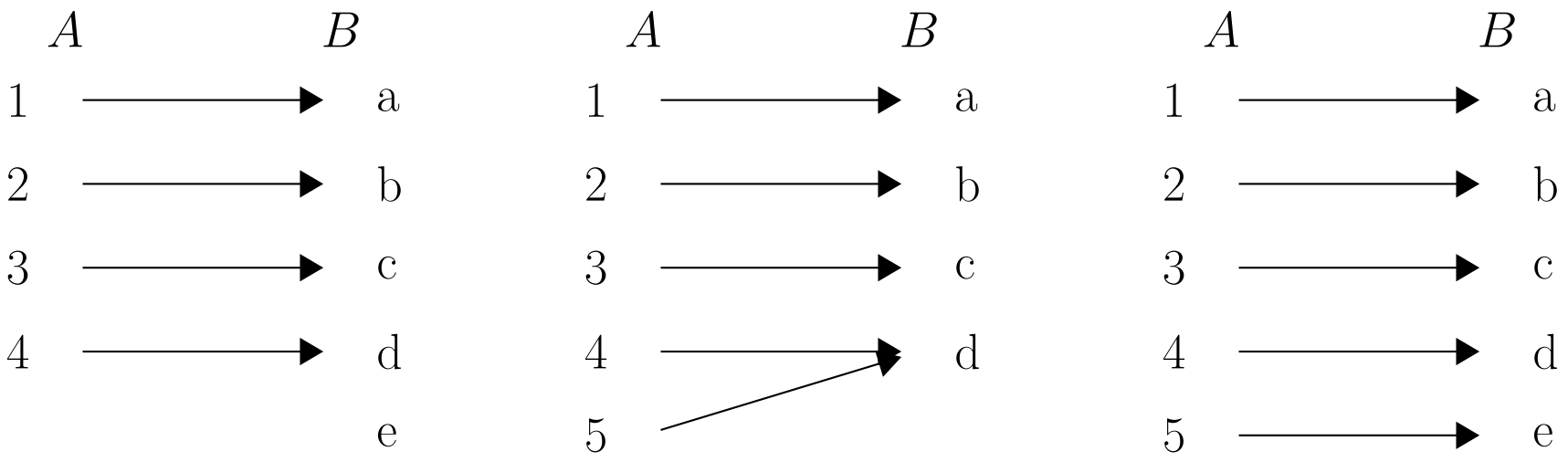
Definition 1.1.16. Injective function (*función inyectiva*)
$f : A \rightarrowtail B$ iff $f(x) = f(z) \Rightarrow x = z$

Remark. This also means that $x \neq z \Rightarrow f(x) \neq f(z)$

Definition 1.1.17. Surjective function (*función sobreyectiva*)
$f : A \twoheadrightarrow B$ iff $Rg(f) = B$

Definition 1.1.18. Bijective function (*función biyectiva*)
$f : A \leftrightarrow B$ iff it is injective and surjetive.

Example 1.1.2. Examples of the three types of functions: injective (left), surjective (center), and bijective function (right).



Definition 1.1.19. Closure of a set under an operation
A set is closed under an operation iff this operation can be defined as a function

$$f : A \times A \to A$$

Definition 1.1.20. Associative property (*propiedad asociativa*)
Being A a closed set for the operation $\bullet$, ($\bullet : A \times A \to A$), $\bullet$ is associative iff

$$(x \bullet y) \bullet z = x \bullet (y \bullet z) \, \forall x, y, z \in A$$

Definition 1.1.21. Semigroup (*semigrupo*)
A semigroup is a pair $(A, \bullet)$ where A is a set closed under $\bullet$, which is an associative operation.

Definition 1.1.22. Neutral element (*elemento neutro*)
Also called identity element, $e \in A$ is the neutral element for an operation $\bullet$ iff

$$\forall a \in A \;\; a \bullet e = e \bullet a = a$$

Definition 1.1.23. Monoid (*monoide*)
It is a semigroup with a neutral element for the operation.

Definition 1.1.24. Strict closure of a set under an operation (*cierre estricto*)
Being $\bullet$ an operation on A and $B \subseteq A$, $B^\bullet$ is the strict closure of B for $\bullet$ defined as:

— $x \in B \Rightarrow x \in B^\bullet$

— $x, y \in B^\bullet \Rightarrow x \bullet y \in B^\bullet$

— no other element belongs to $B^\bullet$.

Definition 1.1.25. Wide closure of a set under an operation (*cierre amplio*)
Being $(A, \bullet)$ a monoid and $B \subseteq A$, the wide closure of B under $\bullet$ is

$$B^{\bullet e} = B^\bullet \cup \{e\}$$

1.2 Finite and Infinite Sets

Definition 1.2.1. Equipotent sets (*conjuntos equipotenciales*)
$A \sim B$ (are equipotent) iff $\exists f : A \leftrightarrow B$

Definition 1.2.2. Cardinality of a set (*cardinalidad de un conjunto*)

$$|A| = \begin{cases} 0 & \text{if } A = \emptyset \\ n \in \mathbb{N} & \text{if } A \sim \{1, 2, ..., n\} \\ \aleph_0 & \text{if } A \sim \mathbb{N} \\ \aleph_i & \text{if } A \sim \mathscr{P}(B), |B| = \aleph_{i-1}, i > 0 \end{cases}$$

Remark.

$\emptyset = \{\ \}$ is the empty set (*conjunto vacío*)

$|\mathbb{N}| = \aleph_0$ (*aleph-null* (*álef cero*))

$|\mathbb{R}| = \aleph_1$ (*aleph-one* (*álef uno*)), according to the continuum hipothesis.

Proposition 1.2.1. $\aleph_1 - \aleph_0 = \aleph_1$

Definition 1.2.3. Finite set (*conjunto finito*)
A is a finite set iff $|A| \in \mathbb{N}_0$.

Definition 1.2.4. Infinite set (*conjunto infinito*)
A is an infinite set iff $|A|$ is not finite.

Remark. A is infinite iff $|A| \in \{\aleph_i : i \in \mathbb{N}_0\}$.

Definition 1.2.5. Countable infinite set (*conjunto infinito numerable*)
A is a countable infinite set iff $A \sim \mathbb{N}$.

Definition 1.2.6. Countable set (*conjunto numerable*)
A is countable if it is finite or countable infinite.

Definition 1.2.7. Uncountable set (*conjunto no numerable*)
A is uncountable if it is not countable.

Remark. $\mathbb{R}$ is an uncountable set.

Proposition 1.2.2. Any subset of a countable set is countable.

Proposition 1.2.3. The finite union of countable sets is countable.

Proposition 1.2.4. Countable infinite union of countable sets is countable.

Proposition 1.2.5. The cartesian product of two countable sets is countable.

Proposition 1.2.6. The finite power of a countable set is countable.

Theorem 1.2.1. Canthor's theorem (*teorema de Cantor*)

$$|A| < |\mathscr{P}(A)|$$

Remark. $|\mathbb{N}| < |\mathscr{P}(\mathbb{N})| \Rightarrow \aleph_0 < \aleph_1 \Rightarrow |\mathbb{N}| < |\mathbb{R}|$

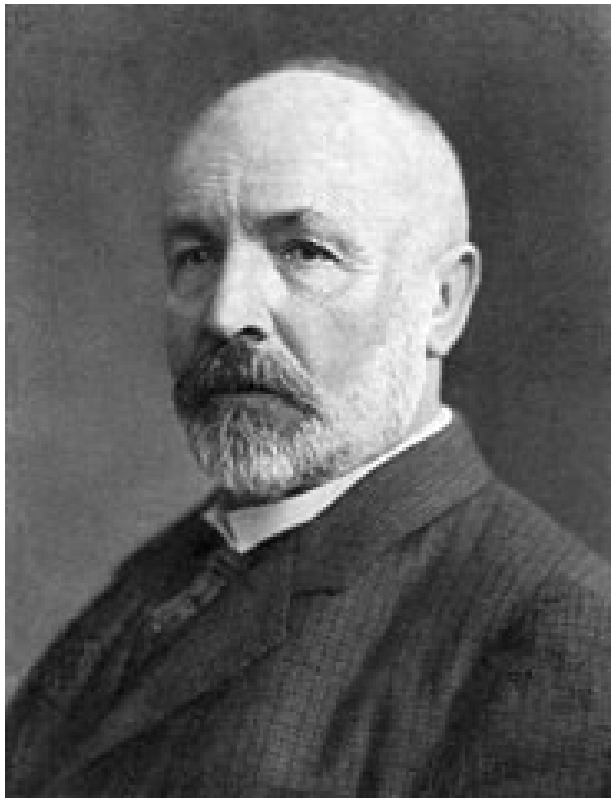


Figure 1.1: Georg Cantor defined the transfinite numbers by the end of the XIX century.

Practice 1.2.1. Get the following tools:

i. a Linux distribution

   — install a native distro (preferred)

   — install on a VM (check the one at VC)

ii. a LaTeX compiler

   — install `texlive-full` in your OS

   — install `gummi`

   — use an online editor like `Overleaf`

iii. an `Octave` compiler

   — install `Octave` in your OS

   — use an online compiler like `Octave-online`

Practice 1.2.2. Perform this tasks:

i. follow this link, edit the LaTeX code, and click on the `TeXLive.net` button for rendering.

ii. in `software/math/` try the `Octave` commands

```
octave:1> help powerrelation
octave:2> powerrelation({"12", "23", "34", "45"}, 2);
octave:3> unionrelation({["a", "b"], ["c", "c"]}, {["b", "a"]});
```

Practice 1.2.3. Given $A = \{1, 2, 3\}$, propose

i. a binary relation on A

ii. a relation between A and $\mathbb{N}$

iii. a function on A

iv. a function between A and $\mathbb{N}$.

Practice 1.2.4. Given $R = \{(1, 2), (2, 2), (3, 4), (2, 1)\}$,

i. is R antisymmetric?

ii. is R transitive?

iii. find the symmetric closure of R.

1.3 Practices of chapter 1

Practice 1.3.1. Given $R = \{(1,2),(2,3),(3,4),(4,5)\}$, solve

i. R^2

ii. R^3

iii. R^{-1}

iv. $R^{2^{-1}}$

v. what can you conclude from the last result?

Practice 1.3.2. Answer these questions:

i. is countable the set of the odd numbers?

ii. is countable the set of the prime numbers?

iii. is countable $\mathbb{Q}$?

Chapter 2

Languages and grammars

2.1 Languages 28
2.2 Grammatical representation systems 36
2.3 Questions about languages 44
2.4 Operations and closures of languages 45
2.5 Closures for the types of languages 48
2.6 Practices of chapter 2 51

In fact, by universal grammar, I mean just that system of principles and structures that are the prerequisites for acquisition of language, and to which every language necessarily conforms.

Noam Chomsky

2.1 Languages

2.1.1 Concept of language

Definition 2.1.1. Alphabet (*alfabeto*)
A non empty, finite set of symbols.

$$\Sigma = \{s_1, \ldots, s_k\}, k \geq 1$$

Definition 2.1.2. String over an alphabet (*cadena sobre un alfabeto*)
A finite sequence of symbols taken from a given alphabet,

$$w = (a_1, \ldots, a_n), a_i \in \Sigma$$

Remark. Defined as a vector (ordered set), a string (or word) will be represented by the sequence of symbols: $w = a_1 \ldots a_n$.

Definition 2.1.3. Empty string (*cadena vacía*)

$$\varepsilon = (\ )$$

Definition 2.1.4. Length of a string

$$|w| = \begin{cases} 0 & \text{if } w = \varepsilon \\ n \in \mathbb{N} & \text{if } w = (a_1, \ldots, a_n) \end{cases}$$

Definition 2.1.5. Occurrence (*ocurrencia*)
Given $w = (a_1, \ldots, a_n)$, and $1 \leq j \leq n$,

$$w(j) = a_j$$

Definition 2.1.6. Number of occurrences (*número de ocurrencias*)

$$|w|_a = |A| : w(j) = a,\ \forall j \in A \subset \mathbb{N}$$

Remark. $|\varepsilon|_a = 0,\ \forall a \in \Sigma$

Definition 2.1.7. String concatenation (*concatenación de cadenas*)
$x \cdot y$ or simply xy is the string verifying

$$|xy| = |x| + |y|$$

$$xy(j) = \begin{cases} x(j) & \text{if } 0 < j \leq |x| \\ y\,(j - |x|) & \text{if } |x| < j \leq |xy| \end{cases}$$

Definition 2.1.8. Σ^+, Σ^* (*sigma más, sigma estrella*)
Σ^+ is the strict closure of Σ under string concatenation, and Σ^* is its wide closure:
$\Sigma^+ = \{w = a_1 \ldots a_k \ :\ a_i \in \Sigma \ \wedge \ k \geq 1\}$
$\Sigma^* = \Sigma^+ \cup \{\varepsilon\}$

Remark. $\cdot$ is associative and ε is its neutral element, hence $(\Sigma^*, \cdot\,)$ is a monoid.

Definition 2.1.9. Prefix (*prefijo*)

$$v \sqsubseteq w \text{ iff } \exists x \,:\, w = vx.$$

Definition 2.1.10. Suffix (*sufijo*)

$$v \sqsupseteq w \text{ iff } \exists x \,:\, w = xv.$$

Definition 2.1.11. Substring (*subcadena*)

$$v \prec w \text{ iff } \exists x, y \,:\, w = xvy.$$

Remark. ε is prefix, suffix and a substring of any string.

Definition 2.1.12. Power of a string (*potencia de una cadena*)

$$w^n = \begin{cases} \varepsilon & \text{if } n = 0 \\ w^{n-1}w & \text{if } n > 0 \end{cases}$$

Definition 2.1.13. Reverse string (*cadena inversa*)

$$w^R = \begin{cases} \varepsilon & \text{if } w = \varepsilon \\ au^R & \text{if } w = ua,\ u \in \Sigma^*, a \in \Sigma \end{cases}$$

Proposition 2.1.1. $\forall x, w \in \Sigma^*$, $(wx)^R = x^R w^R$

Definition 2.1.14. Language over an alphabet

$$L \text{ is a language over } \Sigma \text{ iff } L \subseteq \Sigma^*$$

If a language is finite it can be defined by *extension* (e.g. $L = \{ac, bca, caaaad\}$). If a language is infinite it can only be defined by *compresion* as

$$L = \{w \in \Sigma^* \ : \ P(w)\}$$

where P is a property verified by all the strings in L.

Proposition 2.1.2. $|\Sigma^*| = \aleph_0$

Proof. This means that $\Sigma^* \sim \mathbb{N}$. To prove it, lets assume an order on $\Sigma = \{a_1, \ldots, a_n\}$, then all strings of Σ^* can be enumerated, first by length and secondly by alphabetical order:

length	*strings*		
0	ε		
1	a_1	a_2	...
2	a_1a_1	a_1a_2	...
3	...		

so a bijective function can be defined with $\mathbb{N}$. $\square$

Example 2.1.1. On the use of alphabets, strings and its operations, and languages

$\Sigma = \{a, b, c\}$

$v = bbbcbbba$	$\lvert v \rvert = 8$
$w = cbbca$	$\lvert w \rvert = 5$
$x = vw = bbbcbbbacbbca$	
$x(4) = c$	$\lvert x \rvert_b = 8$
$x(9) = c$	$\lvert x \rvert_c = 3$
$bbbc \sqsubseteq x$	$bacb \prec x$
$acbbca \sqsupseteq x$	$acbbca \prec x$
$w^3 = cbbcacbbcacbbca$	
$w^R = acbbc$	
$L = \{\varepsilon, b, acc, ababc\}$	$\lvert L \rvert = 4$
$L = \{w \in \Sigma^* \ : \ abc \sqsubseteq w\}$	$\lvert L \rvert = \aleph_0$
$L = \{w \in \Sigma^* \ : \ \lvert w \rvert_a = 3n \in \mathbb{N}_0\}$	$\lvert L \rvert = \aleph_0$

Definition 2.1.15. Representation system
Let Σ_M be a meta-alphabet (i.e. the alphabet of the natural language, including all the mathematical symbols)

$$\Sigma_M = \{A, \ldots, Z, a, \ldots, z, 0, \ldots, 9, \forall, \exists, \Rightarrow, \wedge, \vee, ...\}$$

A representation system is a function from strings over a meta-alphabet (Σ_M) to languages over an alphabet (Σ), namely

$$\mathscr{R} : \Sigma_M^* \to \mathscr{P}(\Sigma^*)$$

Remark. $\mathscr{R}$ establishes an unambiguous correspondence of strings and languages:

$$(r, L) \in \mathscr{R} \Rightarrow \nexists (r, L') \in \mathscr{R} : L' \in \mathscr{P}(\Sigma^*) \wedge L' \neq L$$

Definition 2.1.16. Set of the representable languages, $\mathscr{L}$.REP

$$\mathscr{L}.\text{REP} = \{L \in \mathscr{P}(\Sigma^*) \ : \ \exists \mathscr{R} \wedge \exists (r, L) \in \mathscr{R}\}$$

Remark.

Σ	is a finite set
Σ^*	is a countable infinite set
L	is a countable set (either finite or countable infinite)
$\mathscr{L}$.REP	is a countable infinite set
$\mathscr{P}(\Sigma^*)$	is an uncountable set

Proposition 2.1.3. There are uncountable non representable languages

Proof. The set of all language representations for a given $\mathscr{R}$ will be REP $\subseteq \Sigma_M^*$, and each representation can only represent one language (which is then representable), so

$$|\mathscr{L}.\text{REP}| \leq |\text{REP}|$$

and since $|\Sigma_M^*| = \aleph_0$,

$$|\text{REP}| \leq \aleph_0$$

Also, every finte language is representable by extension, and since there is a countable infinite number of finite languages

$$\aleph_0 \leq |\mathscr{L}.\text{REP}|$$

and then

$$\aleph_0 \leq |\mathscr{L}.\text{REP}| \leq |\text{REP}| \leq \aleph_0 \Rightarrow |\mathscr{L}.\text{REP}| = |\text{REP}| = \aleph_0$$

By definition, $L \subseteq \Sigma^*$ and the set of all possible languages over Σ is $\mathscr{P}(\Sigma^*)$, then

$$|\Sigma^*| = \aleph_0 \Rightarrow |\mathscr{P}(\Sigma^*)| = \aleph_1$$

The set of non representable languages is formed by the languages not in $\mathscr{L}$.REP.

$$\mathscr{L}.\text{NOREP} = \mathscr{P}(\Sigma^*) - \mathscr{L}.\text{REP}$$

And given the cardinality of $\mathscr{P}(\Sigma^*)$ and $\mathscr{L}$.REP,

$$\aleph_1 - \aleph_0 = \aleph_1 \Rightarrow |\mathscr{L}.\text{NOREP}| = \aleph_1$$

$\square$

2.1.2 Language representation

Definition 2.1.17. Conclusive algorithm (*algoritmo conclusivo*)
Finite sequence of finite, precise, unambiguous instructions, that, in a finite time, returns an output for a given input.

Definition 2.1.18. Language recognizer (*reconocededor de lenguajes*)
A conclusive algorithm designed for a given language L that determines whether or not a string belongs to L.

Definition 2.1.19. Language generator (*generador de lenguajes*)
A system of conclusive algorithms that are able to produce all, and only the strings of a given language L.

2.2 Grammatical representation systems

2.2.1 Definition and functioning of grammars

Definition 2.2.1. Grammar
A grammar is a language generator defined as a quadruple $G = (N, T, P, S)$ where

— N is the *non-terminal alphabet* (non terminals are capital Latin letters)

— T is the *terminal alphabet* (terminals are lower case Latin letters)
$N \cap T = \emptyset$
$N \cup T = V$

— $P \subset V^+ \times V^*$ is the *production system* or *ruleset* and it is finite
$(\alpha, \beta) \in P$ is a production rule, noted $\alpha \to \beta$

— $S \in N$ is the *axiom*, or *starting symbol*, of the grammar.

Definition 2.2.2. Produce directly (*producir directamente*)
Given $G = (N, T, P, S)$, $x \in V^+$ and $y \in V^*$, then x produces y directly, $x \Rightarrow y$, if $\exists u, v \in V^*$ satisfying
$x = u\alpha v$
$y = u\beta v$
$\exists \alpha \to \beta \in P$

Definition 2.2.3. Produce in n steps (*producir en n pasos*)
x produces y in 0 steps: $x \Rightarrow^0 y$ iff $x = y$
x produces y in 1 steps: $x \Rightarrow^1 y$ iff $x \Rightarrow y$
x produces y in n steps: $x \Rightarrow^n y$ iff

$$\exists z_1, z_2, \ldots, z_{n-1} \in V^+ \,:\, x \Rightarrow z_1, z_1 \Rightarrow z_2, \ldots, z_{n-1} \Rightarrow y$$

Definition 2.2.4. Produce in at least one step

$$x \Rightarrow^+ y \text{ iff } \exists n > 0 \,:\, x \Rightarrow^n y$$

Definition 2.2.5. Produce (*producir*)

$$x \Rightarrow^* y \text{ iff } \exists n \geq 0 \,:\, x \Rightarrow^n y$$

$\Rightarrow$ is a binary relation over the strings in V^*
$\Rightarrow^+$ is its transitive closure
$\Rightarrow^*$ is its reflexive and transitive closure

Definition 2.2.6. Derivation, length of a derivation (*derivación*)
Given $G = (N, T, P, S)$, a derivation of G is a finite sequence of strings over V, $(w_0, w_1, w_2 \dots w_n)$, such that

$$w_0 \Rightarrow w_1 \Rightarrow w_2 \Rightarrow \cdots \Rightarrow w_n$$

where $w_0 = S$ and $n \in \mathbb{N}_0$ is the length of the derivation.

Definition 2.2.7. Sentential form (*forma sentencial*)
$\alpha \in V^*$ is a sentential form of a grammar G iff

$$S \Rightarrow^* \alpha$$

Definition 2.2.8. String generated by a grammar
$y \in T^*$ is generated by G iff

$$S \Rightarrow^* y$$

Definition 2.2.9. Language generated by a grammar
$G = (N, T, P, S)$ generates the language

$$\mathcal{L}(G) = \{y \in T^* \ : \ S \Rightarrow^* y\}$$

Example 2.2.1. Grammatical representation of the language with all the strings with an odd length (i.e. $\{w \in \{@\}^* \ : \ |w| \mod 2 \neq 0\}$). Compare these rules with those in the ‘evenlength‘ grammar (see the ‘grammars.json‘ file).

```
octave> produce('oddlength', 4);
(
  {A, B},
  {@},
  {
    A → @
    A → @B
    B → @A
  },
  A
)

A => @

A => @B => @@A => @@@

A => @B => @@A => @@@B => @@@@A => @@@@@

A => @B => @@A => @@@B => @@@@A => @@@@@B => @@@@@@A => @@@@@@@
```

2.2.2 Classification of rules

A production rule will be classified as type n if it meets, at least, the constraints up to type n (e.g., if a rule satisfies the constraints of type 1, but not those of type 2, then it is classified as type 1, even if it also fits -and it does- with type 0).

Definition 2.2.10. Type 0 (phrase structure) rule (*regla con estructura de frase*)

$$\alpha \to \beta$$

with $\alpha \in V^+, \beta \in V^*$.

Remark. Every rule is type 0.

Definition 2.2.11. Type 1 (context-sensitive) rule (*regla sensible al contexto*)

$$\alpha A \beta \to \alpha \gamma \beta$$

with $\alpha, \beta \in V^*, \gamma \in V^+, A \in N$.

Remark. α and β constitute the *context*.

Definition 2.2.12. Type 2 (context free) rule (*regla independiente del contexto*)

$$A \to \alpha$$

with $\alpha \in V^+, A \in N$.

Definition 2.2.13. Left-regular rule (*regla regular izquierda*)

$$A \to aB$$

Definition 2.2.14. Right-regular rule (*regla regular derecha*)

$$A \to Ba$$

Definition 2.2.15. Terminal-regular rule (*regla regular terminal*)

$$A \to a$$

Definition 2.2.16. Type 3 (or regular) rule (*regla regular*)
A rule is regular iff it is left-regular, or it is right-regular, or it is terminal-regular.

Proposition 2.2.1. Rules hierarchy

$$\text{Type 3 rule} \subset \text{Type 2 rule} \subset \text{Type 1 rule} \subset \text{Type 0 rule}$$

Definition 2.2.17. Epsilon rule

$$A \to \varepsilon$$

2.2.3 Classification of grammars

Definition 2.2.18. Type 0 (phrase structure) grammar
G is type 0 iff r is type 0, $\forall r \in P$.

Remark. Every grammar is type 0.

Definition 2.2.19. Type 1 (context-sensitive) grammar
G is type 1 iff r is type 1, $\forall r \in P$.

Definition 2.2.20. Type 2 (context free) grammar
G is type 2 iff r is type 2, $\forall r \in P$.

Definition 2.2.21. Left-regular type grammar
G is left-regular type iff r is left-regular or terminal-regular type, $\forall r \in P$.

Definition 2.2.22. Right-regular type grammar
G is right-regular type iff r is right-regular or terminal-regular type, $\forall r \in P$.

Definition 2.2.23. Type 3 (regular) grammar
G is a regular grammar iff it is a left-regular grammar or a right-regular grammar.

Proposition 2.2.2. Grammars hierarchy

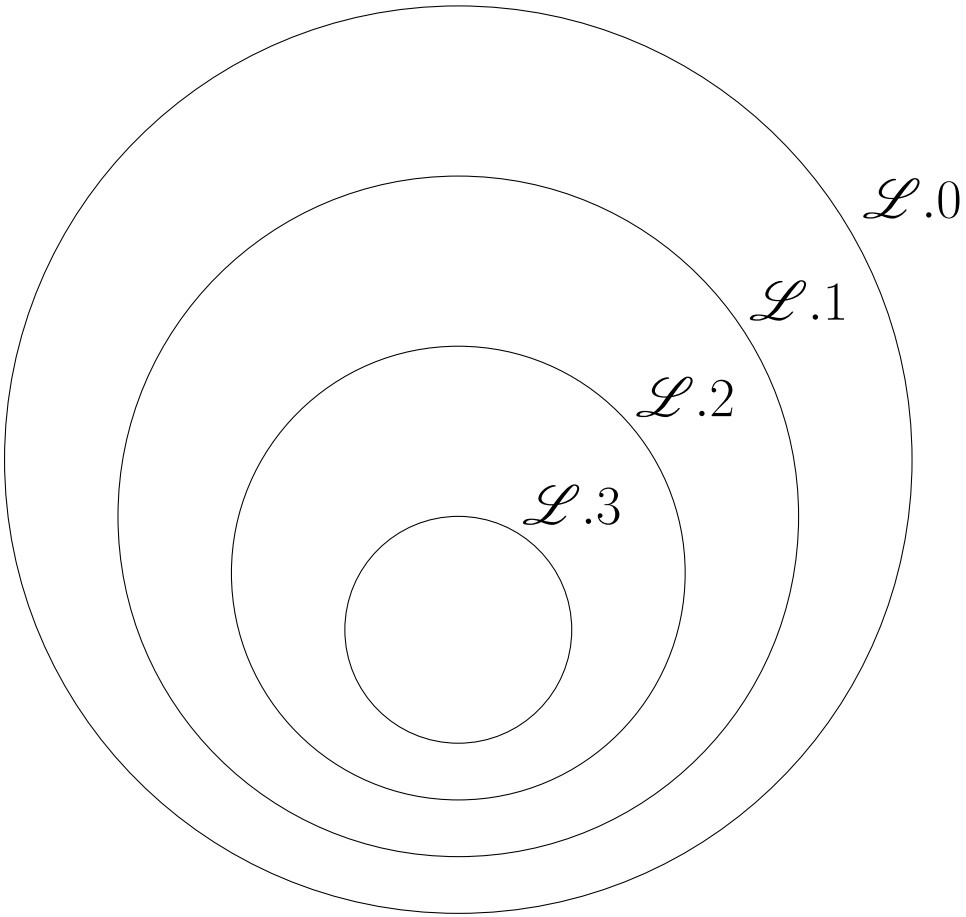
$$\text{Type 3 grammar} \subset \text{Type 2 grammar} \subset \text{Type 1 grammar} \subset \text{Type 0 grammar}$$

2.2.4 Classification of languages

Definition 2.2.24. Type i ($\mathscr{L}.i$) language
A language L is type $i \in \{0 \ldots 3\}$ ($L \in \mathscr{L}.i$) if there exists a type i grammar such that $\mathcal{L}(G) = L - \{\varepsilon\}$.

Proposition 2.2.3. Chomsky's formal languages hierarchy

$$\mathscr{L}.3 \subset \mathscr{L}.2 \subset \mathscr{L}.1 \subset \mathscr{L}.0$$



2.3 Questions about languages

Definition 2.3.1. Equivalent grammars: $G_1 \equiv G_2 \Rightarrow \mathcal{L}(G_1) = \mathcal{L}(G_2)$

The following table shows whether or not it exists a conclusive algorithm to answer these questions according to the type of grammar:

		phrase structure	context-sensitive	context-free	regular
Equivalence	$G_1 \equiv G_2$	no	no	no	yes
Inclusion	$\mathcal{L}(G_1) \subseteq \mathcal{L}(G_2)$	no	no	no	yes
Membership	$x \in \mathcal{L}(G)$	no	yes	yes	yes
Emptiness	$\mathcal{L}(G) = \emptyset$	no	no	yes	yes
Finiteness	$\lvert\mathcal{L}(G)\rvert \in \mathbb{N}$	no	no	yes	yes
Regularity	$\mathcal{L}(G) \in \mathscr{L}.3$	no	no	no	yes

2.4 Operations and closures of languages

Since languages are sets of strings, regular sets operations like union, intersection, difference and complement are also aplicable to them. Besides those mentioned, there are some other operations that are specific to languages, like the following.

Definition 2.4.1. Concatenation of languages

$$L_1 \cdot L_2 = \{xy \in \Sigma^* \,:\, x \in L_1, y \in L_2\}$$

where $L_1, L_2 \subseteq \Sigma^*$.

Remark. Sometimes the symbol '$\cdot$' might be omitted, leaving $L_1 L_2$.

Language concatenation verifies these properties:

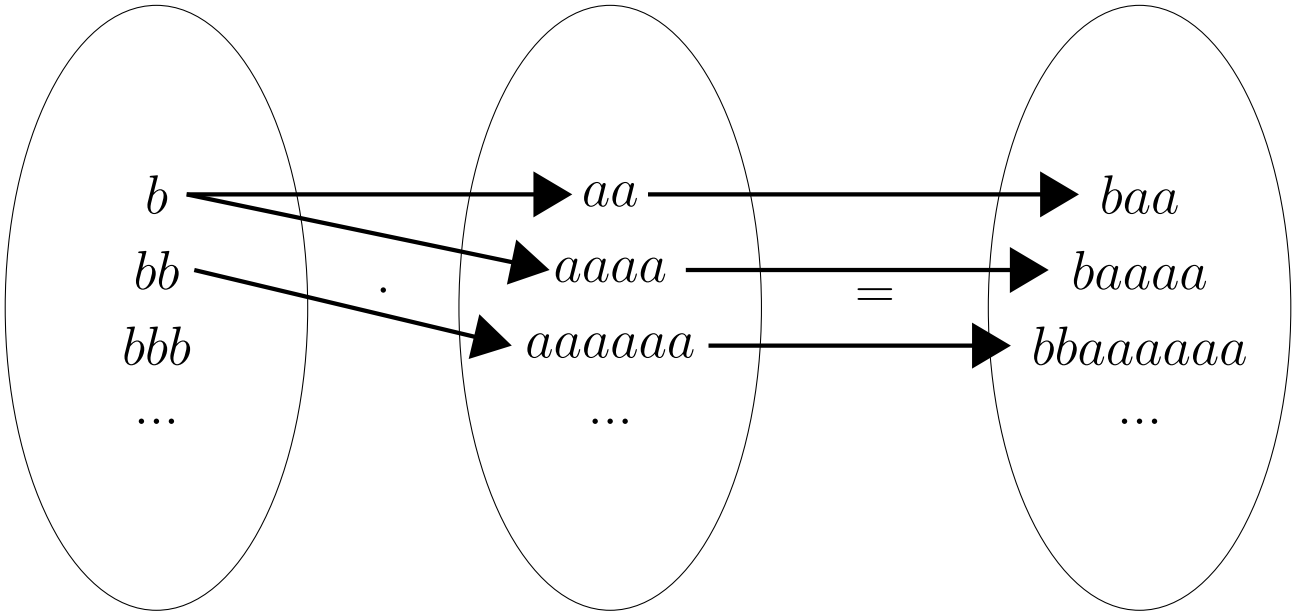
$$L \cdot \emptyset = \emptyset \cdot L = \emptyset$$

$$L \cdot \{\varepsilon\} = \{\varepsilon\} \cdot L = L$$

$$L_1 \cdot (L_2 \cdot L_3) = (L_1 \cdot L_2) \cdot L_3$$

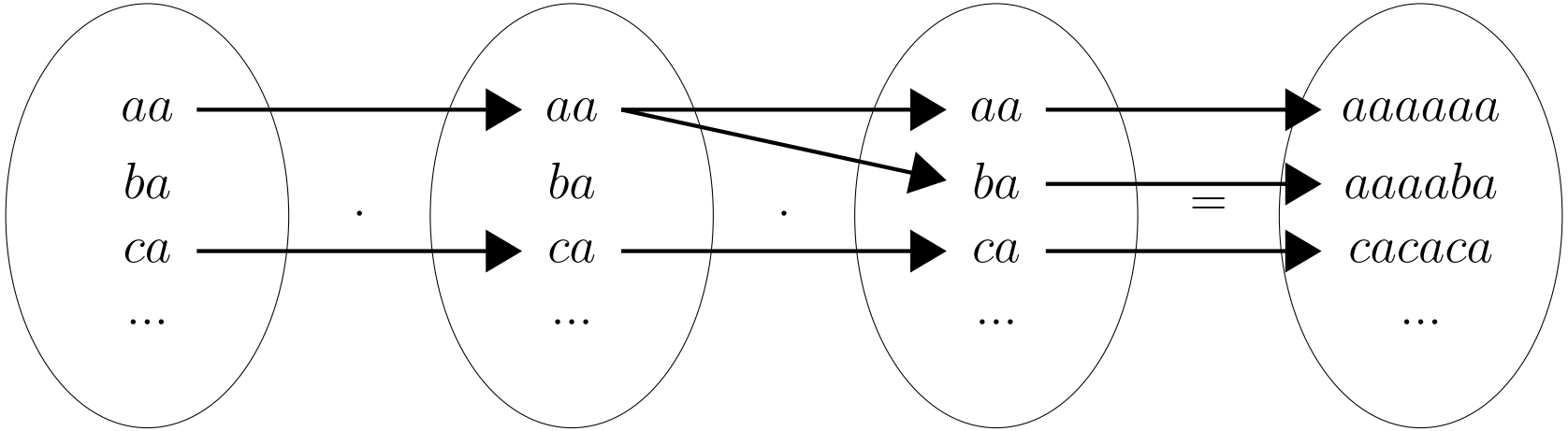
$$L_1 \cdot (L_2 \cup L_3) = L_1 \cdot L_2 \cup L_1 \cdot L_3$$

$$(L_1 \cup L_2) \cdot L_3 = L_1 \cdot L_3 \cup L_2 \cdot L_3$$


b
bb
bbb
...
.
aa
aaaa
aaaaaa
...
=
baa
baaaa
bbaaaaaa
...

Definition 2.4.2. Power of a language (*potencia de un lenguaje*)

$$L^n = \begin{cases} \{\varepsilon\} & \text{if } n = 0 \\ L^{n-1} \cdot L & \textit{otherwise} \end{cases}$$



Definition 2.4.3. Kleene closure (Kleene star) (*cierre o estrella de Kleene*)

$$L^* = \bigcup_{i \in \mathbb{N}_0} L^i$$

Remark. $\emptyset^* = \{\varepsilon\}$

Definition 2.4.4. Language reversal (*inverso de un lenguaje*)

$$L^R = \{x^R \,:\, x \in L\}$$

2.5 Closures for the types of languages

The following table shows whether or not each languaje type is closed under a certain operation[1]:

	phrase structure	context-sensitive	context-free	regular
Union	yes	yes	yes	yes
Intersection	yes	yes	no	yes
Intersection with $\mathscr{L}.3$	yes	yes	yes	yes
Complement	no	yes	no	yes
Concatenation	yes	yes	yes	yes
Power	yes	yes	yes	yes
Kleene closure	yes	yes	yes	yes
Reverse	yes	yes	yes	yes

Proposition 2.5.1. Language types hierarchy

$$\mathscr{L}.\text{finite} \subset \mathscr{L}.3 \subset \mathscr{L}.2 \subset \mathscr{L}.1 \subset \mathscr{L}.0 \subset \mathscr{L}.\text{REP} \subset \mathscr{P}(\Sigma^*)$$

Remark. $\mathscr{L}.0$ not being complement-closed $\Rightarrow \mathscr{L}.0 \neq \mathscr{L}.\text{REP}$.

[1] See .2 and .3 for proofs and examples with $\mathscr{L}.3$.

Example 2.5.1. Lindenmayer systems to model biological development.

```
octave> developlsystem('creeping_plant', 3);
octave> developlsystem('climbing_plant', 3);
```

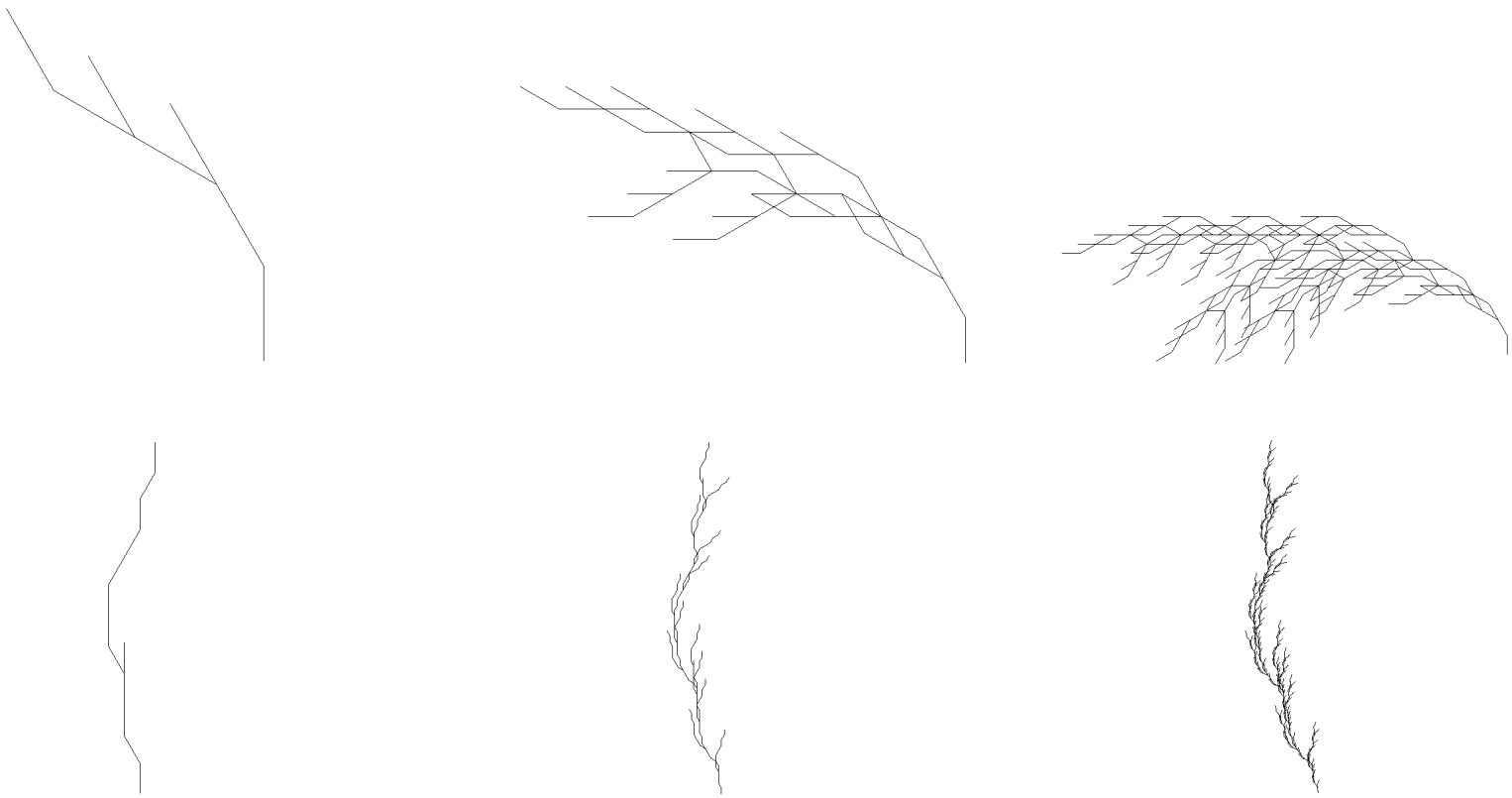


Figure 2.1: Graphical representation of L-systems. Iterations 1 to 3 from left to right.

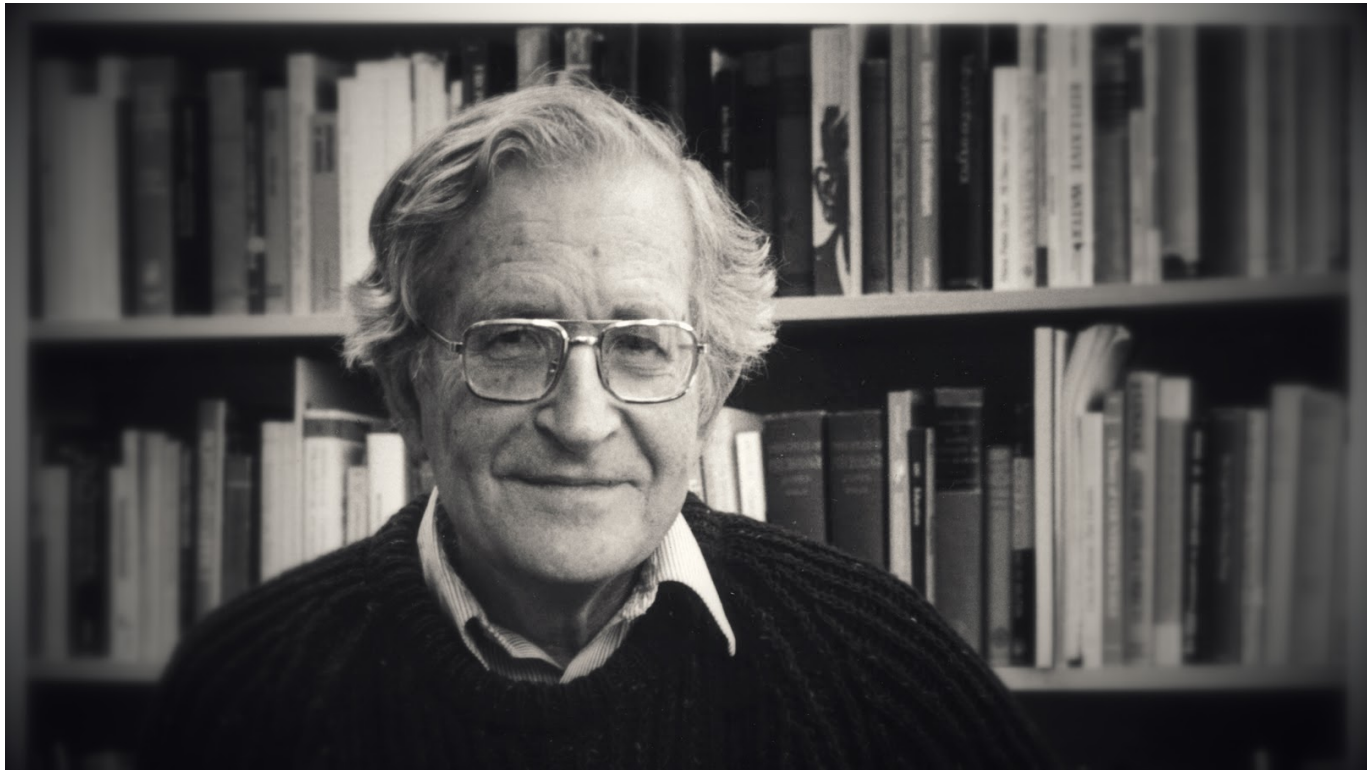


Figure 2.2: Noam Chomsky proposed the generative grammar in 1965.

2.6 Practices of chapter 2

Practice 2.6.1. Install repo and run scripts.
Clone the `talfuma` repo with

```
$ mkdir opt
$ cd opt
$ git clone https://bitbucket.org/umafoss/talfuma.git
$ cd talfuma
```

Run the `GNU Octave` terminal with

```
$ octave -q --no-gui
```

or for the `Flatpak`-installed version in the VM:

```
$ flatpak run org.octave.Octave -q --no-gui
```

Get to the `software/grammars/` folder with

```
octave:1> cd software/grammars
octave:2> pwd
```

and execute this commands:

```
octave:3> help produce
octave:4> type grammars.json
```

Practice 2.6.2. Produce strings with different grammars:

```
octave:5> produce("N_0", 5, 3);
octave:6> produce("N_0", 5);
octave:7> produce("R01", 5, 5);
octave:8> produce("R01", 5);
```

Practice 2.6.3. Grammatically represent a language

Represent the language 0*+(0*10*10*10*)* with a grammar.

Include this grammar in `software/grammar/grammars.json`.

Produce strings of the language with the scripts.

Chapter 3

Regular expressions

3.1 Definition of the regular expressions 54
3.2 Some properties of regular expressions 61
3.3 Practices of chapter 3 64

Some people, when confronted with a problem, think "I know, I'll use regular expressions." Now they have two problems.

Jamie Zawinski

3.1 Definition of the regular expressions

Regular expressions constitute a language representation method, that can only represent regular languages.

Definition 3.1.1. Set of regular expressions ($\mathcal{R}$)
Given $\Sigma_{\mathcal{R}} = \Sigma \cup \{), (, \varnothing, +, ^*\}$

$$\mathcal{R} \subseteq \Sigma_{\mathcal{R}}^*$$

verifying

1. $\varnothing \in \mathcal{R}$
2. $a \in \mathcal{R},\ \forall a \in \Sigma$
3. $(\alpha\beta) \in \mathcal{R},\ \forall \alpha, \beta \in \mathcal{R}$
4. $(\alpha + \beta) \in \mathcal{R},\ \forall \alpha, \beta \in \mathcal{R}$
5. $\alpha^* \in \mathcal{R},\ \forall \alpha \in \mathcal{R}$
6. No other string over $\Sigma_{\mathcal{R}}$ (that cannot be generated with the previous rules) is an element of $\mathcal{R}$.

Definition 3.1.2. Language defined by a regular expression

$$\begin{aligned} \mathcal{L} : \mathcal{R} &\to \mathscr{P}(\Sigma^*) \\ r &\to \mathcal{L}(r) \end{aligned}$$

verifying

1. $\mathcal{L}(\varnothing) = \varnothing$
2. $\mathcal{L}(a) = \{a\},\ \forall a \in \Sigma$
3. $\mathcal{L}((\alpha\beta)) = \mathcal{L}(\alpha) \cdot \mathcal{L}(\beta),\ \forall \alpha, \beta \in \mathcal{R}$
4. $\mathcal{L}((\alpha + \beta)) = \mathcal{L}(\alpha) \cup \mathcal{L}(\beta),\ \forall \alpha, \beta \in \mathcal{R}$
5. $\mathcal{L}(\alpha^*) = \mathcal{L}(\alpha)^*,\ \forall \alpha \in \mathcal{R}$

Remark.

$\cup, \cdot$ and * are the operations of union, concatenation and Kleene's star for languages.

From now on, we will assume that a regular expression is, actually, the language that it represents, i.e. $(10)^* = \mathcal{L}((10)^*) = \{\varepsilon, 10, 1010, \dots\}$

Notational conventions

1. First and last parentheses of a regular expression can be removed.

2. Pairs of parentheses can be replaced by square brackets for readability.

3. To avoid cluttering of parenthesis, we adopt these conventions:

   Order of operations (*orden de operaciones*): $^* > \cdot > +$
   $(R + ((S^*)T)) = R + S^*T$

   Associativity:
   $(R + (S + T)) = ((R + S) + T) = R + S + T$
   $(R(ST)) = ((RS)T) = RST$

4. The meta-alphabet is extended with the symbol ε, $\Sigma_{\mathcal{R}} = \Sigma \cup \{), (, \varnothing, +, ^*, \varepsilon\}$, representing the set with the empty string, $\mathcal{L}(\varepsilon) = \{\varepsilon\}$.

Example 3.1.1. Languages and their representation with regular expressions

Strings that have 001 as a substring
$(0+1)^*001(0+1)^*$

Strings where the number of 1s is a multiple of 3
$0^* + (0^*10^*10^*10^*)^*$

Strings that consist of alternating 0s and 1s
$(10)^* + (01)^* + 0(10)^* + 1(01)^* = (\varepsilon + 1)(01)^*(\varepsilon + 0)$

Strings that do not have two consecutive 0s
$(0+\varepsilon)(1+10)^*$

Example 3.1.2. Regular expression for strings with an odd number of symbols.

```
octave> re('@ @@ @@@ @@@@ @@@@@ @@@@@@', '@(@@)*');
  @     ∈ @(@@)*
  @@@   ∈ @(@@)*
  @@@@@ ∈ @(@@)*
```

Example 3.1.3. Regular expression for strings with an even number of symbols.

```
octave> re('@ @@ @@@ @@@@ @@@@@ @@@@@@', '(@@)*');
  @@     ∈ (@@)*
  @@@@   ∈ (@@)*
  @@@@@@ ∈ (@@)*
```

Example 3.1.4. Regular expression for strings with alternating zeroes and ones.

```
octave> re('10101 0010 10 010 111', '(10)*+(01)*+0(10)*+1(01)*');
  10101 ∈ (10)*+(01)*+0(10)*+1(01)*
  10    ∈ (10)*+(01)*+0(10)*+1(01)*
  010   ∈ (10)*+(01)*+0(10)*+1(01)*
```

Example 3.1.5. The first 15 regular expressions in an enumeration of $\mathcal{R}$.

```
octave> enumerateR('01', 14);
     0   ∅
     1   0
     2   1
     3   (∅∅)
     4   (∅+∅)
     5   ∅*
     6   (0∅)
     7   (0+∅)
     8   0*
     9   (∅0)
    10   (∅+0)
    11   1*
    12   (1∅)
    13   (1+∅)
    14   (∅∅)*
```

Example 3.1.6. A more elaborated regular expression (index 10^7).

```
octave> enumerateR('01', 10000000, 'index');
10000000   (((1+0)(∅+∅))+((00)+(1+∅)))
```

3.2 Some properties of regular expressions

Any $\alpha,\ \beta,\ \gamma \in \mathcal{R}$ verify the following properties:

1) $\alpha + \varnothing = \alpha$

2) $\alpha + \alpha = \alpha$

3) $\alpha + \beta = \beta + \alpha$

4) $\alpha + (\beta + \gamma) = (\alpha + \beta) + \gamma$

5) $\alpha(\beta\gamma) = (\alpha\beta)\gamma$

6) $\alpha\varepsilon = \varepsilon\alpha = \alpha$

7) $(\alpha + \beta)\gamma = \alpha\gamma + \beta\gamma$

8) $\gamma(\alpha + \beta) = \gamma\alpha + \gamma\beta$

9) $\alpha^*\alpha^* = \alpha^*$

10) $(\alpha^*)^* = \alpha^*$

11) $\alpha\alpha^* = \alpha^*\alpha$

12) $\alpha^* = \varepsilon + \alpha + \alpha^2 + \alpha^3 + ... + \alpha^k + \alpha^{k+1}\alpha^* \quad \forall k \geq 0$

13) $\alpha^* = \varepsilon + \alpha\alpha^*$

14) $(\alpha^* + \beta^*)^* = (\alpha + \beta)^*$

15) $(\alpha^*\beta^*)^* = (\alpha + \beta)^*$

16) $(\alpha\beta)^*\alpha = \alpha(\beta\alpha)^*$

17) $(\alpha^*\beta)^*\alpha^* = (\alpha + \beta)^*$

18) $\alpha^*(\beta\alpha^*)^* = (\alpha + \beta)^*$

19) $(\alpha^*\beta)^* = (\alpha + \beta)^*\beta + \varepsilon$

20) $f(\alpha_1, \alpha_2, ..., \alpha_n) \subseteq (\alpha_1 + \alpha_2 + ... + \alpha_n)^*$ where the images obtained by f are combinations of $\alpha_1, \alpha_2, .., \alpha_{n-1}$ and α_n, using only the operations of union $(+)$, concatenation $(\cdot)$ and Kleene star $(^*)$.

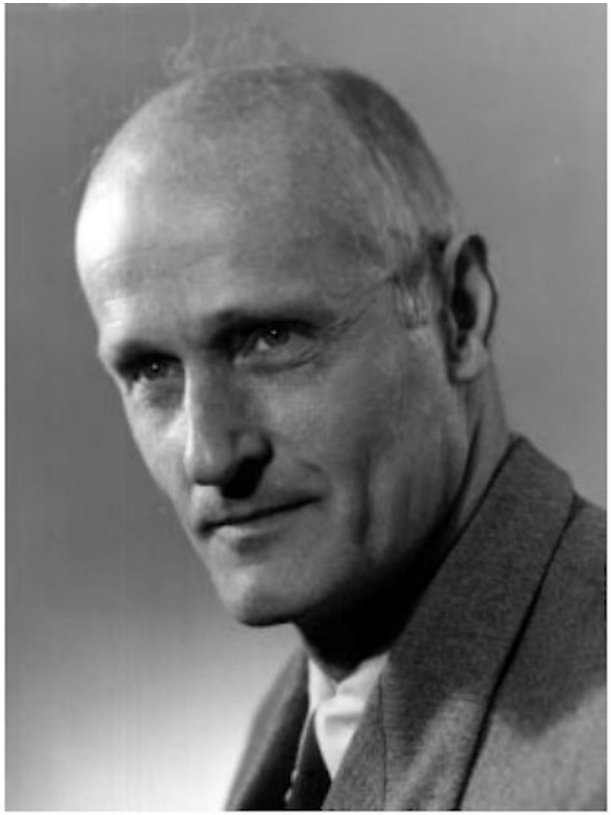


Figure 3.1: Stephen Kleene proposed the regular expressions model in 1951.

3.3 Practices of chapter 3

Practice 3.3.1. Find words in a text file

In a terminal, change to directory `software/regularexpressions`.

Find and count the occurrences of `Open` in the file `guerrillamanifesto.txt`.

Find the occurrences of the string `Open`, followed by any word, followed by `Movement`.

Practice 3.3.2. Find strings in a text file

Find and count the occurrences of `AA`, followed by three characters and then `CC`, in the file `dnasequence.txt`.

Practice 3.3.3. Find strings of a language

With `re.m`, tell what strings of $\{101, 001100, 00, 101101\}$ belong to the language 10*1*0*1.

Chapter 4

Finite automata

4.1 Deterministic finite automata 66
4.2 Nondeterministic finite automata 73
4.3 Minimum deterministic finite automaton 80
4.4 Equivalences 80
4.5 Practices of chapter 4 81

4.1 Deterministic finite automata

Definition 4.1.1. Deterministic finite automaton (*autómata finito determinista*)
A deterministic finite automaton (DFA) is a 5-tuple $(K, \Sigma, \delta, s, F)$, where

K is a non-empty set of states

Σ is an alphabet

$s \in K$ is the initial state

$F \subseteq K$ is a set of final states

$\delta : K \times \Sigma \to K$ is the transition function

Definition 4.1.2. Configuration of a DFA (*configuración*)
A configuration of $(K, \Sigma, \delta, s, F)$ is any $(q, w) \in K \times \Sigma^*$.

Remark. q represents the current state of the DFA, and w is the string that is still to be processed by the DFA.

Definition 4.1.3. Initial configuration of a DFA (*configuración inicial*)
An initial configuration of $(K, \Sigma, \delta, s, F)$ is any pair (s, w), with $w \in \Sigma^*$.

Definition 4.1.4. Terminal configuration of a DFA (*configuración final*)
A terminal configuration of $(K, \Sigma, \delta, s, F)$ is any pair (q, ε), with $q \in K$.

Definition 4.1.5. Direct transition of a DFA (*transición directa*)
$(K, \Sigma, \delta, s, F)$ performs a direct transition from configuration (q, w) to (q', w'), noted $(q, w) \vdash (q', w')$, iff

$$\exists \sigma \in \Sigma \,:\, (w = \sigma w') \wedge (\delta(q, \sigma) = q')$$

Remark. A direct transition of a DFA is the function

$$\begin{aligned} \vdash \colon K \times \Sigma^+ &\to K \times \Sigma^* \\ (q, \sigma w') &\to (\delta(q, \sigma), w') \end{aligned}$$

Definition 4.1.6. Transiting in n steps (*transitar*)
Let (q, w) and (q', w') be configurations of a DFA.
We say that (q, w) transits in n steps to (q', w'), noted $(q, w) \vdash^n (q', w')$, with $n > 1$, iff $\exists\, C_1, C_2, \ldots, C_{n-1}$ configurations of that DFA such that

$$(q, w) \vdash C_1,\ C_1 \vdash C_2,\ \ldots,\ C_{n-1} \vdash (q', w')$$

We say that (q, w) transits in 1 step to (q', w'), noted $(q, w) \vdash^1 (q', w')$, iff

$$(q, w) \vdash (q', w')$$

We say that (q, w) transits in 0 step to (q', w'), noted $(q, w) \vdash^0 (q', w')$, iff

$$(q, w) = (q', w')$$

Definition 4.1.7. Transiting in at least one step
Let (q, w) and (q', w') be configurations of a DFA.
We say that (q, w) transits in at least one step to (q', w'),
noted $(q, w) \vdash^+ (q', w')$, iff

$$\exists n > 0 \ : \ (q, w) \vdash^n (q', w')$$

Definition 4.1.8. Transiting
Let (q, w) and (q', w') be configurations of a DFA.
We say that (q, w) transits to (q', w'), noted $(q, w) \vdash^* (q', w')$, iff

$$\exists n \geq 0 \ : \ (q, w) \vdash^n (q', w')$$

Definition 4.1.9. Computation. Length of a computation
Let M be a DFA, a computation of M is any sequence of configurations $C_0, C_1, C_2, ..., C_n$ such that $C_i \vdash C_{i+1}$ $\forall i \in \{0, ..., n-1\}$ with $n \geq 0$.
It will be noted as $C_0 \vdash C_1 \vdash C_2 \vdash ... \vdash C_n$, and the length of the computation is n.

Definition 4.1.10. Well started/terminated/complete computation
A well started computation is any computation in which its first configuration is an initial configuration.
A terminated computation is any computation in which its last configuration is a terminal configuration.
A computation is complete if it is well started and terminated.

Definition 4.1.11. Inaccessible state
$q \in K$ is an inaccessible state of $(K, \Sigma, s, F, \delta)$ iff $\nexists x \in \Sigma^* : (s, x) \vdash^* (q, \varepsilon)$

Definition 4.1.12. Accepted string
Let $M = (K, \Sigma, \delta, s, F)$ and $w \in \Sigma^*$. w is accepted by M iff

$$\exists q \in F : (s, w) \vdash^* (q, \varepsilon).$$

Definition 4.1.13. Accepted language
Let $M = (K, \Sigma, \delta, s, F)$ a DFA and $w \in \Sigma^*$.

$$\mathcal{L}(M) = \{w \in \Sigma^* : w \text{ is accepted by } M\}$$

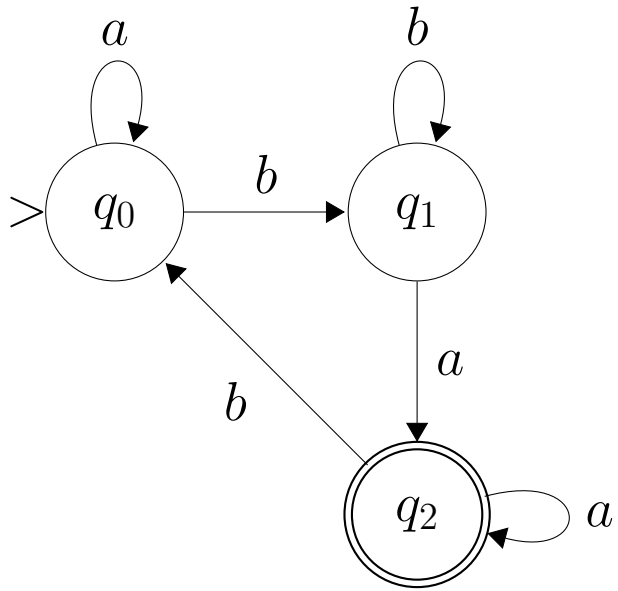
Definition 4.1.14. Set of languages accepted by DFA
The set of the languajes accepted by DFA, denoted as $\mathscr{L}(\text{DFA})$, is defined as:

$$\mathscr{L}(\text{DFA}) = \{L : \exists M \text{ DFA}, \mathcal{L}(M) = L\}$$

Definition 4.1.15. States diagram
The state diagram of a DFA is a directed graph in which each node represents a state, and each edge represents a transition for a certain input between the states the edge conects (i.e. if $\delta(q, \sigma) = q'$ then there will be an edge from q to q' labeled as σ).
Final states are represented with double circles and the initial state is marked with ">".

Example 4.1.1. Let $M = (\{q_0, q_1, q_2\}, \{a, b\}, \delta, q_0, \{q_2\})$ be a DFA with:

$\delta(q, \sigma)$	a	b
q_0	q_0	q_1
q_1	q_2	q_1
q_2	q_2	q_0



$(q_0, ab) \vdash (q_0, b) \vdash (q_1, \varepsilon) \wedge q_1 \notin F \Rightarrow ab \notin \mathcal{L}(M)$
$(q_0, abaa) \vdash (q_0, baa) \vdash (q_1, aa) \vdash (q_2, a) \vdash (q_2, \varepsilon) \wedge q_2 \in F \Rightarrow abaa \in \mathcal{L}(M)$

Example 4.1.2. DFAs for $\{w \in \{0,1\}^* \ : \ \text{even } |w|_1\}$, $\{w \in \{0,1\}^* \ : \ \text{odd } |w|_0\}$

```
octave> finiteautomaton('even number of ones', '11011');
M = ({q0, q1}, {0, 1}, {(q0, 0, q0), (q0, 1, q1), (q1, 0, q1), (q1,
1, q0)}, q0, {q0})
w = 11011
(q0, 11011) ⊢ (q1, 1011) ⊢ (q0, 011) ⊢ (q0, 11) ⊢ (q1, 1) ⊢ (q0, ε)
w ∈ ℒ(M)

octave> finiteautomaton('even number of ones', '01101');
M = ({q0, q1}, {0, 1}, {(q0, 0, q0), (q0, 1, q1), (q1, 0, q1), (q1,
1, q0)}, q0, {q0})
w = 01101
(q0, 01101) ⊢ (q0, 1101) ⊢ (q1, 101) ⊢ (q0, 01) ⊢ (q0, 1) ⊢ (q1, ε)
w ∉ ℒ(M)

octave> finiteautomaton('odd number of zeros', '00000');
M = ({q0, q1}, {0, 1}, {(q0, 0, q1), (q0, 1, q0), (q1, 0, q0), (q1,
1, q1)}, q1, {q0})
w = 00000
(q1, 00000) ⊢ (q0, 0000) ⊢ (q1, 000) ⊢ (q0, 00) ⊢ (q1, 0) ⊢ (q0, ε)
w ∈ ℒ(M)
```

4.2 Nondeterministic finite automata

Nondeterministic finite automaton can transit from a given state to none, one, or more than one state, consuming zero (ε), one, or more than one symbol.

Definition 4.2.1. Nondeterministic finite automaton
A nondeterministic finite automaton (NFA) is a 5-tuple $(K, \Sigma, \Delta, s, F)$, where

K is a non-empty set of states

Σ is an alphabet

$s \in K$ is the initial state

$F \subseteq K$ is a set of final states

$\Delta \subseteq K \times \Sigma^* \times K$ is a transition relation

Definition 4.2.2. Configuration of an NFA
A configuration of $(K, \Sigma, \Delta, s, F)$ is any $(q, w) \in K \times \Sigma^*$.

Definition 4.2.3. Initial configuration of an NFA
An initial configuration of $(K, \Sigma, \Delta, s, F)$ is any pair (s, w), with $w \in \Sigma^*$.

Definition 4.2.4. Terminal configuration of an NFA
A terminal configuration of $(K, \Sigma, \Delta, s, F)$ is any pair (q, ε), with $q \in K$.

Definition 4.2.5. Direct transition of an NFA
An NFA $(K, \Sigma, \Delta, s, F)$ performs a direct transition from configuration (q, w) to configuration (q', w'), noted $(q, w) \vdash (q', w')$, iff

$$\exists u \in \Sigma^* \,:\, (w = uw') \wedge ((q, u, q') \in \Delta)$$

Remark. Transiting directly is not a function since for some configurations (q, w) there can be more than one or none configuration (q', w') such that $(q, w) \vdash (q', w')$.

Definition 4.2.6. Blocked configuration
A blocked configuration of an NFA $M = (K, \Sigma, \Delta, s, F)$ is any non terminal configuration (q, w) that verifies that $\nexists (q', w') \in K \times \Sigma^* \,:\, (q, w) \vdash (q', w')$.

Definition 4.2.7. Transiting in n steps
Let (q, w) and (q', w') be configurations of an NFA.
We say that (q, w) transits in n steps to (q', w'), noted $(q, w) \vdash^n (q', w')$, with $n > 1$, iff $\exists\ C_1, C_2, \ldots, C_{n-1}$ configurations of that NFA :

$$(q, w) \vdash C_1,\ C_1 \vdash C_2,\ \ldots,\ C_{n-1} \vdash (q', w')$$

We say that (q, w) transits in 1 step to (q', w'), noted $(q, w) \vdash^1 (q', w')$, iff

$$(q, w) \vdash (q', w')$$

We say that (q, w) transits in 0 step to (q', w'), noted $(q, w) \vdash^0 (q', w')$, iff

$$(q, w) = (q', w')$$

Definition 4.2.8. Transiting in at least one step
Let (q, w) and (q', w') be configurations of an NFA.
We say that (q, w) transits in at least one step to (q', w'), noted $(q, w) \vdash^+ (q', w')$, iff

$$\exists n > 0\ :\ (q, w) \vdash^n (q', w')$$

Definition 4.2.9. Transiting
Let (q, w) and (q', w') be configurations of an NFA.
We say that (q, w) transits to (q', w'), noted $(q, w) \vdash^* (q', w')$, iff

$$\exists n \geq 0 \ : \ (q, w) \vdash^n (q', w')$$

Definition 4.2.10. Computation. Length of a computation
Let M be an NFA, a computation of M is any sequence of configurations $C_0, C_1, C_2, ..., C_n$ such as $C_i \vdash C_{i+1}$ $\forall i \in \{0, ..., n-1\}$ with $n \geq 0$.
It will be noted as $C_0 \vdash C_1 \vdash C_2 \vdash ... \vdash C_n$, and the length of the computation is n.

Definition 4.2.11. Well started/terminated/complete/blocked computation
A well started computation is any computation in which its first configuration is an initial configuration.
A terminated computation is any computation in which its last configuration is a terminal configuration.
A computation is complete if it is well started and terminated.
A computation is blocked if its last configuration is blocked.

Definition 4.2.12. Inaccessible state
$q \in K$ is an inaccessible state $(K, \Sigma, s, F, \Delta)$ iff $\nexists x \in \Sigma^* \ : \ (s, x) \vdash^* (q, \varepsilon)$

Definition 4.2.13. Accepted string
Let $M = (K, \Sigma, \Delta, s, F)$ an NFA and $w \in \Sigma^*$. w is accepted by M iff

$$\exists q \in F \,:\, (s, w) \vdash^* (q, \varepsilon).$$

Definition 4.2.14. Accepted language
Let $M = (K, \Sigma, \Delta, s, F)$ an NFA, the language accepted by M is defined as:

$$\mathcal{L}(M) = \{w \in \Sigma^* \,:\, w \text{ is accepted by } M\}$$

Definition 4.2.15. Set of languages accepted by NFA
The set of the languajes accepted by NFA is defined as:

$$\mathscr{L}(\text{NFA}) = \{L \,:\, \exists M \text{ NFA}, \mathcal{L}(M) = L\}$$

Definition 4.2.16. States diagram
The state diagram of an NFA is a directed graph in which each node represents a state, and each edge represents a transition for a certain input between the states the edge connects (i.e. if $(q, w, q') \in \Delta$ then there will be an edge from q to q' labeled as w). Final states are represented with double circles and the initial state is marked with ">".

Example 4.2.1. Let $M = (K, \Sigma, \Delta, s, F)$ be an NFA with:

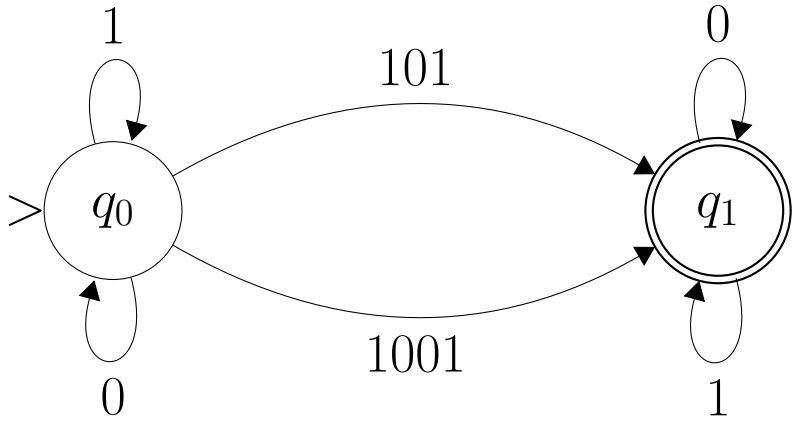
$K = \{q_0, q_1\}$

$\Sigma = \{0, 1\}$

$\Delta = \{(q_0, 0, q_0), (q_0, 1, q_0), (q_0, 101, q_1), (q_0, 1001, q_1), (q_1, 0, q_1), (q_1, 1, q_1)\}$

$s = q_0$

$F = \{q_1\}$



$(q_0, 01) \vdash (q_0, 1) \vdash (q_0, \varepsilon) \wedge q_0 \notin F \Rightarrow 01 \notin \mathcal{L}(M)$

$(q_0, 1101) \vdash (q_0, 101) \vdash (q_0, 01) \vdash (q_0, 1) \vdash (q_0, \varepsilon) \wedge q_0 \notin F \Rightarrow 1101 \notin \mathcal{L}(M)$

$(q_0, 1101) \vdash (q_0, 101) \vdash (q_1, \varepsilon) \wedge q_1 \in F \Rightarrow 1101 \in \mathcal{L}(M)$

Example 4.2.2. Strings accepted or rejected by an NFA representing aa^*bb^*

```
octave> finiteautomaton('aa*bb*', 'aabb');
M = ({q0, q1, q2}, {a, b}, {(q0, a, q1), (q1, a, q1), (q1, b, q2),
(q2, b, q2)}, q0, {q2})
w = aabb
(q0, aabb) ⊢ (q1, abb) ⊢ (q1, bb) ⊢ (q2, b) ⊢ (q2, ε)
w ∈ ℒ(M)

octave> finiteautomaton('aa*bb*', 'ba');
M = ({q0, q1, q2}, {a, b}, {(q0, a, q1), (q1, a, q1), (q1, b, q2),
(q2, b, q2)}, q0, {q2})
w = ba
(q0, ba)
Blocked computation, w not accepted by M.

octave> finiteautomaton('aa*bb*', 'aabba');
M = ({q0, q1, q2}, {a, b}, {(q0, a, q1), (q1, a, q1), (q1, b, q2),
(q2, b, q2)}, q0, {q2})
w = aabba
(q0, aabba) ⊢ (q1, abba) ⊢ (q1, bba) ⊢ (q2, ba) ⊢ (q2, a)
Blocked computation, w not accepted by M.
```

4.3 Minimum deterministic finite automaton

Definition 4.3.1. Equivalent finite automata ($M_1 \equiv M_2$)
Two finite automata, M_1 and M_2 are equivalent iff:

$$\mathcal{L}(M_1) = \mathcal{L}(M_2)$$

Definition 4.3.2. Minimum deterministic finite automaton (MDFA)
For a DFA $M = (K, \Sigma, \delta, s, F)$, M is a MDFA iff it verifies:

$$M' \equiv M \Rightarrow |K'| \geq |K|,\ \forall M' = (K', \Sigma, \delta', s', F')$$

Proposition 4.3.1. $\forall M$ DFA $\exists M'$ MDFA : $M \equiv M'$

4.4 Equivalences

Definition 4.4.1. Set of the languages represented by finite automata

$$\mathscr{L}(\mathcal{F}) = \mathscr{L}(\text{DFA}) = \mathscr{L}(\text{NFA})$$

Theorem 4.4.1. $L \in \mathscr{L}(\mathcal{R}) \Leftrightarrow L \in \mathscr{L}(\mathcal{F})$

Theorem 4.4.2. $L \in \mathscr{L}.3 \Leftrightarrow L \in \mathscr{L}(\mathcal{F})$

Corollary 4.4.1. $\mathscr{L}.3 = \mathscr{L}(\mathcal{R}) = \mathscr{L}(\mathcal{F})$

4.5 Practices of chapter 4

Practice 4.5.1. Get to know the script.

open a Linux terminal (`Alt-Ctrl-T`)

change to `software/automata/`

run `$ octave`

get help about the script `finiteautomaton.m`

Practice 4.5.2. Make your own automaton.
Define a DFA and a NFA for the language $0^* + (0^*10^*10^*10^*)^*$, insert them in `finiteautomata.json`, and simulate them.

Practice 4.5.3. Make your own automaton.
Define a NFA for the language $\{w \in \{0,1\} : w \text{ ends with } 01\}$, and simulate it.

Practice 4.5.4. Make your own automaton.
Define a DFA for the language $\{ava \in \{0,1\} : a \in \Sigma, v \in \Sigma^+\}$, and simulate it.

Chapter 5

Regularity conditions

5.1 Myhill-Nerode 84
5.2 Pumping lemma 88

5.1 Myhill-Nerode

Definition 5.1.1. Indistinguishable strings (*cadenas indistinguibles*)

$$x \approx_L y \Leftrightarrow \ \forall z \in \Sigma^* \ (xz \in L \wedge yz \in L) \vee (xz \notin L \wedge yz \notin L)$$

If the condition meets, $x, y \in \Sigma^*$ are indistinguishable strings with respect to $L \subseteq \Sigma^*$.

Remark. Conversely, $x, y \in \Sigma^*$ are distinguishable strings with respect to $L \subseteq \Sigma^*$ iff

$$x \not\approx_L y \Leftrightarrow \ \exists z \in \Sigma^* \ : \ (xz \in L \wedge yz \notin L) \vee (xz \notin L \wedge yz \in L)$$

Definition 5.1.2. Indistinguishability relation (*relación de indistinguibilidad*)

$$I_L = \{(x, y) \in \Sigma^* \times \Sigma^* \ : \ x \approx_L y\}$$

Proposition 5.1.1. I_L is an equivalence relation over Σ^*.

Remark. Since I_L is an equivalence relation, it establishes a partition over Σ^* where each element is an equivalence class, this partition is noted Π_L.

Theorem 5.1.1. Myhill-Nerode theorem: $L \in \mathscr{L}.3 \Leftrightarrow |\Pi_L| \in \mathbb{N}$

Proof. It has to be proven in both directions.

a) $L \in \mathscr{L}.3 \Rightarrow |\Pi_L| \in \mathbb{N}$

$L \in \mathscr{L}.3 \Rightarrow L \in \mathscr{L}(\text{AFD}) \Rightarrow \exists M$ AFD without inaccessible states $:\ \mathcal{L}(M) = L$.

Let $M = (K = \{q_0, \ldots, q_n\}, \Sigma, \delta, q_0, F)$ and $\delta^* : K \times \Sigma^* \to K$ defined as

$$\delta^*(q_i, x) = q_j \ \Leftrightarrow\ (q_i, x) \vdash^* (q_j, \varepsilon)$$

Let $A_i = \{x \in \Sigma^* \ :\ \delta^*(q_0, x) = q_i\}$ with $i = 0, \ldots, n$, and $\Pi = \{A_0, \ldots, A_n\}$.
M is a DFA, so Π is a partition of Σ^* and

$$\forall x, y \in A_i\ (x, y) \in I_L, i = 0, \ldots, n$$

Let $\Pi' = \{B \subseteq \Sigma^* : B = A_{i_1} \cup \ldots \cup A_{i_k}, A_{i_j} \in \Pi \quad \wedge$

$$\forall x, y \in B\ (x, y) \in I_L \quad \wedge$$

$$(x \in B \wedge y \notin B) \Rightarrow (x, y) \notin I_L\}$$

By definition, we have that Π' is a partition of Σ^*, meeting the criteria:

$$(\Pi' = \Pi_L \wedge |\Pi'| \leq |\Pi| = n + 1) \Rightarrow |\Pi_L| \leq n + 1 \Rightarrow |\Pi_L| \in \mathbb{N}$$

b) $|\Pi_L| \in \mathbb{N} \Rightarrow L \in \mathscr{L}.3$

$|\Pi_L| \in \mathbb{N} \Rightarrow |\Pi_L| = m \geq 1 \Rightarrow \Pi_L = \{A_0, \ldots, A_{m-1}\}$
Let $\Sigma = \{a_1, \ldots, a_n\}$ the alphabet over which L is defined.
Let $M = (K, \Sigma, \delta, s, F)$ be a DFA with:

$$K = \{q_0, \ldots, q_{m-1}\}$$

$$\Sigma = \{a_1, \ldots, a_n\}$$

$$\delta(q_i, a_k) = q_j \;\Leftrightarrow\; x \in A_i \Rightarrow xa_k \in A_j$$

$$s = q_j \;\Leftrightarrow\; \varepsilon \in A_j$$

$$F = \{q_i \in K \,:\, x \in A_i \Rightarrow x \in L\}$$

Constructively we have that $\mathcal{L}(M) = L \Rightarrow L \in \mathscr{L}.3$ $\square$

Remark.

This is a necessary and sufficient condition for a language to be regular.

M is an MDFA that represents L.

If infinite strings that are distinguishable with respect to L can be found, then L is not a regular language.

Example 5.1.1. Determine that $L = \{0^n 1^n \,:\, n \in \mathbb{N}\} \notin \mathscr{L}.3$
Given $S = \{0^n \,:\, n \in \mathbb{N}\}$, any two strings in S, say 0^k and 0^l (with $k \neq l$), are distinguishable with respect to L, since $0^k 1^k \in L \wedge 0^l 1^k \notin L$.

Example 5.1.2. Determine that $\mathcal{R} \notin \mathscr{L}.3$
According to rules 1 and 3 in definition 3.1.1, the strings $\emptyset, (\emptyset\emptyset) \in \mathcal{R}$, and so are regular expressions the following strings:

$(\emptyset(\emptyset\emptyset))$
$(\emptyset(\emptyset(\emptyset\emptyset)))$
$(\emptyset(\emptyset(\emptyset(\emptyset\emptyset))))$
$\ldots$

which can be represented as $\{[(\emptyset]^n\emptyset)^n \,:\, n \in \mathbb{N}\}$[1].
Given $S = \{[(\emptyset]^n \,:\, n \in \mathbb{N}\}$, any two strings in S, say $[(\emptyset]^k$ and $[(\emptyset]^l$ (with $k \neq l$), are distinguishable with respect to $\mathcal{R}$, since $[(\emptyset]^k\emptyset)^k \in \mathcal{R} \wedge [(\emptyset]^l\emptyset)^k \notin \mathcal{R}$.

[1] Squared brackets (instead of parentheses) are used here for convenience.

5.2 Pumping lemma

Theorem 5.2.1. Pumping (*bombeo*)
Let $(K, \Sigma, \delta, s, F)$ DFA with $|K| = n$, and $L = \mathcal{L}(M)$.
Then, $\forall x \in L \,:\, |x| \geq n, \exists u, v, w \in \Sigma^* \,:$

1. $x = uvw$
2. $|uv| \leq n$
3. $|v| > 0$
4. $\forall m \geq 0 \; uv^m w \in L$

Definition 5.2.1. Regular pumping condition (RPC)
L verifies the RPC iff $\exists n \in \mathbb{N} \,:\, \forall x \in L, |x| \geq n, \exists u, v, w \in \Sigma^*$ with

1. $x = uvw$
2. $|uv| \leq n$
3. $|v| > 0$
4. $\forall m \geq 0 \; uv^m w \in L$

Pumping lemma for $\mathscr{L}.3$:
$L \in \mathscr{L}.3 \Rightarrow L$ verifies the regular pumping condition.

Example 5.2.1. Determine that $L = \{0^n 1^n \,:\, n \in \mathbb{N}\} \notin \mathscr{L}.3$
It must be shown that

$$\forall n \in \mathbb{N}\ \exists x \in L\ |x| \geq n \,:\, \forall u, v, w \in \Sigma^* \text{ they do not meet the RPC.}$$

Let $n \in \mathbb{N}$ and $x = 0^n 1^n \in L$, then $|x| = 2n \geq n$. Now, let assume that $\exists u, v, w \in \Sigma^* \,:$ they verify the RPC. Then

1. $x = uvw$
2. $|uv| \leq n \overset{1}{\Rightarrow} uv = 0^k, k \leq n \overset{1}{\Rightarrow} w = 0^{n-k}1^n$
3. $|v| > 0 \overset{2}{\Rightarrow} v = 0^j,\ 0 < j < k$
4. $\forall m \geq 0\ uv^m w \in L \Rightarrow \forall m > 1\ uv^m w \in L$

Considering any $m > 1$ we can se that

$$uv^m w = (uv)v^{m-1}w = 0^k(0^j)^{m-1}0^{n-k}1^n = 0^{k+j(m-1)+(n-k)}1^n \notin L$$

because $k+j(m-1)+(n-k) = j(m-1)+n \neq n$, since $j > 0 \wedge m > 1 \Rightarrow j(m-1) > 0$. Meaning that any selection of u, v, w will not meet the RPC, and then, the language is not regular.

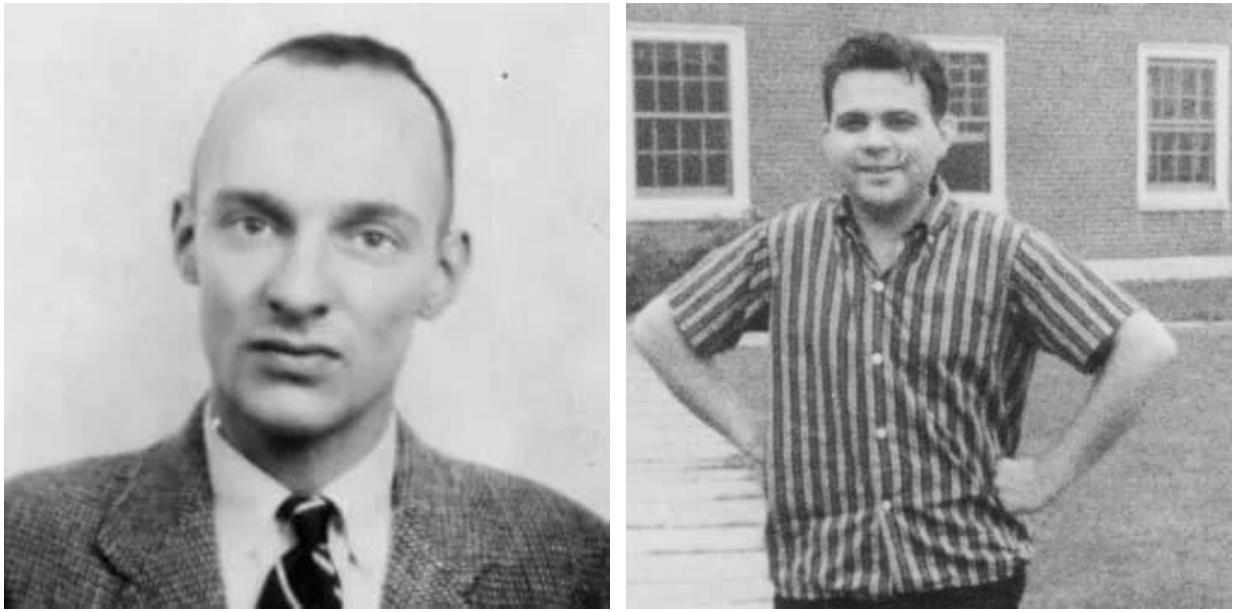


Figure 5.1: John Myhill (left) and Anil Nerode (right) proved the theorem named after them in 1957.

Chapter 6

Context-free languages

6.1 Derivations and Ambiguity 92
6.2 Recursion 97
6.3 CFG Simplification 99
6.4 Normal forms 101
6.5 Closure properties 104
6.6 Non-Deterministic Pushdown Automata 105
6.7 Pumping 114

6.1 Derivations and Ambiguity

Definition 6.1.1. Derivation A-subtree (*Subárbol-A de derivación*)
For a context free grammar $G = (N, T, P, S)$, a tree is a derivation A-subtree associated to G, with $A \in N$, if:

1. Each node is labeled with a symbol from V.
2. The root is labeled as A
3. If a node is labeled as B and its children nodes from left to right are labeled as $X_1, X_2, ..., X_k$, then:

$$B \to X_1 X_2 \ldots X_k \in P$$

Definition 6.1.2. Product of a derivation A-subtree
The product of a derivation A-subtree, is the string obtained from reading its leaves from left to right.

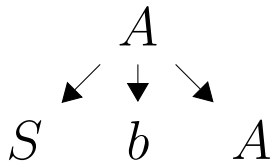
Definition 6.1.3. A-derivation
An A-derivation associated to a derivation A-subtree is the finite sequence of direct productions starting from A used to obtain the A-subtree.

Definition 6.1.4. Derivation tree
A derivation tree is a derivation S-subtree (S is the grammar's axiom)

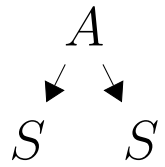
Definition 6.1.5. Derivation
A derivation is an S-derivation (S is the grammar's axiom).

Definition 6.1.6. Left-most and right-most derivations
If in every step of a derivation it is appliead a rule of production to the non terminal character most to the left/right then the derivation is a left/right-most derivation.

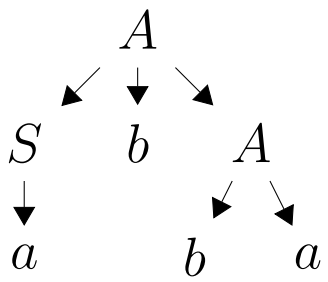
Example 6.1.1. Derivation A-subtrees: products and A-derivations
Given $G = (\{S, A\}, \{a, b\}, \{S \to aAS \mid a, A \to SbA \mid SS \mid ba\ \}, S)$, these are some examples of derivation A-subtrees, their products and A-derivations:



Product: SbA
A-derivation: $A \Rightarrow SbA$



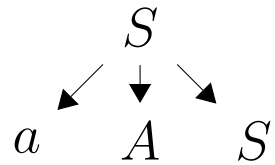
Product: SS
A-derivation: $A \Rightarrow SS$



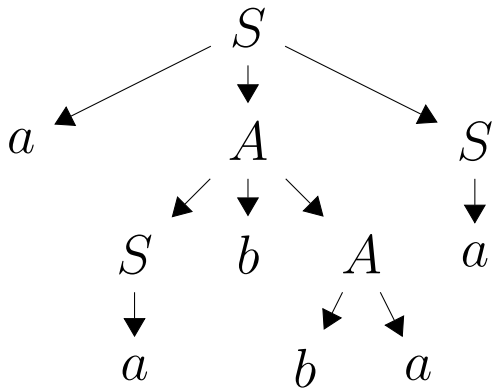
Product: $abba$
A-derivations:
$A \Rightarrow Sb\boldsymbol{A} \Rightarrow \boldsymbol{S}bba \Rightarrow abba$
$A \Rightarrow \boldsymbol{S}bA \Rightarrow ab\boldsymbol{A} \Rightarrow abba$

Example 6.1.2. Derivation trees (S-subtrees): products and derivations

Product: a
Derivation:
$S \Rightarrow a$



Product: aAS
Derivation:
$S \Rightarrow aAS$



Product: $aabbaa$
Derivations:

$$S \Rightarrow a\boldsymbol{A}S \Rightarrow aSb\boldsymbol{A}S \Rightarrow aSbba\boldsymbol{S} \Rightarrow a\boldsymbol{S}bbaa \Rightarrow aabbaa$$

$$S \Rightarrow a\boldsymbol{A}S \Rightarrow a\boldsymbol{S}bAS \Rightarrow aab\boldsymbol{A}S \Rightarrow aabba\boldsymbol{S} \Rightarrow aabbaa \text{ (left-most derivation)}$$

$$S \Rightarrow aA\boldsymbol{S} \Rightarrow a\boldsymbol{A}a \Rightarrow aSb\boldsymbol{A}a \Rightarrow a\boldsymbol{S}bbaa \Rightarrow aabbaa \text{ (right-most derivation)}$$

Definition 6.1.7. Ambiguous grammar
Let G be a CFG. It is said that G is ambiguous iff
$\exists w \in \mathcal{L}(G) : w$ is a product of more than one derivation tree.

Remark. Ambiguity is an undecidable property, i.e. there is no conclusive algorithm that for any given CFG it can answer in a finite time whether such CFG is ambiguous or not.

Example 6.1.3. Let $G = (\{S, A, B\}, \{a\}, \{S \to A \mid B, A \to a, B \to a\}, S)$.

$$\begin{array}{cc} S & S \\ \downarrow & \downarrow \\ A & B \\ \downarrow & \downarrow \\ a & a \end{array}$$

The string a is the product of two distinct trees, which means that G is ambiguous.

Definition 6.1.8. Inherently ambiguous language (*lenguaje inherentemente ambiguo*)
A context-free language L is said to be inherently ambiguous iff:

$$L = \mathcal{L}(G) \Rightarrow G \text{ is ambiguous}$$

6.2 Recursion

Definition 6.2.1. Recursion (*recursividad*)

A CFG (N, T, P, S) is directly left-recursive iff:

$$\exists A \in N : A \Rightarrow A\alpha, \text{ with } \alpha \in V^*.$$

G is directly right-recursive iff:

$$\exists A \in N : A \Rightarrow \alpha A, \text{ with } \alpha \in V^*.$$

G is left-recursive iff:

$$\exists A \in N : A \Rightarrow^+ A\alpha, \text{ with } \alpha \in V^*.$$

G is right-recursive iff:

$$\exists A \in N : A \Rightarrow^+ \alpha A, \text{ with } \alpha \in V^*.$$

G is recursive iff:

$$\exists A \in N : A \Rightarrow^+ \alpha A\beta, \text{ with } \alpha, \beta \in V^*.$$

Proposition 6.2.1. Let G be a CFG, therefore there is a G' CFG that is not directly left-recursive such that $G \equiv G'$.

Proposition 6.2.2. Let G be a CFG, therefore there is a G' CFG that is not left-recursive such that $G \equiv G'$.

6.3 CFG Simplification

Definition 6.3.1. Useful and useless symbols (*símbolos útiles e inútiles*)
Let $G = (N, T, P, S)$ be a CFG and let $X \in V$. X is an useful symbol iff

$$\exists \alpha, \beta \in V^* \,:\, S \Rightarrow^* \alpha X \beta \Rightarrow^* w \in T^+$$

Otherwise, X is said to be a useless symbol.

Definition 6.3.2. Generating symbol, non-generating symbol (*símbolo terminable*)
X is a generating symbol iff

$$\exists w \in T^+ \,:\, X \Rightarrow^* w$$

Otherwise X is a non-generating symbol.

Definition 6.3.3. Reachable symbol, unreachable symbol (*símbolo alcanzable*)
X is a reachable symbol iff

$$\exists \alpha, \beta \in V^* \,:\, S \Rightarrow^* \alpha X \beta$$

Otherwise, X is an unreachable symbol.

Remark. X being both generating and reachable does not imply that it is useful.

Definition 6.3.4. Unit production (*regla unitaria*)
A unit production is any rule of the form $A \to B \,:\, A, B \in N$.

Definition 6.3.5. Proper grammar (*gramática propia*)
A CFG is said to be proper iff it is not left-recursive and it does not have useless symbols.

Proposition 6.3.1. Let G be a CFG that generates a language different than the empty language, therefore there exists a proper CFG G' such that $G \equiv G'$

Proposition 6.3.2. Let G be a CFG, therefore there exists a CFG G' without unit productions such that $G \equiv G'$.

6.4 Normal forms

Definition 6.4.1. Greibach normal form (*forma normal de Greibach*)
Let $G = (N, T, P, S)$ be a CFG. G is in Greibach normal form (GNF) iff

$$\forall (A \to \alpha) \in P \quad \alpha = a\beta \qquad \text{with } a \in T \text{ and } \beta \in N^*$$

Proposition 6.4.1. $\forall G$ CFG $\exists G'$ in GNF : $G \equiv G'$.

Proposition 6.4.2. Given a CFG G in GNF, then any string $w \in \mathcal{L}(G)$ is produced in a derivation of lenght $|w|$.

Definition 6.4.2. Chomsky normal form (*forma normal de Chomsky*)
Let $G = (N, T, P, S)$ be a CFG. G is in Chomsky normal form (CNF) iff

$$\forall (A \to \alpha) \in P \quad \alpha = BC \vee \alpha = a \qquad \text{with } B, C \in N \text{ and } a \in T$$

Proposition 6.4.3. $\forall G$ CFG $\exists G'$ in CNF : $G \equiv G'$.

Proposition 6.4.4. Given a CFG G in CNF, then any string $w \in \mathcal{L}(G)$ is produced in a derivation of lenght $2|w| - 1$.

Example 6.4.1. Derivations of a CFG in GNF.

```
octave> produce('GNF', 2);
(
  {S, A, B},
  {0, 1},
  {
    S → 0ABS
    S → 1AB
    A → 1B
    B → 0
  },
  S
)

S => 1AB => 11BB => 110B => 1100

S => 0ABS => 01BBS => 010BS => 010B1AB => 010B11BB => 010B110B =>
010B1100 => 01001100
```

Example 6.4.2. Derivations of a CFG in CNF.

```
octave> produce('CNF', 2);
(
  {S, A, B, C, D},
  {0, 1},
  {
    S → AB
    A → CD
    A → CB
    B → DC
    B → AD
    C → 0
    D → 1
  },
  S
)

S => AB => ADC => AD0 => A10 => CD10 => C110 => 0110

S => AB => CBB => CADB => CA1B => CA1DC => CA11C => CA110 => CCB110
=> CCDC110 => CCD0110 => CC10110 => C010110 => 0010110
```

6.5 Closure properties

Theorem 6.5.1. $\mathscr{L}.2$ is closed for the operations of union, concatenation, and Kleene star.

Proposition 6.5.1. $\mathscr{L}.2$ is not closed for the operations of intersection and complement.

6.6 Non-Deterministic Pushdown Automata

Definition 6.6.1. Non-deterministic pushdown automaton (*autómata con pila*)
A non-deterministic pushdown automaton (NPA) is a sextuple $M = (K, \Sigma, \Gamma, \Delta, s, F)$ where:

K is a finite set of states.

Σ is an alphabet (input symbols).

Γ is an alphabet (stack symbols).

$s \in K$ is the start state.

$F \subseteq K$ is the set of final states.

Δ is a transition relation defined as a subset of $(K \times \Sigma^* \times \Gamma^*) \times (K \times \Gamma^*)$.

Definition 6.6.2. Transition
Let $M = (K, \Sigma, \Gamma, \Delta, s, F)$ be an NPA. Each pair $((p, u, \beta), (q, \gamma)) \in \Delta$ is called a transition of M.
Hence, Δ is a finite set of transitions.

Definition 6.6.3. Configuration
C is a configuration of $M = (K, \Sigma, \Gamma, \Delta, s, F)$ iff $C \in K \times \Sigma^* \times \Gamma^*$.

Definition 6.6.4. Initial configuration
Given an NPA $M = (K, \Sigma, \Gamma, \Delta, s, F)$, an initial configuration of M is defined as any configuration (s, w, ε).

Definition 6.6.5. Terminal configuration
Given a NPA $M = (K, \Sigma, \Gamma, \Delta, s, F)$, a terminal configuration of M is defined as any configuration $(q, \varepsilon, \varepsilon)$.

Definition 6.6.6. Blocked configuration
Let $M = (K, \Sigma, \Gamma, \Delta, s, F)$ be an NPA and (q, u, α) a configuration of M. (q, u, α) is a blocked configuration if it is not terminal and

$$\nexists (q', u', \alpha') \in K \times \Sigma^* \times \Gamma^* \ : \ (q, u, \alpha) \vdash (q', u', \alpha')$$

Definition 6.6.7. Direct transition
Let (q, u, α) and (q', u', α') be two configurations of a NPA $M = (K, \Sigma, \Gamma, \Delta, s, F)$.
(q, u, α) directly transits to (q', u', α'), noted as $(q, u, \alpha) \vdash (q', u', \alpha')$, iff

$$\exists v \in \Sigma^* \text{ and } \beta, \beta', \gamma \in \Gamma^* \ : \\ (u = vu') \wedge (\alpha = \beta\gamma) \wedge (\alpha' = \beta'\gamma) \wedge ((q, v, \beta), (q', \beta')) \in \Delta$$

Definition 6.6.8. Transition in n steps
Let M be an NPA and (q, u, α) and (q', u', α') configurations of M.
(q, u, α) transits in n steps to (q', u', α'), $n > 1$, noted as $(q, u, \alpha) \vdash^n (q', u', \alpha')$, iff

$$\exists C_1, \ldots, C_{n-1} \ : \ (q, u, \alpha) \vdash C_1, \ C_1 \vdash C_2, \ldots, C_{n-1} \vdash (q', u', \alpha')$$

Definition 6.6.9. To transit in at least one step
Let M be an NPA and (q, u, α) and (q', u', α') configurations of M.
(q, u, α) transits in at least one step to (q', u', α'), noted as $(q, u, \alpha) \vdash^+ (q', u', \alpha')$, iff

$$\exists n > 0 \ : \ (q, u, \alpha) \vdash^n (q', u', \alpha')$$

Definition 6.6.10. To transit
Let M be an NPA and (q, u, α) and (q', u', α') configurations of M.
(q, u, α) transits to (q', u', α'), noted as $(q, u, \alpha) \vdash^* (q', u', \alpha')$, iff

$$\exists n \geq 0 \ : \ (q, u, \alpha) \vdash^n (q', u', \alpha')$$

Definition 6.6.11. Computation and length of a computation
Let M be an NPA, a computation of M is any finite sequence of configurations $C_0, C_1, \ldots, C_n : C_i \vdash C_{i+1}, i \in \{0 \ldots n-1\}$, noted as

$$C_0 \vdash C_1 \vdash \cdots \vdash C_n$$

The length of such computation is n.

Definition 6.6.12. Well-initiated, terminal, complete and blocked computations
A well-initiated computation is any computation whose first configuration is initial.
A terminal computation is any computation whose last configuration is terminal.
A computation is complete when it is both well-initiated and terminal.
A configuration is blocked when its last configuration is a blocking configuration.

Definition 6.6.13. Accepted string
Let $w \in \Sigma^*$ and M be an NPA. It is said that w is accepted by M iff

$$\exists q \in F \,:\, (s, w, \varepsilon) \vdash^* (q, \varepsilon, \varepsilon).$$

Definition 6.6.14. Accepted language
Let M be an NPA. The language accepted by M, noted as $\mathcal{L}(M)$ is defined as

$$\mathcal{L}(M) = \{w \in \Sigma^* \,:\, w \text{ is accepted by } M\}.$$

Definition 6.6.15. Set of languages accepted by NPA
The set of the languages accepted by NPA, noted as $\mathscr{L}(\text{NPA})$, is defined as:

$$\mathscr{L}(\text{NPA}) = \{L \,:\, \exists M \text{ NPA with } \mathcal{L}(M) = L\ \}$$

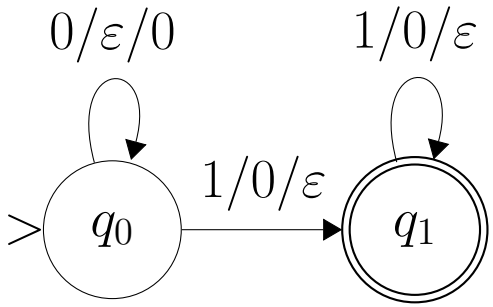
Proposition 6.6.1. $L \in \mathscr{L}.2 \Leftrightarrow L \in \mathscr{L}(\text{NPA})$

Definition 6.6.16. States diagram
The state diagram of an NPA is a directed graph in which each node represents a state, and each edge represents a transition for a certain input between the states the edge connects (i.e. if $((q, u, \beta), (q', \gamma)) \in \Delta$ then there will be an edge from q to q' labeled as $u/\beta/\gamma$). Final states are represented with double circles and the initial state is indicated with ">".

Example 6.6.1. Let $M = (\{q_0, q_1\}, \{0, 1\}, \{0, 1\}, \Delta, q_0, \{q_1\})$ be an NPA, with

$$\Delta = \{((q_0, 0, \varepsilon), (q_0, 0)), ((q_0, 1, 0), (q_1, \varepsilon)), ((q_1, 1, 0), (q_1, \varepsilon))\}$$

then $\mathcal{L}(M) = 0^n 1^n, n \in \mathbb{N}$, and its state-diagram is:



$(q_0, 0011, \varepsilon) \vdash (q_0, 011, 0) \vdash (q_0, 11, 00) \vdash (q_1, 1, 0) \vdash (q_1, \varepsilon, \varepsilon) \Rightarrow 0011 \in \mathcal{L}(M)$
$(q_0, 001, \varepsilon) \vdash (q_0, 01, 0) \vdash (q_0, 1, 00) \vdash (q_1, \varepsilon, 0) \Rightarrow 001 \notin \mathcal{L}(M)$

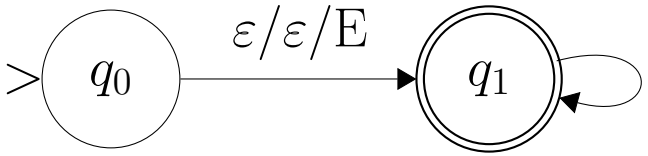
Example 6.6.2. Strings computed by an NPA representing $|w|_a = |w|_b$

```
octave> pushdownautomaton('|w|a=|w|b', 'aabb', 'accept');
M = ({q0}, {a, b}, {a, b}, {((q0, a, ε), (q0, a)), ((q0, b, ε), (q0,
b)), ((q0, a, b), (q0, ε)), ((q0, b, a), (q0, ε))}, q0, {q0})
w = aabb
```

Example 6.6.3. Matching parentheses in an arithmetic expression

```
octave> pushdownautomaton('arithmeticexpression', '(12/(7-5)+6)/3',
'accept');
M = ({q0, q1}, {0, 1, 2, 3, 4, 5, 6, 7, 8, 9, +, -, *, /, (, )}, {0,
1, 2, 3, 4, 5, 6, 7, 8, 9, +, -, *, /, (, )}, {((q0, (, ε), (q0, ())),
((q0, ), (), (q0, ε)), ((q0, 0, ε), (q0, ε)), ((q0, 1, ε), (q0, ε)),
((q0, 2, ε), (q0, ε)), ((q0, 3, ε), (q0, ε)), ((q0, 4, ε), (q0, ε)),
((q0, 5, ε), (q0, ε)), ((q0, 6, ε), (q0, ε)), ((q0, 7, ε), (q0, ε)),
((q0, 8, ε), (q0, ε)), ((q0, 9, ε), (q0, ε)), ((q0, +, ε), (q0, ε)),
((q0, -, ε), (q0, ε)), ((q0, *, ε), (q0, ε)), ((q0, /, ε), (q0, ε))},
q0, {q0})
w = (12/(7-5)+6)/3
```

Example 6.6.4. NPA for the language of regular expressions



$\varepsilon/\mathrm{E}/\emptyset$	$\emptyset/\emptyset/\varepsilon$
$\varepsilon/\mathrm{E}/0$	$0/0/\varepsilon$
$\varepsilon/\mathrm{E}/1$	$1/1/\varepsilon$
$\varepsilon/\mathrm{E}/(\mathrm{EE})$	$(/(/\varepsilon$
$\varepsilon/\mathrm{E}/(\mathrm{E+E})$	$)/(/\varepsilon$
$\varepsilon/\mathrm{E}/\mathrm{E}^*$	$*/(/\varepsilon$
	$+/(/\varepsilon$

```
octave> pushdownautomaton('R01', '(10)*');
M = ({q0, q1}, {∅, 0, 1, *, +, (, )}, {∅, 0, 1, *, +, (, ), E},
{((q0, ε, ε), (q1, E)), ((q1, ε, E), (q1, ∅)), ((q1, ε, E), (q1, 0)),
((q1, ε, E), (q1, 1)), ((q1, ε, E), (q1, (EE))), ((q1, ε, E), (q1,
(E+E))), ((q1, ε, E), (q1, E*)), ((q1, 0, 0), (q1, ε)), ((q1, 1, 1),
(q1, ε)), ((q1, (, (), (q1, ε)), ((q1, ), )), (q1, ε)), ((q1, +, +),
(q1, ε)), ((q1, *, *), (q1, ε))}, q0, {q1})
w = (10)*
```

6.7 Pumping

Definition 6.7.1. Context free pumping condition (*condición de bombeo independiente del contexto*)
Let $L \subseteq \Sigma^*$.
L satisfies the context free pumping condition (CFPC) iff
$\exists n \in \mathbb{N} : \forall x \in L \; with \; |x| \geq n, \exists u, v, w, y, z \in \Sigma^*$ veryfing:

1) $x = uvwyz$

2) $|vwy| < n$

3) $|vy| > 0$

4) $\forall m \geq 0 \;\; uv^m wy^m z \in L$

Pumping lemma for $\mathscr{L}.2$:
$L \in \mathscr{L}.2 \Rightarrow L$ satisfies the CFPC.

Chapter 7

The Turing machine

7.1 Formal definition 116
7.2 Computability, decidability, enumerability 131

7.1 Formal definition

Definition 7.1.1. Turing Machine (TM)
A TM is a 5-tuple $(K, q_0, \Sigma, \gamma, \delta)$, where

K is a non-empty, finite set of states,

$q_0 \in K$ is the initial state,

Σ is an alphabet, and if $\Sigma_R = \{a_0, l, r, h\}$ are reserved words, then $\Sigma \cap \Sigma_R = \emptyset$,

$\gamma : K \times \Sigma_S \to \Sigma_I$ is the instruction function,

$\delta : K \times \Sigma_S \to K$ is the transition function

where $\Sigma_S = \Sigma \cup \{a_0\}$, $\Sigma_I = \Sigma \cup \Sigma_R$ and a_0 is often referred as blank symbol.

Remark. The instructions set is Σ_I, consequently the TM is able to:

— write a symbol of Σ or a blank symbol (a_0) in a cell,

— move its head to the neighbouring left cell (l) or right cell (r),

— halt the computation (h).

If a TM is in a state $q \in K$ and reads $a \in \Sigma$, it will execute the instruction $\gamma(q, a) \in \Sigma_I$ and will transit to the state $\delta(q, a) \in K$.

Definition 7.1.2. Table of a TM

A TM can be represented as a matrix of four columns and $|K| \cdot |\Sigma_S|$ rows.

$$\begin{bmatrix} q_0 & a_0 & \gamma(q_0, a_0) & \delta(q_0, a_0) \\ q_0 & a_1 & \gamma(q_0, a_1) & \delta(q_0, a_1) \\ \vdots & \vdots & \vdots & \vdots \\ q_0 & a_n & \gamma(q_0, a_n) & \delta(q_0, a_n) \\ q_1 & a_0 & \gamma(q_1, a_0) & \delta(q_1, a_0) \\ q_1 & a_1 & \gamma(q_1, a_1) & \delta(q_1, a_1) \\ \vdots & \vdots & \vdots & \vdots \\ q_1 & a_n & \gamma(q_1, a_n) & \delta(q_1, a_n) \\ \vdots & \vdots & \vdots & \vdots \\ q_{|K|-1} & a_0 & \gamma(q_{|K|-1}, a_0) & \delta(q_{|K|-1}, a_0) \\ q_{|K|-1} & a_1 & \gamma(q_{|K|-1}, a_1) & \delta(q_{|K|-1}, a_1) \\ \vdots & \vdots & \vdots & \vdots \\ q_{|K|-1} & a_n & \gamma(q_{|K|-1}, a_n) & \delta(q_{|K|-1}, a_n) \end{bmatrix}$$

Example 7.1.1. Example of a two-state Turing-machine:

$$\begin{array}{cccc} q_0 & * & l & q_1 \\ q_0 & | & r & q_0 \\ q_1 & * & h & q_0 \\ q_1 & | & * & q_1 \end{array}$$

Definition 7.1.3. Tape expression of a TM
A tape expression of a TM is any function $E : \mathbb{Z} \to \Sigma_S$, satisfying

$$|\{z \in \mathbb{Z} \,:\, E(z) \neq a_0\}| \in \mathbb{N}$$

Remark. The (infinite) tape contains a finite number of symbols of Σ.
From now on, $\mathbb{N} = \mathbb{N}_0$.

Definition 7.1.4. Configuration of a TM
A configuration of a TM $M = (K, q_0, \Sigma, \gamma, \delta)$ is any $C = (q, E, z)$, with $q \in K, E$ is a tape expression of M, and $z \in \mathbb{Z}$.

Remark. q represents the current state of the TM, E represents the current content of the tape, and z is the cell where M is reading (which, for representation purposes, can be replaced by underlining the symbol in that cell position, or both notations can coexist, e.g. $(q_0, *||*, 3) = (q_0, *||\underline{*}) = (q_0, *||\underline{*}, 3)$).

Definition 7.1.5. Initial configuration of a TM
An initial configuration of a TM $M = (K, q_0, \Sigma, \gamma, \delta)$ is any configuration (q_0, E, z).

Definition 7.1.6. Terminal and non-terminal configuration of a TM
A configuration (q, E, z) of a given TM is a terminal configuration iff $\gamma\big(q, E(z)\big) = h$.
Otherwise, the configuration would be non-terminal.

Remark. A configuration could be initial and terminal.

$\boxed{hi}$

Definition 7.1.7. Place a TM over a cell
We place a TM M over a cell $z \in \mathbb{Z}$, iff its initial configuration is (q_0, E, z), where q_0 is its initial state, and E is a tape expression.

Example 7.1.2. Place a TM over a cell
Having a TM with $\Sigma = (j, b, c, d)$ we place it over a cell iff $z \in \mathbb{Z}$, q_0 is its initial state, this is the TM configuration $(q_0, E, 11)$ and $\underline{E} = ajjdcbbdcaa$ placing it over a cell would look like this:

			d	j	j	d	c	b	b	d	c	b	$\underline{\text{b}}$			

Remark. The underlined symbol represents the cell that is read from the tape.

Definition 7.1.8. Place a TM behind a string
We place a TM M behind a string w_y, iff we place it over the cell $y + 1$, and

$$E(z) = \begin{cases} a \in \Sigma & \text{if } z \in [x, y] \\ a_0 & \text{otherwise} \end{cases}$$

where $w_y = E(x)E(x+1)...E(y)$, and $M = (K, q_0, \Sigma, \gamma, \delta)$.

Example 7.1.3. Place a TM behind a string
Let $M = (K, q_0, \Sigma, \gamma, \delta)$ be a TM, with $\Sigma = (j, b, c, d)$ and having $\underline{\text{E}}$:

			t	a	l	f			

Where $w = talf$.
Placing a TM $((q_0, E, z)$ with $\underline{E} = bbcdjc)$ behind w_y in cell *y+1* would look like this:

			t	a	l	f	b	b	c	d	j	c			

Definition 7.1.9. Place a TM behind some strings
Let $M = (K, q_0, \Sigma, \gamma, \delta)$ be a TM, and $w_1, w_2, ..., w_n$ strings over Σ. We say that we place M behind $w_1, w_2, ..., w_n$ iff we place it behind the string $w = w_1 a_0 w_2 a_0 ... a_0 w_n$.

Example 7.1.4. Place a TM behind some strings
We place a TM M, where $M = (K, q_0, \Sigma, \gamma, \delta)$, with $\Sigma = (j, b, c, d)$, behind some strings if we have $\underline{w}$ strings over Σ and we have $\underline{E}$ where $\underline{w}$ is placed:

			t	a	l	f		u	m	a				

Where $w_1 = talf$ and $w_2 = uma$ and between w_1, w_2 a_0 is inserted.
Having (q_0, E, z) with $\underline{E} = jdcbd)$ our tape expression would be:

			t	a	l	f		u	m	a	j	d	c	b	d			

Definition 7.1.10. Direct transition
A TM $M = (K, q_0, \Sigma, \gamma, \delta)$ performs a transition from (q, E, z) to (q', E', z'), noted as $(q, E, z) \vdash (q', E', z')$, iff M satisfies all these conditions:

1. $\delta\big(q, E(z)\big) = q'$

2. $$\begin{aligned} &\big(z' = z - 1 \ \wedge \ \gamma\big(q, E(z)\big) = l\big) \quad \vee \\ &\big(z' = z + 1 \ \wedge \ \gamma\big(q, E(z)\big) = r\big) \quad \vee \\ &\big(z' = z \ \wedge \ \gamma\big(q, E(z)\big) \in \Sigma_S\big) \end{aligned}$$

3. $$E'(x) = \begin{cases} E(x) & \text{if} \quad x \neq z \\ E(x) & \text{if} \quad x = z \ \wedge \ \gamma\big(q, E(z)\big) \in \{l, r\} \\ \gamma\big(q, E(x)\big) & \text{if} \quad x = z \ \wedge \ \gamma\big(q, E(z)\big) \in \Sigma_S \end{cases}$$

Example 7.1.5. Direct transition as a piecewise function
Being the TM $M = (K, q_0, \Sigma, \gamma, \delta)$, in the actual state (q_0, E, z), which is about to transit to (q_0', E', z'), represented as $(q, E, z) \vdash (q', E', z')$

$$\gamma\big(q, E(x)\big) = \begin{cases} l & \text{if} \quad z' = z - 1 \\ r & \text{if} \quad z' = z + 1 \\ a \in \Sigma_S & \text{if} \quad z' = z \end{cases}$$

Remark. This represents the movement and actions taken in a tape expression of a TM.

Definition 7.1.11. To transit in n steps
A TM M transits in n steps from a configuration C to C', denoted as $C \vdash^n C'$, iff:
$\exists C_1, C_2, ..., C_{n-1}$ configurations of M:

$$C \vdash C_1 \vdash C_2 \vdash ... \vdash C_{n-1} \vdash C'$$

Definition 7.1.12. To transit in one step
A TM M transits in 1 step from C to C', iff $C \vdash C'$.

Definition 7.1.13. To transit in 0 steps
A TM M transits in 0 steps from C to C', denoted as $C \vdash^0 C'$, iff $C = C'$.

Definition 7.1.14. To transit in at least one step
A TM M transits in at least one step from C to C', denoted as $C \vdash^+ C'$, iff:

$$\exists n > 0 \ : \ C \vdash^n C'$$

Definition 7.1.15. To transit
A TM M transits from C to C', denoted as $C \vdash^* C'$, iff:

$$\exists n \geq 0 \ : \ C \vdash^n C'$$

Definition 7.1.16. Computation, length of a computation
We define a computation of a TM, denoted as $C_0 \vdash C_1 \vdash ... \vdash C_n$, as a finite sequence of configurations $C_0, C_1, ..., C_n$, such that $C_i \vdash C_{i+1}$, where $0 \leq i < n$, with $n \geq 0$. The length of this computation is n.

Definition 7.1.17. Well-started computation
A computation of a TM is well started, if its first configuration is initial.

Definition 7.1.18. Finished computation
A computation of a TM is finished, if its last configuration is terminal.

Definition 7.1.19. Completed computation
A computation of a TM is completed, iff it is well-started and finished.

Definition 7.1.20. Halt after a configuration
Let M be a TM, and C a configuration of M. We say that M halts after C iff:

$$\exists C' \text{ terminal configuration of } M \ : \ C \vdash^* C'$$

Remark. Conversely, M does not halt after a configuration C iff

$$\forall n \geq 0, \exists C' \text{ configuration of } M \ : \ C \vdash^n C'$$

Definition 7.1.21. Halt over or halt behind a string
Let M be a TM, C a configuration, and $C' = (q, E, z)$, a terminal one, such that $C \vdash^* C'$.

$$E(z) \neq a_0 \quad \Rightarrow \quad M, \text{ after } C, \text{ halts over } w_z$$

$$E(z-1) \neq a_0 \wedge E(z) = a_0 \quad \Rightarrow \quad M, \text{ after } C, \text{ halts behind } w_{z-1}$$

$$\forall n \leq z, E(n) = a_0 \quad \Rightarrow \quad M, \text{ after } C, \text{ halts behind } \varepsilon$$

Definition 7.1.22. Expression of the computation of a TM
Let M be a TM, the computation performed by M transforms the tape expression in a way that can be represented with two strings, before placing the TM and after halting, where the underlined symbol is the cell read by M, like this:

$$a_0 w_1 a_0 w_2 \cdots w_n \underline{a_0} \Rightarrow a_0 w'_1 a_0 w'_2 \cdots w'_m \underline{a_0}$$

Example 7.1.6. The predicate $a > b$ is Turing-decidable by a Turing machine defined by the table[1]:

q_0	$*$	l	q_1	right-most symbol of w_2
q_0	$\mid$	l	q_0	void
q_1	$*$	l	q_2	skip separator
q_1	$\mid$	l	q_1	skip w_2
q_2	$*$	l	q_2	skip further separators
q_2	$\mid$	$*$	q_3	erase right-most symbol of $w_1 \Rightarrow w_1 := w_1 - 1$
q_3	$*$	l	q_4	shift left to right-most symbol of w_1
q_3	$\mid$	$\mid$	q_3	void
q_4	$*$	h	q_4	stop and decide $w_1 \leq w_2$
q_4	$\mid$	r	q_5	w_1 not empty yet
q_5	$*$	r	q_5	skip separators
q_5	$\mid$	$*$	q_6	erase left-most symbol of $w_2 \Rightarrow w_2 := w_2 - 1$
q_6	$*$	r	q_7	sifht right to left-most symbol of w_2
q_6	$\mid$	$\mid$	q_6	void
q_7	$*$	$\mid$	q_8	write $\mid$ as true
q_7	$\mid$	l	q_2	w_2 not empty yet
q_8	$*$	$*$	q_8	void
q_8	$\mid$	h	q_8	stop and decide $w_1 > w_2$

[1]Lines and comments added for clarity.

Placing it behind the strings w_1, w_2, these are two computations that it can perform:

$w_1 > w_2 \Rightarrow E(z) = \mid$	$w_1 = w_2 \Rightarrow E(z) = *$
$(q_0, *\mid\mid * \mid\underline{*}, 5) \vdash$	$(q_0, *\mid * \mid\underline{*}, 5) \vdash$
$(q_1, *\mid\mid * \underline{\mid}*, 4) \vdash$	$(q_1, *\mid * \underline{\mid}*, 4) \vdash$
$(q_1, *\mid\mid\underline{*}\mid*, 3) \vdash$	$(q_2, *\mid\underline{*}\mid*, 3) \vdash$
$(q_2, *\mid\underline{\mid} * \mid*, 2) \vdash$	$(q_2, *\underline{\mid} * \mid*, 2) \vdash$
$(q_3, *\mid\underline{*} * \mid*, 2) \vdash$	$(q_3, *\underline{*} * \mid*, 2) \vdash$
$(q_4, *\underline{\mid} * *\mid*, 1) \vdash$	$(q_4, \underline{*} * *\mid*, 1)$
$(q_5, *\mid\underline{*} * \mid*, 2) \vdash$	
$(q_5, *\mid * \underline{*}\mid*, 3) \vdash$	
$(q_5, *\mid * *\underline{\mid}*, 4) \vdash$	
$(q_6, *\mid * *\underline{*}*, 4) \vdash$	
$(q_7, *\mid * * * \underline{*}, 5) \vdash$	
$(q_8, *\mid * * * \underline{\mid}, 5)$	

Example 7.1.7. Turing machine to compute the successsor of a number in binary notation.

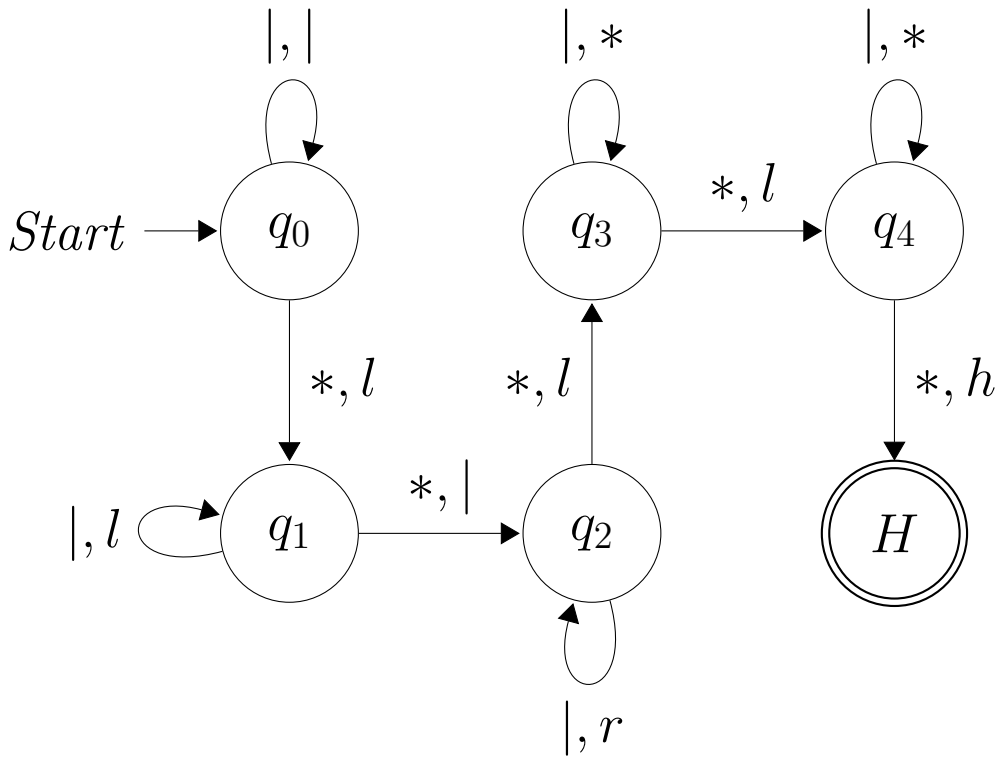
```
octave> turingmachine('successorbinary', '*11*');
q0 * l q1
q0 0 0 q0
q0 1 1 q0
q1 * 1 q3
q1 0 1 q2
q1 1 l q1
q2 * h q3
q2 0 0 q2
q2 1 r q2
q3 * l q4
q3 0 0 q0
q3 1 r q4
q4 * h q4
q4 0 r q4
q4 1 0 q4

(q0, *11*̲, 4) ⊢ (q1, *1̲1*, 3) ⊢ (q1, *̲11*, 2) ⊢ (q1, *̲11*, 1) ⊢ (q3,
*̲111*, 1) ⊢ (q4, *1̲11*, 2) ⊢ (q4, *1̲01*, 2) ⊢ (q4, *10̲1*, 3) ⊢ (q4,
*10̲0*, 3) ⊢ (q4, *100̲*, 4)
```

Definition 7.1.23. States diagram

The states diagram of a TM is a labelled, directed graph where: 1) each <u>node</u> represents a state; 2) each <u>edge</u> represents a transition between states; 3) the <u>label</u> contains two symbols separated by comma: the first symbol represents the character that is read from the tape, and the second is either the character that is written on the tape, or the movement instruction that is performed; 4) the <u>initial state</u> is identified with "$Start \rightarrow$" pointing to it; and 5) the <u>halting</u> situation is represented with an "H" inside a double circle (even if halting it *is not* a state).

Example 7.1.8. *add* function as a table and as a states diagram

q_0	$*$	l	q_1
q_0	$\vert$	$\vert$	q_0
q_1	$*$	$\vert$	q_2
q_1	$\vert$	l	q_1
q_2	$*$	l	q_3
q_2	$\vert$	r	q_2
q_3	$*$	l	q_4
q_3	$\vert$	$*$	q_3
q_4	$*$	h	q_4
q_4	$\vert$	$*$	q_4



Example 7.1.9. A random Turing machine.

```
octave> TM = randomturingmachine(3, '01'); turingmachine(TM, '*10*');
q0 * 1 q2
q0 0 * q1
q0 1 * q0
q1 * h q2
q1 0 1 q1
q1 1 1 q2
q2 * 1 q1
q2 0 r q0
q2 1 r q1

(q0, *10*, 4) ⊢ (q2, *101*, 4) ⊢ (q1, *101*, 5)
```

7.2 Computability, decidability, enumerability

Definition 7.2.1. Turing-computable function
$f : \mathbb{N}^n \to \mathbb{N}$ is a Turing-computable function iff it exists a TM M, such that $\forall (x_1, \ldots, x_n) \in \mathbb{N}^n$:

$$M \text{ halts behind } f(x_1, \ldots, x_n) \text{ if we placed it behind } x_1, \ldots, x_n$$

F(MT) is the set of all Turing-computable functions.

Remark. If the function is not defined for certain values (i.e. is not a total function), then for those values the TM will not halt.

Definition 7.2.2. T-MT

$$\text{T-MT} = \{ f \in \text{F(MT)} : \ f \text{ is a total function } \}$$

Definition 7.2.3. Turing-decidable set
$A \subseteq \mathbb{N}^n$ is Turing-decidable iff it exists a TM M, such that $\forall(x_1, x_2, ..., x_n) \in \mathbb{N}^n$, if M is placed behind the strings $x_1, x_2, .., x_n$, then it halts in a terminal configuration (q, E, z), and:

$$E(z) \neq a_0 \Leftrightarrow (x_1, ..., x_n) \in A$$

DEC(MT) is the set of all Turing-decidable sets.

Definition 7.2.4. Turing enumerable set
$A \subseteq \mathbb{N}^n$ is Turing-enumerable iff it exists a TM M, such that $\forall(x_1, x_2, ..., x_n) \in \mathbb{N}^n$, if M is placed behind the strings $x_1, x_2, ..., x_n$, then

$$M \text{ halts in a terminal configuration } (q, E, z) \wedge E(z) \neq a_0 \Leftrightarrow (x_1, ..., x_n) \in A$$

ENU(MT) is the set of all Turing-enumerable sets.

Remark. DEC(MT) $\subset$ ENU(MT)

Definition 7.2.5. Turing-decidable predicate
A predicate P is Turing-decidable iff $P \in$ PRED(T-MT). We define this set as:

$$\text{PRED(T-MT)} = \{P_f \,:\, f \in \text{T-MT}\}$$

Definition 7.2.6. Turing-enumerable predicate
A predicate P is Turing-enumerable, iff $P \in$ PRED(MT). We define this set as:

$$\text{PRED(MT)} = \{P_f \,:\, f \in \text{F(MT)}\}$$

Remark. PRED(T-MT) $\subset$ PRED(MT)

nddocument

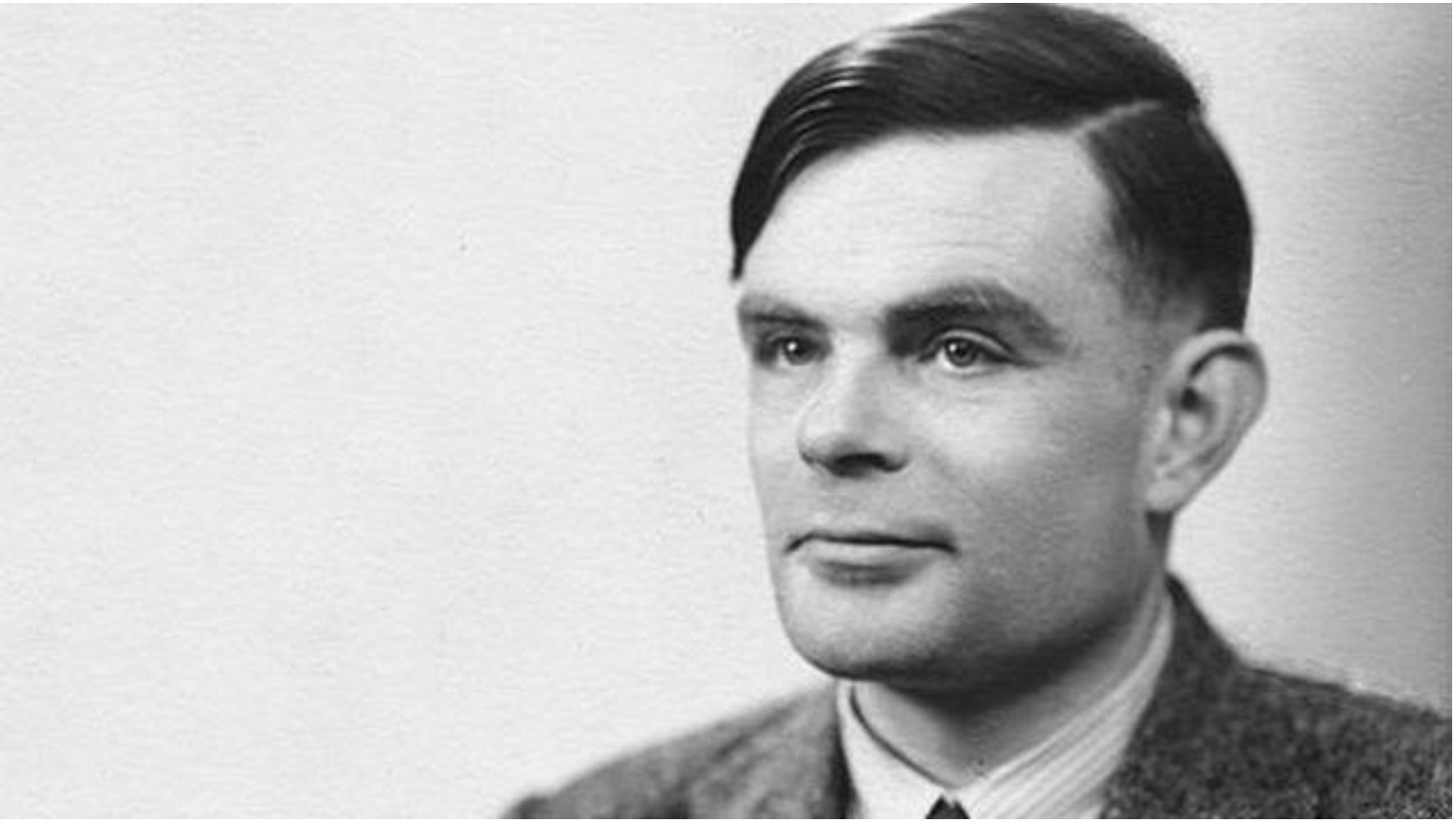


Figure 7.1: Alan Turing proposed the Turing Machine in 1936.

Chapter 8

Recursive functions

8.1 Formal definition 136
8.2 Enumerability, decidability, computability 145

8.1 Formal definition

Definition 8.1.1. Zero function (θ) (*función cero*)
$\theta : \mathbb{N}^0 \to \mathbb{N}$
$\theta() = 0$

Definition 8.1.2. Successor function (*función sucesor*)
$\sigma : \mathbb{N} \to \mathbb{N}$
$\sigma(n) = n + 1$

Definition 8.1.3. Projection function π_i^k (*función proyección de k argumentos*)
For each pair $(k, i) \in \mathbb{N}^2$ with $k \geq 1$ and $1 \leq i \leq k$, the k-ary projection function $\pi_i^k : \mathbb{N}^k \to \mathbb{N}$, that returns the i-th argument, is defined as:

$$\pi_i^k(n_1, \ldots, n_k) = n_i$$

The set π of all the projection functions is defined as

$$\pi = \{\pi_i^k \,:\, (k, i) \in \mathbb{N}^2 \text{ with } k \geq 1 \wedge 1 \leq i \leq k\}$$

Definition 8.1.4. Initial functions (*funciones iniciales*)

$$\text{INI} = \pi \cup \{\theta, \sigma\}$$

Definition 8.1.5. Composition (*composición de funciones*)
Let $m > 0, k \geq 0$ and the functions:
$g : \mathbb{N}^m \to \mathbb{N}$
$h_1, \ldots, h_m : \mathbb{N}^k \to \mathbb{N}$
If the function $f : \mathbb{N}^k \to \mathbb{N}$ is

$$f(\underline{n}) = g(h_1(\underline{n}), \ldots, h_m(\underline{n}))$$

then f is a composition of g and h_1, ..., h_m, expressed as $f(\underline{n}) = g(h_1, \ldots, h_m)(\underline{n})$, or simply $f = g(h_1, \ldots, h_m)$.

Definition 8.1.6. Primitive recursion (*recursión primitiva*)
Let $k \geq 0$ and the functions
$g : \mathbb{N}^k \to \mathbb{N}$
$h : \mathbb{N}^{k+2} \to \mathbb{N}$
If the function $f : \mathbb{N}^{k+1} \to \mathbb{N}$ is

$$f(\underline{n}, m) = \begin{cases} g(\underline{n}) & \text{if} \quad m = 0 \\ h(\underline{n}, m-1, f(\underline{n}, m-1)) & \text{if} \quad m > 0 \end{cases}$$

then f is obtained from g and h by primitive recursion.
We will express it as $f(\underline{n}) = \langle g \mid h \rangle (\underline{n})$, or simply $f = \langle g \mid h \rangle$.

Definition 8.1.7. Minimization (*minimización no acotada*)
Let $k \geq 0$ and the function:
$g : \mathbb{N}^{k+1} \to \mathbb{N}$
If the function $f : \mathbb{N}^k \to \mathbb{N}$ is:

$$f(\underline{n}) = \begin{cases} minimum(A) & \text{if } A \neq \emptyset \wedge \forall t \leq minimum(A) \;\; g(\underline{n}, t) \in \mathbb{N} \\ \uparrow & \text{otherwise} \end{cases}$$

where $A = \{t \in \mathbb{N} \; : \; g(\underline{n}, t) = 0\}$ and $\underline{n} \in \mathbb{N}^k$. Then, f is obtained from g by minimization.
We will express it as $f(\underline{n}) = \mu\,[g]\,(\underline{n})$, or simply $f = \mu\,[g]$.

Remark. The symbol " $\uparrow$ " means that the function is not defined (diverges) for that input.

Example 8.1.1. *additon* is a recursive function

$$\begin{aligned} &addition : \mathbb{N}^2 \to \mathbb{N} \\ &addition(x, y) = x + y \end{aligned}$$

It can be defined like:

$$addition = \langle \pi_1^1 | successor_3 \rangle$$

where

$$\begin{aligned} &successor_3 : \mathbb{N}^3 \to \mathbb{N} \\ &successor_3(x, y, z) = z + 1 \end{aligned}$$

is also a recursive function, because it can be defined as

$$successor_3 = \sigma(\pi_3^3)$$

yielding

$$addition = \left\langle \pi_1^1 \mid \sigma(\pi_3^3) \right\rangle$$

Similarly, the *subtraction* function is also a recursive function

$$subtraction = \left\langle \pi_1^1 \mid \left\langle \theta \mid \pi_1^2 \right\rangle (\pi_3^3) \right\rangle$$

Example 8.1.2. Adding two numbers recursively

$$
\begin{aligned}
\mathit{addition}(4,3) &= \left\langle \pi_1^1 \mid \sigma(\pi_3^3) \right\rangle (4,3) \\
&= \sigma(\pi_3^3)(4,2,\left\langle \pi_1^1 \mid \sigma(\pi_3^3) \right\rangle (4,2)) \\
&= \sigma(\pi_3^3)(4,2,\sigma(\pi_3^3)(4,1,\left\langle \pi_1^1 \mid \sigma(\pi_3^3) \right\rangle (4,1))) \\
&= \sigma(\pi_3^3)(4,2,\sigma(\pi_3^3)(4,1,\sigma(\pi_3^3)(4,0,\left\langle \pi_1^1 \mid \sigma(\pi_3^3) \right\rangle (4,0)))) \\
&= \sigma(\pi_3^3)(4,2,\sigma(\pi_3^3)(4,1,\sigma(\pi_3^3)(4,0,\pi_1^1(4)))) \\
&= \sigma(\pi_3^3)(4,2,\sigma(\pi_3^3)(4,1,\sigma(\pi_3^3)(4,0,4))) \\
&= \sigma(\pi_3^3)(4,2,\sigma(\pi_3^3)(4,1,\sigma(\pi_3^3(4,0,4)))) \\
&= \sigma(\pi_3^3)(4,2,\sigma(\pi_3^3)(4,1,\sigma(4))) \\
&= \sigma(\pi_3^3)(4,2,\sigma(\pi_3^3)(4,1,5)) \\
&= \sigma(\pi_3^3)(4,2,\sigma(\pi_3^3(4,1,5))) \\
&= \sigma(\pi_3^3)(4,2,\sigma(5)) \\
&= \sigma(\pi_3^3)(4,2,6) \\
&= \sigma(\pi_3^3(4,2,6)) \\
&= \sigma(6) \\
&= 7
\end{aligned}
$$

Example 8.1.3. *product* is a recursive function

$$\begin{aligned} &product : \mathbb{N}^2 \to \mathbb{N} \\ &product(x, y) = x \cdot y \end{aligned}$$

and it is a recursive function, because it can be defined like:

$$product = \left\langle \theta^1 \mid addition^3_{1,3} \right\rangle$$

being θ^1 and $addition^3_{1,3}$ recursive functions as well:

$$\begin{aligned} &\theta^1 : \mathbb{N}^1 \to \mathbb{N} \\ &\theta^1(x) = 0 \\ &\theta^1(x) = \left\langle \theta \mid \pi^2_2 \right\rangle \end{aligned}$$

$$\begin{aligned} &addition^3_{1,3} : \mathbb{N}^3 \to \mathbb{N} \\ &addition^3_{1,3}(x, y, z) : x + z \\ &addition^3_{1,3} = addition(\pi^3_1, \pi^3_3) = \left\langle \pi^1_1 \mid \sigma(\pi^3_3) \right\rangle (\pi^3_1, \pi^3_3) \end{aligned}$$

yielding

$$product = \left\langle \left\langle \theta \mid \pi^2_2 \right\rangle \mid \left\langle \pi^1_1 \mid \sigma(\pi^3_3) \right\rangle (\pi^3_1, \pi^3_3) \right\rangle$$

Example 8.1.4. *division* is a recursive function

$$\begin{aligned} division &: \mathbb{N}^2 \to \mathbb{N} \\ division(x, y) &= \frac{x}{y} \end{aligned}$$

it can be defined with the minimization operator:

$$division = \mu \left[subtraction(\pi_1^3, product(\pi_2^3, \pi_3^3)) \right]$$

namely,

$$division = \mu \left[\left\langle \pi_1^1 \mid \left\langle \theta \mid \pi_1^2 \right\rangle (\pi_3^3) \right\rangle (\pi_1^3, \left\langle \left\langle \theta \mid \pi_2^2 \right\rangle \mid \left\langle \pi_1^1 \mid \sigma(\pi_3^3) \right\rangle (\pi_1^3, \pi_3^3) \right\rangle (\pi_2^3, \pi_3^3)) \right]$$

In this approach, the quotient is found in a search process, as the minimum number which multiplied by the divisor equals (or exceeds) the dividend.
For example,

$$\begin{aligned} &division(6, \quad 3) = 2, \text{ since } 6 - 3 \cdot 0 = 6, 6 - 3 \cdot 1 = 3 \text{ and } 6 - 3 \cdot 2 = 0 \\ &division(25, \ 3) = 9, \text{ since } 25 - 3 \cdot 8 = 1 \text{ and } 25 - 3 \cdot 9 = 0 \end{aligned}$$

Example 8.1.5. List of recursive functions

```
software/recursivefunctions$ more recursivefunctions

  constant^1_0    <θ|π^2_2>
  constant^1_1    σ(<θ|π^2_2>)
  constant^2_3    <<σ(σ(σ(θ)))|π^2_2>|π^3_3>
  addition        <π^1_1|σ(π^3_3)>
  predecessor     <θ|π^2_1>
  subtraction     <π^1_1|predecessor(π^3_3)>
  product         <constant^1_0|<π^1_1|σ(π^3_3)>(π^3_1,π^3_3)>
  division        μ[subtraction(π^3_1,product(π^3_2,π^3_3))]
  power           <constant^1_1|product(π^3_1,π^3_3)>
  squareroot      μ[subtraction(π^2_1,product(π^2_2,π^2_2))]
  cuberoot        μ[subtraction(π^2_1,power(π^2_2,constant^2_3))]
```

Definition 8.1.8. Total function, Partial function

$$f : \mathbb{N}^k \to \mathbb{N}, k > 0 \text{ is a total function if } Dom(f) = \mathbb{N}^k$$
$$f : \mathbb{N}^0 \to \mathbb{N} \text{ is a total function if } f() \in \mathbb{N}$$

A function is a partial function iff it is not total.

Definition 8.1.9. Set of the recursive functions
The set REC is defined as

1. $\text{INI} \subset \text{REC}$
2. $g, h_1, \ldots, h_m \in \text{REC} \wedge \exists g(h_1, \ldots, h_m) \Rightarrow g(h_1, \ldots, h_m) \in \text{REC}$
3. $g, h \in \text{REC} \wedge \exists \langle g|h \rangle \Rightarrow \langle g|h \rangle \in \text{REC}$
4. $g \in \text{REC} \wedge \exists \mu\,[g] \Rightarrow \mu\,[g] \in \text{REC}$
5. No other function is in REC.

f is a recursive function iff $f \in \text{REC}$.

Definition 8.1.10. Total recursive functions

$$\text{TREC} = \{f \in \text{REC} \,:\, f \text{ is a total function}\}$$

8.2 Enumerability, decidability, computability

Definition 8.2.1. Set of truth values for a predicate (*valores de verdad*)
The set of truth values for a predicate P with $k \geq 0$ arguments is

$$V_P = \{\underline{x} \in \mathbb{N}^k \,:\, P(\underline{x})\}$$

Definition 8.2.2. Predicate associated to a function
The predicate associated to the function f of $k \geq 0$ arguments, is

$$P_f(\underline{x}) = \begin{cases} true & \text{if } f(\underline{x}) > 0 \\ false & \text{otherwise} \end{cases}$$

Definition 8.2.3. Characteristic function of a set (*función característica*)
The characteristic function of $V \subseteq \mathbb{N}^k$, $X_V \,:\, \mathbb{N}^k \to \mathbb{N}$, with $k \geq 0$, is

$$X_V(\underline{x}) = \begin{cases} 1 & \text{if } \underline{x} \in V \\ 0 & \text{if } \underline{x} \notin V \end{cases}$$

Definition 8.2.4. Characteristic function of a predicate
X_{V_P} is the characteristic function of predicate P.

Definition 8.2.5. Recursively decidable predicates (*predicado r.d.*)

$$\mathrm{PRED(TREC)} = \{P_f \,:\, f \in \mathrm{TREC}\}$$

Definition 8.2.6. Recursively enumerable predicates (*predicado r.e.*)

$$\mathrm{PRED(REC)} = \{P_f \,:\, f \in \mathrm{REC}\}$$

Remark. PRED(TREC) $\subset$ PRED(REC) since TREC $\subset$ REC.

Definition 8.2.7. Recursively decidable sets

$$\mathrm{DEC} = \{V_P \,:\, P \in \mathrm{PRED(TREC)}\}$$

Definition 8.2.8. Recursively enumareable sets

$$\mathrm{ENU} = \{V_P \,:\, P \in \mathrm{PRED(REC)}\}$$

Remark. DEC $\subset$ ENU since PRED(TREC) $\subset$ PRED(REC).

Definition 8.2.9. Recursively generable set
V is a recursively generable set iff V is recursively enumerable.

Figure 8.1: Alonzo Church proposed the recursive functions in 1936.

Chapter 9

The WHILE language

9.1 Formal definition 150
9.2 Computability, decidability, enumerability 160

9.1 Formal definition

Given the alphabets
$\Sigma_d = \{0, 1, 2, 3, 4; 5, 6, 7, 8, 9\}$
$\Sigma_w = \{\text{X}, :=, 1, +, -, \text{while}\ , \neq, 0,\ \text{do}\ ,\ \text{od}, ;\}$

Let $G = (N, T, P, <\text{code}>)$ be a grammar with:

$$
\begin{array}{ll}
N = \{ & <\text{number}>, <\text{firstdigit}>, <\text{restnumber}>, <\text{digit}>, \\
 & <\text{code}>, <\text{sentence}>, <\text{assignment}>, <\text{loop}>, <\text{id}>\} \\
T = & \Sigma_d \cup \Sigma_w \\
P = \{ & \\
 & <\text{number}> \rightarrow <\text{firstdigit}> \mid <\text{firstdigit}><\text{restnumber}>, \\
 & <\text{firstdigit}> \rightarrow 1|2|3|4|5|6|7|8|9, \\
 & <\text{digit}> \rightarrow 0|1|2|3|4|5|6|7|8|9, \\
 & <\text{restnumber}> \rightarrow <\text{digit}> \mid <\text{digit}><\text{restnumber}>, \\
 & <\text{code}> \rightarrow <\text{sentence}> \mid <\text{code}>; <\text{sentence}>, \\
 & <\text{sentence}> \rightarrow <\text{assignment}> \mid <\text{loop}>, \\
 & <\text{id}> \rightarrow \text{X} <\text{number}>, \\
 & <\text{loop}> \rightarrow \text{while}\ <\text{id}> \neq 0\ \text{do}\ <\text{code}>\ \text{od}, \\
 & <\text{assignment}> \rightarrow <\text{id}> := 0 \mid <\text{id}> := <\text{id}> \mid \\
 & \qquad <\text{id}> := <\text{id}> +1 \mid <\text{id}> := <\text{id}> -1\}
\end{array}
$$

Definition 9.1.1. Set of While codes (CODE) (*códigos While*)
$s \in T^+$ is a While code iff $s \in \text{CODE} = \mathcal{L}(G)$.

Remark. CODE contains all the syntactically correct codes written in While language.

Example 9.1.1. Code in While

X2:=X1; while X2 $\neq$ 0 do X1:=X1+1; X2:=X2-1 od

Definition 9.1.2. Set of While programs (WHILE) (*programas While*)
Q is a While program iff $Q \in \text{WHILE} = \{(n, s) \in \mathbb{N} \times \text{CODE}\}$.

Example 9.1.2. *double* function in While

double = (1, X2:=X1; while X2 $\neq$ 0 do X1:=X1+1; X2:=X2-1 od)

Example 9.1.3. *double* function in While - numbered

$double = (1, s)$

s :

```
1 X2 := X1;
2 while X2 ≠ 0 do
3     X1 := X1 + 1;
4     X2 := X2 - 1
5 od
```

Example 9.1.4. *double* function in While - labelled

$double = (1, s)$

s :

```
1 X2 := X1;
2 while X2 ≠ 0 do :6
3     X1 := X1 + 1;
4     X2 := X2 - 1
5 od :2
```

Definition 9.1.3. Size of a While code and a While program
$size \ : \ \text{CODE} \to \mathbb{N}$

$$size(s) = \begin{cases} 1 & \text{if } < assignment > \Rightarrow^* s \\ \\ size(s_1) + 2 & \text{if } s = \text{while X}i \neq 0 \text{ do } s_1 \text{ od} \wedge \\ & \quad i \in \Sigma_d^+ \wedge s_1 \in \text{CODE} \\ \\ size(s_1) + size(s_2) & \text{if } s = s_1; s_2 \wedge s_1, s_2 \in \text{CODE} \end{cases}$$

Similarly, for a While program (overloading the function):
$size \ : \ \text{WHILE} \to \mathbb{N}$
$size\big((n, s)\big) = size(s)$

Example 9.1.5. Size of a While program for the *double* function

```
octave> Size('(1, X2:=X1; while X2!=0 do X1:=X1+1; X2:=X2-1 od)')
ans = 5
```

Definition 9.1.4. Line function

$$line \ : \ \text{CODE} \times \mathbb{N} \to T^*$$

$$line(s,n) = \begin{cases} s & \text{if } < assignment > \Rightarrow^* s \wedge n = 1 \\ \\ \text{while Xi} \neq 0 \text{ do} & \text{if } s = \text{while Xi} \neq 0 \text{ do } s_1 \text{ od} \wedge \\ & \quad i \in \Sigma_d^+ \wedge s_1 \in \text{CODE} \wedge n = 1 \\ line(s_1, n-1) & \text{if } s = \text{while Xi} \neq 0 \text{ do } s_1 \text{ od} \wedge \\ & \quad i \in \Sigma_d^+ \wedge s_1 \in \text{CODE} \wedge 1 < n < size(s) \\ \text{ od} & \text{if } s = \text{while Xi} \neq 0 \text{ do } s_1 \text{ od} \wedge \\ & \quad i \in \Sigma_d^+ \wedge s_1 \in \text{CODE} \wedge n = size(s) \\ \\ line(s_1, n) & \text{if } s = s_1; s_2 \wedge \\ & \quad s_1, s_2 \in \text{CODE} \wedge 0 < n \leq size(s_1) \\ line(s_2, n - size(s_1)) & \text{if } s = s_1; s_2 \wedge \\ & \quad s_1, s_2 \in \text{CODE} \wedge size(s_1) < n \leq size(s) \\ \\ \varepsilon & \text{otherwise} \end{cases}$$

Definition 9.1.5. Control flow function
$go \ : \ \text{CODE} \times \mathbb{N} \to \mathbb{N}$

$$go(s,n) = \begin{cases} size(s)+1 & \text{if } s = \text{while Xi} \neq 0 \text{ do } s_1 \text{ od} \wedge \\ & \quad i \in \Sigma_d^+ \wedge s_1 \in \text{CODE} \wedge n = 1 \\ go(s_1, n-1)+1 & \text{if } s = \text{while Xi} \neq 0 \text{ do } s_1 \text{ od} \wedge \\ & \quad i \in \Sigma_d^+ \wedge s_1 \in \text{CODE} \wedge 1 < n < size(s) \\ 1 & \text{if } s = \text{while Xi} \neq 0 \text{ do } s_1 \text{ od} \wedge \\ & \quad i \in \Sigma_d^+ \wedge s_1 \in \text{CODE} \wedge n = size(s) \\ \\ go(s_1, n) & \text{if } s = s_1; s_2 \wedge \\ & \quad s_1, s_2 \in \text{CODE} \wedge 0 < n \leq size(s_1) \\ go(s_2, n - size(s_1)) + size(s_1) & \text{if } s = s_1; s_2 \wedge \\ & \quad s_1, s_2 \in \text{CODE} \wedge size(s_1) < n \leq size(s) \\ \\ 0 & \text{otherwise} \end{cases}$$

Remark. The *go* function returns the non-consecutive line number that could be jumped to from a given line or zero if from the specified line you can not jump to a non-consecutive line.

Definition 9.1.6. Configurations of a While program
The set of configurations of $Q = (n, s) \in \text{WHILE}$ is

$$C_Q = \left\{(m, \underline{x}) \in \mathbb{N}^{p+1} \,:\, 1 \leq m \leq size(Q) + 1\right\}$$

where $p = max\left\{n, m \in \Sigma_d^+ \,:\, \text{Xm} \prec s\right\}$ (also in further definitions).

Definition 9.1.7. Initial configuration
Let $Q = (n, s) \in \text{WHILE}$.
$\underline{c} = (1, \underline{x}) \in C_Q$ is a initial configuration of Q iff $x_{n+1} = \ldots = x_p = 0$.

Definition 9.1.8. Terminal configuration, non-terminal configuration
Let $Q = (n, s) \in \text{WHILE}$.
$\underline{c} = (m, \underline{x}) \in C_Q$ is a terminal configuration of Q iff $m = size(Q) + 1$.
Otherwise we say that it is a non-terminal configuration.

Remark. A configuration can not be both initial and terminal.

Definition 9.1.9. Direct transition
Let $Q = (n, s) \in \text{WHILE}$ and $\underline{c_1} = (m, \underline{x}), \underline{c_2} = (t, \underline{z}) \in C_Q$.
Q transits directly from c_1 to c_2, $c_1 \vdash c_2$, iff

$$
\begin{array}{lll}
line(s,m) = \text{Xi} \coloneqq 0 & \Rightarrow\ t = m+1 \wedge z_i = 0 & \wedge\ z_r = x_r, \forall r \neq i \\
line(s,m) = \text{Xi} \coloneqq \text{Xj} & \Rightarrow\ t = m+1 \wedge z_i = x_j & \wedge\ z_r = x_r, \forall r \neq i \\
line(s,m) = \text{Xi} \coloneqq \text{Xj+1} & \Rightarrow\ t = m+1 \wedge z_i = x_j + 1 & \wedge\ z_r = x_r, \forall r \neq i \\
line(s,m) = \text{Xi} \coloneqq \text{Xj-1} & \Rightarrow\ t = m+1 \wedge z_i = x_j - 1 & \wedge\ z_r = x_r, \forall r \neq i \\
line(s,m) = \text{while Xi} \neq 0 \text{ do} & \Rightarrow\ (x_i \neq 0 \Rightarrow t = m+1) & \wedge \\
 & \quad (x_i = 0 \Rightarrow t = go(s,m)) & \wedge\ \underline{z} = \underline{x} \\
line(s,m) = \text{ od} & \Rightarrow\ t = go(s,m) & \wedge\ \underline{z} = \underline{x}
\end{array}
$$

where $i, j \in \Sigma_d^+$.

Remark. Assignments update the variables, loop headers do not alter the variables, and loop tails redirect to its header.

Definition 9.1.10. Next configuration function (*función siguiente configuración*)
Let $Q = (n, s) \in$ WHILE (also in further definitions).
$next_Q \ : \ \mathbb{N}^{p+1} \to \mathbb{N}^{p+1}$

$$next_Q(\underline{c}) = \begin{cases} \underline{c'} & \text{if } \underline{c}, \underline{c'} \in C_Q \wedge \underline{c} \vdash \underline{c'} \\ \underline{c} & \text{if } \underline{c} \notin C_Q \vee \underline{c} \in C_Q \wedge \nexists \underline{c'} \in C_Q \ : \ \underline{c} \vdash \underline{c'} \end{cases}$$

Definition 9.1.11. Calculation function (*función cálculo*)
$cal_Q \ : \ \mathbb{N}^{n+1} \to \mathbb{N}^{p+1}$

$$cal_Q(\underline{x}, t) = \begin{cases} (1, \underline{x}, \underline{0}) & \text{if } t = 0 \\ next_Q(cal_Q(\underline{x}, t-1)) & \text{if } t > 0 \end{cases}$$

where $t \in \mathbb{N}, \underline{x} \in \mathbb{N}^n \wedge \underline{0} \in \mathbb{N}^{p-n}$.

Definition 9.1.12. Temporal complexity function (*función complejidad temporal*)
$T_Q \ : \ \mathbb{N}^n \to \mathbb{N}$

$$T_Q(\underline{x}) = \begin{cases} minimum(A) & \text{if } A \neq \emptyset \\ \uparrow & \text{if } A = \emptyset \end{cases}$$

where $A = \{t \in \mathbb{N} \ : \ \pi_1^{p+1}(cal_Q(\underline{x}, t)) = size(Q) + 1\}$

Definition 9.1.13. Calculated function (*función calculada*)
$F_Q : \mathbb{N}^n \to \mathbb{N}$
$F_Q(\underline{x}) = \pi_2^{p+1}(cal_Q(\underline{x}, T_Q(\underline{x})))$

Example 9.1.6. $next_Q$ and cal_Q functions

```
octave> Next('(1, X2:=X1; while X2!=0 do X1:=X1+1; X2:=X2-1 od)',
[1,3,0])
ans =
2 3 3

octave> Cal('(1, X2:=X1; while X2!=0 do X1:=X1+1; X2:=X2-1 od)', 5,
1)
ans =
2 5 5
```

9.2 Computability, decidability, enumerability

Definition 9.2.1. While-computable function (*función While-calculable*)
f is a While-computable function iff $\exists Q \in \text{WHILE} : F_Q = f$.
F(WHILE) is the set of all While-computable functions.

Definition 9.2.2. Total While-computable function
$\text{T-WHILE} = \{f \in \text{F(WHILE)} : f \text{ is a total function}\}$.

Definition 9.2.3. While-decidable predicates (*predicados While-decidibles*)
$\text{PRED(T-WHILE)} = \{P_f : f \in \text{T-WHILE}\}$.

Definition 9.2.4. While-enumerable predicates (*predicados While-enumerables*)
$\text{PRED(WHILE)} = \{P_f : f \in \text{F(WHILE)}\}$.

Definition 9.2.5. While-decidable sets (*conjuntos While-decidibles*)
$\text{DEC(WHILE)} = \{V_P : P \in \text{PRED(T-WHILE)}\}$.

Definition 9.2.6. While-enumerable sets (*conjuntos While-enumerables*)
$\text{ENU(WHILE)} = \{V_P : P \in \text{PRED(WHILE)}\}$.

Equivalences among functions, predicates and sets:

F(MT)	REC	F(WHILE)
DEC(MT)	DEC	DEC(WHILE)
ENU(MT)	ENU	ENU(WHILE)
T-MT	TREC	T-WHILE
PRED(T-MT)	PRED(TREC)	PRED(T-WHILE)
PRED(MT)	PRED(REC)	PRED(WHILE)

Chapter 10

Turing completeness

10.1 Extended While language 164
10.2 Recursive ⇒ While-computable 166

10.1 Extended While language

EXWHILE is the set of all programmes in extended While language, F(EXWHILE) is the set of all the functions represented by EXWHILE.

It can be shown that WHILE $\subseteq$ EXWHILE and F(WHILE) = F(EXWHILE).

Example 10.1.1. *factorial* function in Extended While

$Factorial = (1, s)$

s :

```
(* counting starts from the previous value *)
X2 := X1 - 1;
while X2 ≠ 0 do
    (* macros with prefix or infix notation *)
    X1 := Product(X1, X2);
    X2 := X2 - 1
od
```

Some other control flow structures are allowed:

while C **do**

s

od

do $f(x)$ **times**

s

od

if C **then**

s

fi

if C **then**

s_1

else

s_2

fi

C is defined from a WHILE-enumerable predicate, and $f(x)$ is a WHILE-computable function.

10.2 Recursive $\Rightarrow$ While-computable

Theorem 10.2.1. $\mathrm{REC} = \mathrm{F}(\mathrm{WHILE})$

Proposition 10.2.1. $\mathrm{REC} \subseteq \mathrm{F}(\mathrm{WHILE})$

Proposition 10.2.2. $\mathrm{F}(\mathrm{WHILE}) \subseteq \mathrm{REC}$

Proposition 10.2.3. INI $\subseteq$ F(WHILE)

Proof.
$\theta \in F(WHILE)$
Given $Q = (0, \text{X1} \coloneqq 0\ )$,
$F_Q = \theta \Rightarrow \theta \in$ F(WHILE)

$\sigma \in F(WHILE)$
Given $Q = (1, \text{X1} \coloneqq \text{X1} + 1\ )$,
$F_Q = \sigma \Rightarrow \sigma \in$ F(WHILE)

$\pi \subseteq$ F(WHILE)
Given $Q = (k, \text{X1} \coloneqq \text{Xi}\ ), k \geq 1, 1 \leq i \leq k$,
$F_Q = \pi_i^k \Rightarrow \pi_i^k \in$ F(WHILE) $\Rightarrow \pi \subseteq$ F(WHILE)

$INI = \pi \cup \{\theta,\ \sigma\} \Rightarrow INI \subseteq$ F(WHILE)

$\square$

Proposition 10.2.4.
$f = g(h_1, \ldots, h_m) \ \wedge \ g, h_1, \ldots, h_m \in \mathrm{F(WHILE)} \Rightarrow f \in \mathrm{F(WHILE)}$

Proof.
Given $f = g(h_1, \ldots, h_m)$,

$$g : \mathbb{N}^m \to \mathbb{N}, \qquad F_{\mathrm{G}} = g,$$
$$h_i : \mathbb{N}^k \to \mathbb{N}, \qquad F_{\mathrm{Hi}} = h_i,$$
$$m \geq 1, \ k \geq 0$$

and

$C = (k, s)$
s :

```
Xk+1 := H1(X1, ..., Xk);
...
Xk+m := Hm(X1, ..., Xk);
X1 := G(Xk+1, ..., Xk+m)
```

then $F_C = f \Rightarrow f \in \mathrm{F(WHILE)}$ $\square$

Proposition 10.2.5.
$f = \langle g|h \rangle \wedge g, h \in \mathrm{F(WHILE)} \Rightarrow f \in \mathrm{F(WHILE)}$

Proof.
Given $f = \langle g|h \rangle$,

$$g : \mathbb{N}^k \to \mathbb{N}, \quad F_{\mathrm{G}} = g,$$
$$h : \mathbb{N}^{k+2} \to \mathbb{N}, \quad F_{\mathrm{H}} = h,$$
$$k \geq 0$$

and

$R = (k+1, s)$
s :

```
Xk+2 := G(X1, ..., Xk);
do Xk+1 times
    Xk+2 := H(X1, ..., Xk, Xk+3, Xk+2);
    Xk+3 := Xk+3 + 1
od
X1 := Xk+2
```

then $F_R = f \Rightarrow f \in \mathrm{F(WHILE)}$

$\square$

Proposition 10.2.6.
$f = \mu[g] \wedge g \in \mathrm{F(WHILE)} \Rightarrow f \in \mathrm{F(WHILE)}$

Proof.
Given $f = \mu[g]$,

$$g : \mathbb{N}^{k+1} \to \mathbb{N},\ F_{\mathrm{G}} = g,$$

$$k \geq 0$$

and

$M = (k, s)$
s :

```
while G(X1, ..., Xk, Xk+1) ≠ 0 do
    Xk+1 := Xk+1 + 1
od
X1 := Xk+1
```

then $F_M = f \Rightarrow f \in \mathrm{F(WHILE)}$

$\square$

Chapter 11

Universality

11.1 Program numbering 172
11.2 Universal function 186
11.3 Universal program 188

No non-poetic account of reality can be complete.
John Myhill

11.1 Program numbering

Definition 11.1.1. Index of a set of functions
An index of a countable set of functions $\mathbf{F}$ is any surjective function $h : \mathbb{N} \to \mathbf{F}$
An index of a function $f \in \mathbf{F}$ under the index h is any $i \in \mathbb{N}$ such that $h(i) = f$.

In order to encode programs, we need to introduce the concepts of Gödel's encoding and decoding. These functions are defined using the set of all vectors of natural numbers.

$$\mathbb{N}^* = \bigcup_{n=0}^{\infty} \mathbb{N}^n$$

Remark. $\mathbb{N}^0 = \{(\,)\}$.

Definition 11.1.2. Cantor pairing function (*cantorización de* $\mathbb{N}^2$)
Cantor pairing establishes a bijection between $\mathbb{N}$ and $\mathbb{N}^2$.

$$\begin{aligned}
&\sigma_1^2 \ : \ \mathbb{N} \times \mathbb{N} \to \mathbb{N} \\
&\sigma_1^2(x, y) = \frac{(x+y)(x+y+1)}{2} + y
\end{aligned}$$

$$\begin{aligned}
&\sigma_{2,2}^1 \ : \ \mathbb{N} \to \mathbb{N} \times \mathbb{N} \\
&\sigma_{2,2}^1(z) = z - \sigma_1^2(d(z), 0)
\end{aligned}$$

$$\begin{aligned}
&\sigma_{2,1}^1 \ : \ \mathbb{N} \to \mathbb{N} \times \mathbb{N} \\
&\sigma_{2,1}^1(z) = d(z) - \sigma_{2,2}^1(z)
\end{aligned}$$

where

$$d(z) = min\{t \in \mathbb{N} \ : \ \sigma_1^2(t, 0) > z\} - 1$$

(See figure in Cantor pairing function Wikipedia article.)

Definition 11.1.3. Cantor tuple function (*cantorización de* $\mathbb{N}^{k+1}$)
Each Cantor tuple function establishes a biyective function between $\mathbb{N}$ and $\mathbb{N}^{k+1}, k \in \mathbb{N}_0$.

$$\begin{aligned}
&\sigma_1^{k+1} \ : \ \mathbb{N}^{k+1} \to \mathbb{N} \\
&\sigma_1^1(x) = x \\
&\sigma_1^{k+1}(x_1, \ldots, x_{k+1}) = \sigma_1^2\left(\sigma_1^k(x_1, \ldots, x_k), x_{k+1}\right)
\end{aligned}$$

$$\begin{aligned}
&\sigma_{k+1,p}^1 \ : \ \mathbb{N} \to \mathbb{N} \\
&\sigma_{k+1,p}^1(z) = \begin{cases} \sigma_{k,p}^1\left(\sigma_{2,1}^1(z)\right) & \text{if } 1 \leq p \leq k \\ \\ \sigma_{2,2}^1(z) & \text{if } p = k+1 \end{cases}
\end{aligned}$$

Remark. Cantor tuple is a family of countable infinite bijective functions.

Example 11.1.1. Cantor encoding and decoding

```
octave> cantorencoding(4, 5)
ans = 50

octave> cantordecoding(50,2)
ans =
4 5
```

Definition 11.1.4. Gödel encoding of $\mathbb{N}^*$ (*godelización de* $\mathbb{N}^*$)
Gödel encoding and decoding establishes a biyection between $\mathbb{N}$ and $\mathbb{N}^* = \bigcup_{n=0}^{\infty} \mathbb{N}^n$.

$$\Gamma \ : \ \mathbb{N}^* \to \mathbb{N}$$

$$\Gamma(\underline{x}) = \begin{cases} 0 & \text{if } |\underline{x}| = 0 \text{ (i.e. } x = ()) \\ \sigma_1^2\left(|\underline{x}| - 1, \sigma_1^{|\underline{x}|}(\underline{x})\right) + 1 & \text{if } |\underline{x}| > 0 \end{cases}$$

$$\gamma \ : \ \mathbb{N} \times \mathbb{N} \to \mathbb{N}$$

$$\gamma(z, k) = \begin{cases} 0 & \text{if } z = 0 \vee k > \lambda(z) \\ \sigma_{\lambda(z),k}^1\left(\sigma_{2,2}^1(z-1)\right) & \text{otherwise} \end{cases}$$

where $\lambda(z)$ refers to the length of the vector obtained from decoding z, namely

$$\lambda(z) = m \text{ iff } \Gamma(\underline{x}) = z \ \wedge \ \underline{x} \in \mathbb{N}^m$$

or, in terms of the previous encoding:

$$\lambda(z) = \begin{cases} 0 & \text{if } z = 0 \\ \sigma_{2,1}^1(z-1) + 1 & \text{if } z > 0 \end{cases}$$

Remark. Indeed, Gödel encoding enumerates $\mathbb{N}^*$:

0	1	2	3	4	5	6	7	8	9	10	$\cdots$
$()$	(0)	$(0,0)$	(1)	$(0,0,0)$	$(1,0)$	(2)	$(0,0,0,0)$	$(1,0,0)$	$(0,1)$	(3)	$\cdots$

11.1.1 Encoding of While programs

Definition 11.1.5. Encoding of While sentences
$sent2N:\ \mathrm{CODE}^1 \to \mathbb{N}$ with $\mathrm{CODE}^1 = \mathcal{L}\big((N, T, P, <\text{sentence}>)\big)$

$$sent2N(c) = \begin{cases} 5(i-1) & \text{if } c = \text{Xi} := 0 \\ 5\sigma_1^2(i-1, j-1) + 1 & \text{if } c = \text{Xi} := \text{Xj} \\ 5\sigma_1^2(i-1, j-1) + 2 & \text{if } c = \text{Xi} := \text{Xj + 1} \\ 5\sigma_1^2(i-1, j-1) + 3 & \text{if } c = \text{Xi} := \text{Xj - 1} \\ 5\sigma_1^2\big(i-1, code2N(b)\big) + 4 & \text{if } c = \text{while Xi} \neq 0 \text{ do} b \text{od} \end{cases}$$

Example 11.1.2. Encoding of While sentences

```
octave> sent2N('X2:=0')
ans = 5

octave> sent2N('X2:=X1+1')
ans = 7

octave> sent2N('while X2!=0 do X1:=0 od')
ans = 9
```

Definition 11.1.6. Encoding of While codes
Given $c = s_1; \ldots; s_m \in \text{CODE}$, where each s_i is a sentence,

$$code2N(c) = \Gamma\big(sent2N(s_1), \ldots, sent2N(s_m)\big) - 1$$

Definition 11.1.7. Encoding of While programs
Given $Q = (n, c) \in \text{WHILE}$,

$while2N : \text{WHILE} \to \mathbb{N}$
$while2N(Q) = \sigma_1^2\big(n, code2N(c)\big)$

Example 11.1.3. Encoding of a While code and program

```
octave> CODE2N('while X2!=0 do X1:=0 od')
ans = 54

octave> WHILE2N(1,'while X2!=0 do X1:=0 od')
ans = 1594
```

Example 11.1.4. Encoding of the double program

$double = (1, s)$

s :

```
    X2 := X1;
    while X2 ≠ 0 do
        X1 := X1 + 1;
        X2 := X2 - 1
    od
```

$code2N(s) \approx 1 \times 10^{39}$

$while2N(double) \approx 5 \times 10^{77}$

11.1.2 Decoding While programs

Definition 11.1.8. Type of sentence function
$senttype : \mathbb{N} \to \{0, 1, 2, 3, 4\}$
$senttype(z) = z \ mod \ 5$

Definition 11.1.9. LHS and RHS functions
$LHS, RHS : \mathbb{N} \to \mathbb{N}$

$$LHS(z) = \begin{cases} 1 + \frac{1}{5}z & \text{if } senttype(z) = 0 \\ 1 + \sigma^1_{2,1}\left(\frac{1}{5}(z - senttype(z))\right) & \text{if } senttype(z) \in \{1, 2, 3, 4\} \end{cases}$$

$$RHS(z) = 1 + \sigma^1_{2,2}\left(\frac{1}{5}(z - senttype(z))\right)$$

Definition 11.1.10. Decoding a While sentence
$N2sent : \mathbb{N} \to \text{CODE}$

$$N2sent(z) = \begin{cases} \text{Xi} \coloneqq 0 & \text{if } senttype(z) = 0 \\ \text{Xi} \coloneqq \text{Xj} & \text{if } senttype(z) = 1 \\ \text{Xi} \coloneqq \text{Xj} + 1 & \text{if } senttype(z) = 2 \\ \text{Xi} \coloneqq \text{Xj - 1} & \text{if } senttype(z) = 3 \\ \text{while Xi} \neq 0 \text{ do} N2code(j-1) \text{od} & \text{if } senttype(z) = 4 \end{cases}$$

where $i = LHS(z),\ j = RHS(z)$

Example 11.1.5. Decoding of While sentences

```
octave> N2sent(5)
ans = X2:=0

octave> N2sent(7)
ans = X2:=X1+1

octave> N2sent(9)
ans = while X2!=0 do X1:=0 od
```

Definition 11.1.11. Decoding While codes
$N2code : \mathbb{N} \to \text{CODE}$

$$N2code(z) = \begin{cases} N2sent\big(\gamma(z+1,1)\big) & \text{if } z = 0 \\ N2sent\big(\gamma(z+1,1)\big); \ldots ; N2sent\big(\gamma(z+1,\lambda(z+1))\big) & \text{if } z > 0 \end{cases}$$

Definition 11.1.12. Decoding While programs
$N2while : \mathbb{N} \to \text{WHILE}$
$N2while(z) = \big(\sigma^1_{2,1}(z), N2code(\sigma^1_{2,2}(z))\big)$

Example 11.1.6. Decoding of While codes and programs

```
octave> N2CODE(54)
ans = while X2!=0 do X1:=0 od

octave> N2WHILE(1594)
ans = (1, while X2!=0 do X1:=0 od)
```

Definition 11.1.13. REC^n
$\mathrm{REC}^n = \{f \in \mathrm{REC} \ : \ f : \mathbb{N}^n \to \mathbb{N}\}$.

Remark. REC^n is the set of REC functions with arity n.

Definition 11.1.14. $\mathrm{F(WHILE)}^n$
$\mathrm{F(WHILE)}^n = \{f \in \mathrm{F(WHILE)} \ : \ f = F_{(n,s)} \wedge (n, s) \in \mathrm{WHILE}\}$.

Definition 11.1.15. Indexing of REC
$h : \mathbb{N} \to \mathrm{F(WHILE)}$
$h(z) = F_{N2while(z)}$

Given the equivalence theorem, the function is also an indexing of REC.

Definition 11.1.16. Indexing of REC^n
$h : \mathbb{N} \to \mathrm{F(WHILE)}^n$
$h(z) = F_{(n,\ N2code(z))}$

Given the equivalence theorem, the function is also an indexing of REC^n.

11.2 Universal function

Definition 11.2.1. Universal function
Given a countable set of functions $\mathbf{F}$ verifying $f \in \mathbf{F} \Rightarrow f : \mathbb{N}^n \to \mathbb{N}$, $n \geq 0$,

$U[\mathbf{F}] : \mathbb{N}^{n+1} \to \mathbb{N}$ is universal for $\mathbf{F}$ iff

$$\exists\, h : \mathbb{N} \to \mathbf{F} \text{ that is an indexing of } \mathbf{F} \;:\; U[\mathbf{F}](i, \underline{x}) = h(i)(\underline{x})\ \forall i \in \mathbb{N} \ \wedge\ \forall \underline{x} \in \mathbb{N}^n.$$

Remark. $U[\mathbf{F}]$ is the universal function for $\mathbf{F}$ under the indexing h.

Theorem 11.2.1. $U[\mathrm{REC}^n] \in \mathrm{REC}^{n+1}$
Assuming

$$\begin{aligned}
&\exists\, U \in \mathrm{WHILE} \;\; : \;\; F_U = U[\mathrm{REC}^n] \Rightarrow \\
&U[\mathrm{REC}^n] \in \mathrm{F(WHILE)}^{n+1} \Rightarrow \\
&U[\mathrm{REC}^n] \in \mathrm{REC}^{n+1}
\end{aligned}$$

it has to be demonstrated that $\exists\, U \in \mathrm{WHILE} \;\; : \;\; F_U = U[\mathrm{REC}^n]$

Remark. U will be denoted as the universal program.

11.3 Universal program

Definition 11.3.1. Universal program

$U = (k+1, s)$

s :

```
(* X1 stores the code of the program *)
(* X2,...,Xk+1 store the arguments of the program *)
(* arguments are encoded in a single Gödel number *)
Xk+2 := godelk(X2,...,Xk+1);
(* variables after simulating the program *)
(* code in first argument, variables in second argument *)
Xk+2 := simulate(X1,Xk+2);
(* first variable is the output *)
X1 := degodel(Xk+2,1)
```

$simulate = (2, s)$

s :

```
X1 := X1 + 1;
while X1 ≠ 0 do
    (* extract first sentence and check type *)
    X3 := degodel(X1,1);
    (* check if assignment or loop *)
    if senttype(X3) ≤ 3 then
        (* change variable and remove *)
        X2 := execute(X3,X2);
        X1 := pop(X1)
    else
        (* value of control variable *)
        X4 := lhs(X3);
        if degodel(X2,X4) ≠ 0 then
            (* push body of the loop to top of the stack *)
            X4 := rhs(X3);
            X1 := merge(X1, X4)
        else
            (* remove sentence from the stack *)
            X1 := pop(X1)
        fi
    fi
od
(* return the encoded vector of variables *)
X1 := X2
```

$execute = (2, s)$

s :

```
(* X1 = sentences, X2 = variables *)
X3 := senttype(X1);
(* index of target variable *)
X4 := lhs(X1);
if X3 = 0 then
    X1 := replace(X2,X4,0)
else
    (* value of source variable *)
    X5 := rhs(X1);
    X5 := degodel(X2,X5);
    if X3 = 1 then
        X1 := replace(X2,X4,X5)
    fi
    if X3 = 2 then
        X5 := X5 + 1;
        X1 := replace(X2,X4,X5)
    fi
    if X3 = 3 then
        X5 := X5 - 1;
        X1 := replace(X2,X4,X5)
    fi
fi
```

$pop = (2, s)$

s :

```
(* X1 = sentences, X4 = number of sentences *)
X4 := length(X1);
(* return 0 if the program is just one sentence long *)
if X4 = 1 then
    X2 := 0
else
    (* godel of sentences except the first one *)
    X2 := godelk(degodel(X1,2));
    X4 := X4 - 1;
    X4 := X4 - 1;
    while X4 ≠ 0 do
        X2 := godelk(X2,degodel(X1,X3));
        X3 := X3 + 1;
        X4 := X4 - 1
    od
    X1 := X2
fi
```

$merge = (2, s)$

s :

```
(* X1 = sentences, X2 = sentences to move to the top of the stack *)
if length(X2) > 0 then
    X4 := godelk(degodel(X2,1));
    X3 := 2;
    X5 := length(X2);
    X5 := X5 - 1;
    while X5 ≠ 0 do
        X4 := godelk(X4,degodel(X2,X3));
        X3 := X3 + 1;
        X5 := X5 - 1
    od
    (* once all the sentences of X2 are on top, we introduce the rest of the program *)
    X5 := length(X1);
    X5 := X5 - 1;
    while X5 ≠ 0 do
        X4 := godelk(X4,degodel(X1,X3));
        X3 := X3 + 1;
        X5 := X5 - 1
    od
    X1 := X4
```

$replace = (3, s)$

s :

```
(* X1 = variables, X2 = variable index, X3 = value *)
X7 := 2;
if X2 = 1 then
    X6 := godelk(X3)
else
    X6 := godelk(degodel(X1,1))
fi
X4 := length(X1);
(* check how many variables are represented *)
if X4 >= X2 then
    X5 := X4
else
    X5 := X2
fi
X4 := X4 - 1;
while X5 ≠ 0 do
    (* fill with zeros when needed *)
    if X4 = 0 then
        if X7 = X2 then
            X6 := godelk(X6,X3)
        else
            X6 := godelk(X6,0)
        fi
    else
        if X7 = X2 then
```

Chapter 12

Theoretical limits of computing

12.1 Halting problem 196
12.2 The busy beaver function 199

12.1 Halting problem

Definition 12.1.1. Problem associated to a predicate

Problem		**Predicate**	**Function**
solvable	(*resoluble*)	decidable	$\in TREC$
partially solvable	(*parcialmente resoluble*)	enumerable	$\in REC$
unsolvable	(*irresoluble*)	undecidable	$\notin TREC$
totally unsolvable	(*totalmente irresoluble*)	not enumerable	$\notin REC$

Definition 12.1.2. H predicate
Given $Q \in$ WHILE, $g = while2N(Q)$, and $\underline{x} \in \mathbb{N}^*$,

$$H(g, \underline{x}) \Leftrightarrow F_Q(\underline{x}) \in \mathbb{N}$$

Remark. $H(g, \underline{x})$ is true iff Q halts with input $\underline{x}$.

Definition 12.1.3. H^n predicate
Given $Q \in$ WHILE : $F_Q \in \mathrm{F(WHILE)}^n$, $g = while2N(Q)$, and $\underline{x} \in \mathbb{N}^n$.

$$H^n(g, \underline{x}) \Leftrightarrow F_Q(\underline{x}) \in \mathbb{N}.$$

Remark. From now on, g will be referred as 'a program' (i.e. the program it encodes), or 'the number' of a program.

Theorem 12.1.1. Problem H^1 is not solvable

Proof. By contradiction:

$$H^1 \text{ solvable} \;\Rightarrow\; H^1 \text{ decidable} \;\Rightarrow\; \exists h^1 \in \text{TREC} \,:\, h^1 = X_{V_{H^1}}$$

$$h^1(g,x) = \begin{cases} 0 & \text{if } \overline{H^1(g,x)} \\ 1 & \text{if } H^1(g,x) \end{cases}$$

Let's consider the function

$$bomb(x) = \begin{cases} 1 & \text{if } h^1(x,x) = 0 \\ \uparrow & \text{if } h^1(x,x) = 1 \end{cases}$$

$Bomb = (1, s)$

s :

```
while H1(X1,X1) ≠ 0 do
    X1 := X1
od
X1 := X2 + 1;
```

If $b = while2N(Bomb)$ then

$h^1(b,b) = 1 \;\Rightarrow\; bomb(b) = \uparrow \;\Rightarrow\; h^1(b,b) = 0 \quad \Rightarrow\Leftarrow$

$h^1(b,b) = 0 \;\Rightarrow\; bomb(b) = 1 \;\Rightarrow\; h^1(b,b) = 1 \quad \Rightarrow\Leftarrow$

$\square$

Corollary 12.1.1. H is not solvable

Remark. H^1 not solvable $\Rightarrow H^n$ not solvable $\Rightarrow H$ not solvable.

Proposition 12.1.1. H^1 is partially solvable

Proof. Let's consider
$f : \mathbb{N}^2 \to \mathbb{N}$
$f = \sigma\left(U[\mathrm{REC}^1]\right)$

$$P_f = H^1 \underset{\substack{\uparrow \\ f \in \mathrm{REC}}}{\Rightarrow} H^1 \in \mathrm{PRED(REC)} \quad \Rightarrow \quad H^1 \text{ is partially solvable.}$$

□

Remark. Being H^1 enumerable means that all the pairs $(g, x) \in \mathbb{N} : F_{N2while(g)}(x) \in \mathbb{N}$ can be generated. This can be done by simulating each program with a given input for a number of steps, and checking if it has halted.

Corollary 12.1.2. H is partially solvable

Remark. H^1 partially solvable $\Rightarrow H^n$ partially solvable $\Rightarrow H$ partially solvable.

12.2 The busy beaver function

Definition 12.2.1. Length of a While program (*longitud de un programa While*)
$length : \text{WHILE} \to \mathbb{N}$
$length\big((n, s)\big) = |s|_{do} + |s|_{:=}$

Definition 12.2.2. WHILE^n
$\text{WHILE}^n = \{Q \in \text{WHILE} \,:\, length(Q) = n\}$

Remark. WHILE^n is the set of the While programs of lenght n.

Definition 12.2.3. Busy beaver function (Σ)
$\Sigma : \mathbb{N} \to \mathbb{N}$
$\Sigma(n) = max\{F_Q(0) \,:\, F_Q \in \text{F(WHILE)}^1 \wedge Q \in \text{WHILE}^n \wedge F_Q(0) \in \mathbb{N}\}$

Proposition 12.2.1. $\Sigma(n+1) > \Sigma(n)$

Proof. Let's consider $Q = (1, s) \in \text{WHILE}^n \ \wedge \ F_Q(0) = \Sigma(n)$, then

$Q' = (1, s')$
s' :

s ;
X1 := X1 + 1

$$length(Q') = n + 1 \wedge F_{Q'}(0) = \Sigma(n) + 1 \ \Rightarrow$$
$$\Sigma(n+1) \geq \Sigma(n) + 1 > \Sigma(n) \quad \Rightarrow \ \Sigma(n+1) > \Sigma(n)$$

□

Proposition 12.2.2. $\exists Q \in \text{WHILE}^k \ : \ f = F_Q \Rightarrow \Sigma(n+k) \geq f(n)$

Proof. Let's consider $Q = (1, s) \in \text{WHILE}^k \wedge F_Q = f$, then

$Q' = (1, s')$
s' :

$$\left.\begin{array}{l} \text{X1} := \text{X1} + 1; \\ \quad \cdots \\ \text{X1} := \text{X1} + 1; \end{array}\right\} n \text{ times}$$

s

$$Q' \in \text{WHILE}^{n+k} \wedge F_{Q'}(0) = f(n) \ \Rightarrow \ \Sigma(n+k) \geq f(n)$$

□

Theorem 12.2.1. $\Sigma \notin \mathrm{F(WHILE)}$

Proof. By contradiction: let's consider

$$\left.\begin{array}{r} \Sigma \in \mathrm{F(WHILE)} \\ \text{let } g(n) = 2n \underset{\substack{\uparrow \\ \text{example 9.1.2}}}{\Rightarrow} g \in \mathrm{F(WHILE)} \\ \text{let } f(n) = \Sigma(2n) \end{array}\right\} \Rightarrow f \in \mathrm{F(WHILE)} \Rightarrow$$

$$\exists Q \in \mathrm{WHILE}^k : F_Q = f \underset{\substack{\uparrow \\ \text{proposition 12.2.2}}}{\Rightarrow} \Sigma(n+k) \geq f(n),\ \forall n \in \mathbb{N} \underset{\substack{\uparrow \\ f(n)=\Sigma(2n)}}{\Rightarrow}$$

$$\Sigma(n+k) \geq \Sigma(2n),\ \forall n \in \mathbb{N} \underset{\substack{\uparrow \\ n=k+1}}{\Rightarrow} \Sigma(2k+1) \geq \Sigma(2k+2) \underset{\substack{\uparrow \\ \text{proposition 12.2.1}}}{\Rightarrow\Leftarrow}$$

□

Proposition 12.2.3. $\Sigma \notin \mathrm{F(WHILE)} \Rightarrow H^1$ is not solvable

Proof. By contradiction: let's consider $\Sigma \notin \mathrm{F(WHILE)} \wedge H^1$ is solvable.
Since there is a finite number of (equivalent) programs of a given length, then the program that halts (with zero input) for each length can be found, and so the biggest output.
This implies $\Sigma \in \mathrm{F(WHILE)} \quad \Rightarrow\Leftarrow$

□

Appendices

.1 Proposed solutions to the practices

Solution to Practice 1.2.3:
Given $A = \{1, 2, 3\}$, propose

i. a binary relation on A

   $R = \{(1, 1), (1, 2), (1, 3)\}$

ii. a relation between A and $\mathbb{N}$

   $R = \{(2, 103), (1, 90378342)\}$

iii. a function on A

   $R = \{(1, 1), (2, 3), (3, 3)\}$

iv. a biyective function between A and $\mathbb{N}$.

   A and $\mathbb{N}$ are not equipotent, so a biyection can not be stablished between them.

Solution to Practice 1.2.4:
Given $R = \{(1,2),(2,2),(3,4),(2,1)\}$,

i. is R antisymmetric?

   No, because R contains the pairs $(1,2)$ and $(2,1)$.

ii. is R transitive?

   No, because R contains the pairs $(1,2)$ and $(2,1)$, but not the pair $(1,1)$.

iii. find the symmetric closure of R

   $R \cup \{(4,3)\}$

Solution to Practice 1.3.1:

Given $R = \{(1,2),(2,3),(3,4),(4,5)\}$, solve

i. R^2

$R^2 = \{(1,3),(2,4),(3,5)\}$

ii. R^3

$R^3 = \{(1,4),(2,5)\}$

iii. R^{-1}

$R^2 = \{(2,1),(3,2),(4,3),(5,4)\}$

iv. $R^{2^{-1}}$

$R^2 = \{(3,1),(4,2),(5,3)\}$

v. what can you conclude from the last result?

$R^{2^{-1}} = R^{-1^2}$

Solution to Practice 1.3.2:

Answer these questions:

i. is countable the set of the odd numbers?

   Yes, because we can establish a biyection between them $(2n - 1)$.

ii. is countable the set of the prime numbers?

   Yes, because any subset of a countable set is countable.

   It is actually countably infinite, since it has been proven to be infinite, and the n-th prime can be computed by iterating through natural numbers, checking for primality, and counting primes until the n-th has been reached.

iii. is countable $\mathbb{Q}$?

   Yes, because the cartesian product of two countable sets is countable.

.2 Proof of closure properties for regular languages

Closure properties are really important if we want to construct complex regular languages. The set of languages accepted by regular grammars is closed for the operations of union, concatenation, Kleene star, complement, intersection, power and reverse.

Proposition .2.1. $\mathscr{L}.3$ is closed under the union operation

Proof. Given two regular grammars $G_1 = (N_1, T_1, P_1, S_1)$ and $G_2 = (N_2, T_2, P_2, S_2)$, such that $N_1 \cap N_2 = \emptyset$, a new regular grammar $G = (N, T, P, S)$ can be defined as:
$N = N_1 \cup N_2 \cup \{S\}$
$T = T_1 \cup T_2$
$P = P_1 \cup P_2 \cup \{(S, S_1), (S, S_2)\}$
$S \notin N_1 \cup N_2$
such that $\mathcal{L}(G) = \mathcal{L}(G_1) \cup \mathcal{L}(G_2)$.

Let's see that this grammar generates all and only the strings of the union set.

1. $w \in \mathcal{L}(G_1) \cup \mathcal{L}(G_2) \Rightarrow S_1 \Rightarrow^* w \vee S_2 \Rightarrow^* w$

   and either $S \Rightarrow S_1 \Rightarrow^* w$ or $S \Rightarrow S_2 \Rightarrow^* w$, so $w \in \mathcal{L}(G)$

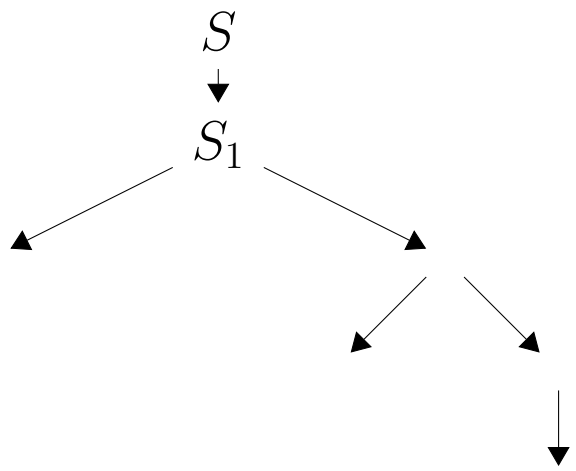
2. $w \in \mathcal{L}(G) \Rightarrow S \Rightarrow S_1 \Rightarrow^* w \vee S \Rightarrow S_2 \Rightarrow^* w$

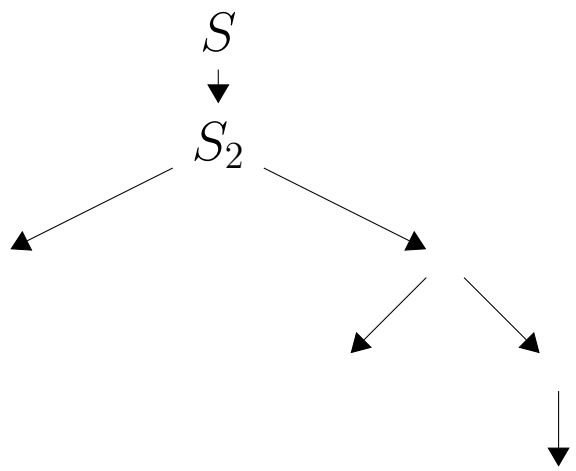
   then $w \in \mathcal{L}(G_1) \cup \mathcal{L}(G_2)$.

$\square$

To build G, we create a new axiom S, and we link it with the starting symbols of G_1 and G_2, adding (S, S_1) and (S, S_2) to P.

These are two derivation trees of G:





Proposition .2.2. $\mathscr{L}.3$ is closed under the concatenation operation

Proof. Given two regular grammars $G_1 = (N_1, T_1, P_1, S_1)$ and $G_2 = (N_2, T_2, P_2, S_2)$ such that $N_1 \cap N_2 = \emptyset$, a new regular grammar $G = (N, T, P, S)$ can be defined as:
$N = N_1 \cup N_2$
$T = T_1 \cup T_2$
$P = P_1 \cup P_2 \cup \{(S, S_1S_2)\}$
$S \notin N_1 \cup N_2$
such that $\mathcal{L}(G) = \mathcal{L}(G_1) \cdot \mathcal{L}(G_2)$

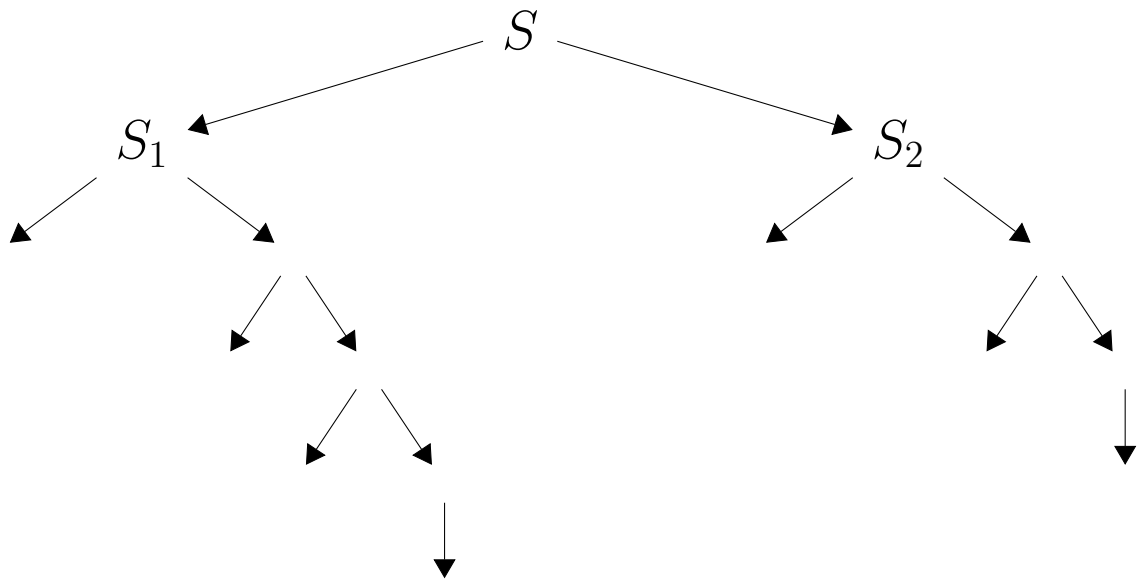
Let's see that this grammar generates all and only the strings of the concatenation set.

1. $w \in \mathcal{L}(G_1) \cdot \mathcal{L}(G_2) \Rightarrow w = xy \wedge S_1 \Rightarrow^* x \wedge S_2 \Rightarrow^* y$
   since $S \Rightarrow S_1S_2 \Rightarrow^* xy$, then $w \in \mathcal{L}(G)$
2. $w \in \mathcal{L}(G) \Rightarrow S \Rightarrow S_1S_2 \Rightarrow^* xy = w$, where $x \in \mathcal{L}(G_1) \wedge y \in \mathcal{L}(G_2)$
   then $w \in \mathcal{L}(G_1) \cdot \mathcal{L}(G_2)$.

$\square$

To build G, we have to add the rule (S, S_1S_2) to P, so we can concatenate strings generated by G_1 and G_2.

This is a derivation tree of G:



Proposition .2.3. $\mathscr{L}.3$ is closed under the Kleene star operation

Proof. Given a left regular grammar $G_1 = (N_1, T_1, P_1, S_1)$, a new regular grammar $G = (N, T, P, S)$ can be defined as:
$N = N_1$
$T = T_1$
$P = P_1 \cup \{(A, aS) : (A, a) \in P_1, A \in N_1, a \in T_1\}$
$S = S_1$
such that $\mathcal{L}(G) = \mathcal{L}(G_1)^* - \{\varepsilon\}$

Let's see that this grammar generates all and only the strings of the Kleene star set.

1. $w^* \in \mathcal{L}(G_1)^* \Rightarrow S_1 \Rightarrow^* w$

   since $S \Rightarrow^* w$ and $S \Rightarrow^* wS$, then $w^* \in \mathcal{L}(G)$

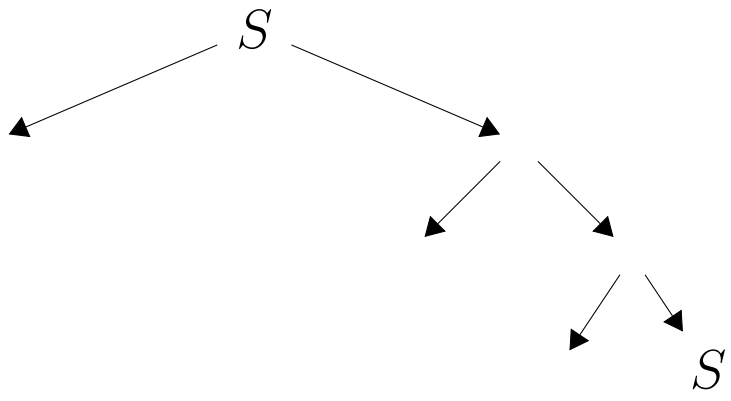
2. $w^* \in \mathcal{L}(G) \Rightarrow S \Rightarrow^* w \wedge S \Rightarrow^* wS$

   then $w^* \in \mathcal{L}(G_1)^*$.

$\square$

To build G, we make S_1 the axiom, and we add to every single Terminal-Regular Rule of G_1 a link to the axiom.

This is a derivation tree of G:



Proposition .2.4. $\mathscr{L}.3$ is closed under the complement operation

Proof. Given a left regular grammar $G_1 = (N_1, T_1, P_1, S_1)$, a new regular grammar $G = (N, T, P, S)$ can be defined as:
$N = N_1 \cup S'$ where $S' \notin N_1$
$T = T_1$
$P = (P_1 - R) \cup \{N_1 \times T_1 - R\} \cup$
$\cup \{(A, aS') : (A, aB) \notin P_1, A \in N_1, B \in N_1, a \in T_1\} \cup$
$\cup \{(S', a) : a \in T_1\} \cup \{(S', aA) : A \in N_1, a \in T_1\}$
$S = S_1$
such that $\mathcal{L}(G) = \overline{\mathcal{L}(G_1)} - \{\varepsilon\}$

Where $R = \{(A, a) : (A, a) \in P_1, A \in N_1, a \in T_1\}$ and it makes reference to the Terminal-Regular rules. $\square$

This proof will be better explained with the following example:

Let $G = (\{S\}, \{a, b\}, \{S \to aS|b\}, S)$
$\mathcal{L}(G) = \{b, ab, aab, aaab, ...\}$

These sets are the ones defined previously:
$R = \{S \to b\}$
$\{N_1 \times T_1 - R\} = \{S \to a\}$
$\{(A, aS') : (A, aB) \notin P_1, A \in N_1, B \in N_1, a \in T_1\} = \{S \to bS'\}$
$\{(S', a) : a \in T_1\} \cup \{(S', aA) : A \in N_1, a \in T_1\} = \{S' \to aS'|bS'|a|b\}$

Now we can define G'
$G' = (\{S, S'\}, \{a, b\}, \{S \to aS|bS'|a, S' \to aS'|bS'|a|b\}, S)$
$\mathcal{L}(G') \in \mathscr{L}.3$
$\mathcal{L}(G') = \{a, aabb, bbba, aaaaaabb...\}$

G' never generates strings that are in $\mathcal{L}(G)$, it generates strings that are in $\Sigma^+ - \mathcal{L}(G)$, so we conclude that $\mathcal{L}(G') = \overline{\mathcal{L}(G)} - \{\varepsilon\}$.

Proposition .2.5. $\mathscr{L}.3$ is closed under the intersection operation

Proof. Let G_1 and G_2 be regular grammars, being $L_1 = \mathcal{L}(G_1)$ and $L_2 = \mathcal{L}(G_2)$.

$$\mathcal{L}(G) = \mathcal{L}(G_1) \cap \mathcal{L}(G_2) = (\overline{\overline{L_1} \cup \overline{L_2}})$$

We have proven that regular languages are closed for union and complement so:
$\exists\ G\ \textit{regular grammar} \mid \mathcal{L}(G) = \mathcal{L}(G_1) \cap \mathcal{L}(G_2)$

□

Proposition .2.6. $\mathscr{L}.3$ is closed under the power operation

Proof. Let G_1 be a regular grammar, being $L_1 = \mathcal{L}(G_1)$.

$$L_1^n = \begin{cases} \{\varepsilon\} & \text{if } n = 0 \\ L_1^{n-1} \cdot L_1 & \textit{otherwise} \end{cases}$$

We have proven that regular languages are closed for concatenation so:
$\exists\ G\ \textit{regular grammar} \mid \mathcal{L}(G) = \mathcal{L}(G_1)^n$

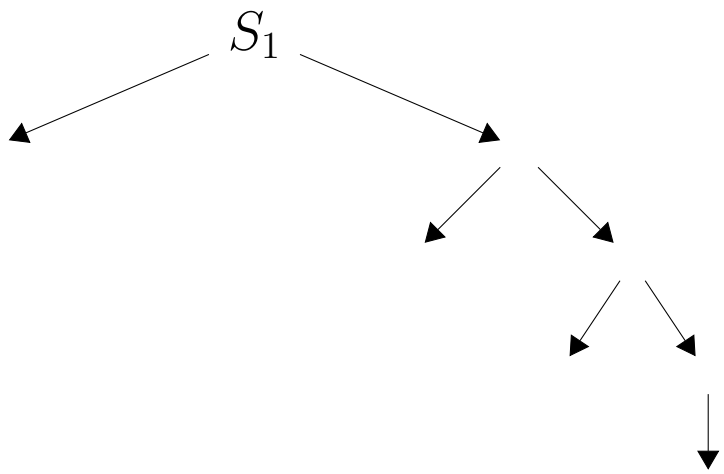
□

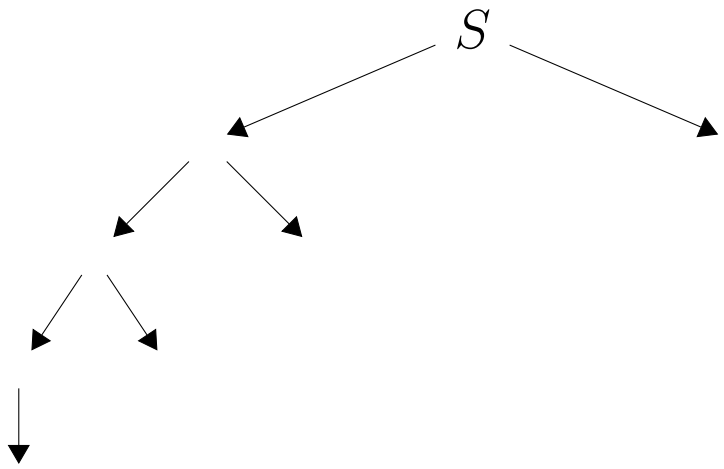
Proposition .2.7. $\mathscr{L}.3$ is closed under the reverse operation

Proof. Given a left regular grammar $G_1 = (N_1, T_1, P_1, S_1)$, a new regular grammar $G = (N, T, P, S)$ can be defined as:
$N = N_1$
$T = T_1$
$P = \{(A, a) : (A, a) \in P_1, A \in N_1, a \in T_1\} \cup$
$\cup \{(A, bB) : (A, Bb) \in P_1, A \in N_1, B \in N_1, b \in T_1\}$
$S = S_1$
such that $\mathcal{L}(G) = \mathcal{L}(G_1)^R$

$\square$

To build G, we have to switch the non-terminal element and the terminal one in every non-terminal rule.
Instead of generating strings from the left, we generate them from the right.

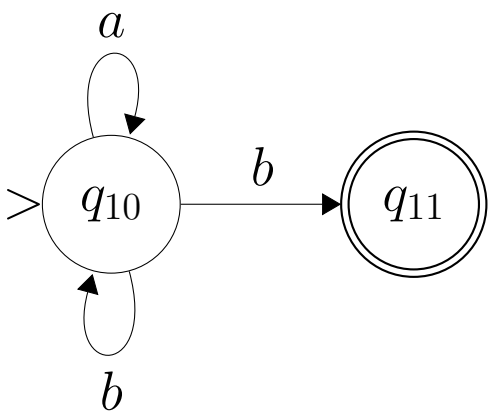




.3 Examples of closure properties with NDFA

Example .3.1. Union

Let $M_1 = (K_1, \Sigma_1, \Delta_1, s_1, F_1)$ be an NFA with:
$K_1 = \{q_{10}, q_{11}\}$
$\Sigma_1 = \{a, b\}$
$\Delta_1 = \{(q_{10}, a, q_{10}), (q_{10}, b, q_{10}), (q_{10}, b, q_{11})\}$
$s_1 = q_{10}$
$F_1 = \{q_{11}\}$



we have that $L(M_1) = (a + b)^* b$

Let $M_2 = (K_2, \Sigma_2, \Delta_2, s_2, F_2)$ be an NFA with:

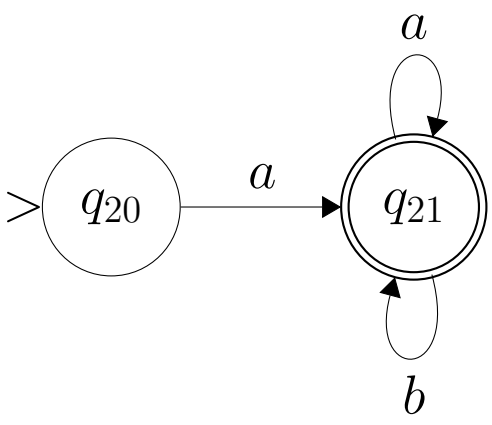
$K_2 = \{q_{20}, q_{21}\}$

$\Sigma_2 = \{a, b\}$

$\Delta_2 = \{(q_{20}, a, q_{21}), (q_{21}, a, q_{21}), (q_{21}, b, q_{21})\}$

$s_2 = q_{20}$

$F_2 = \{q_{21}\}$

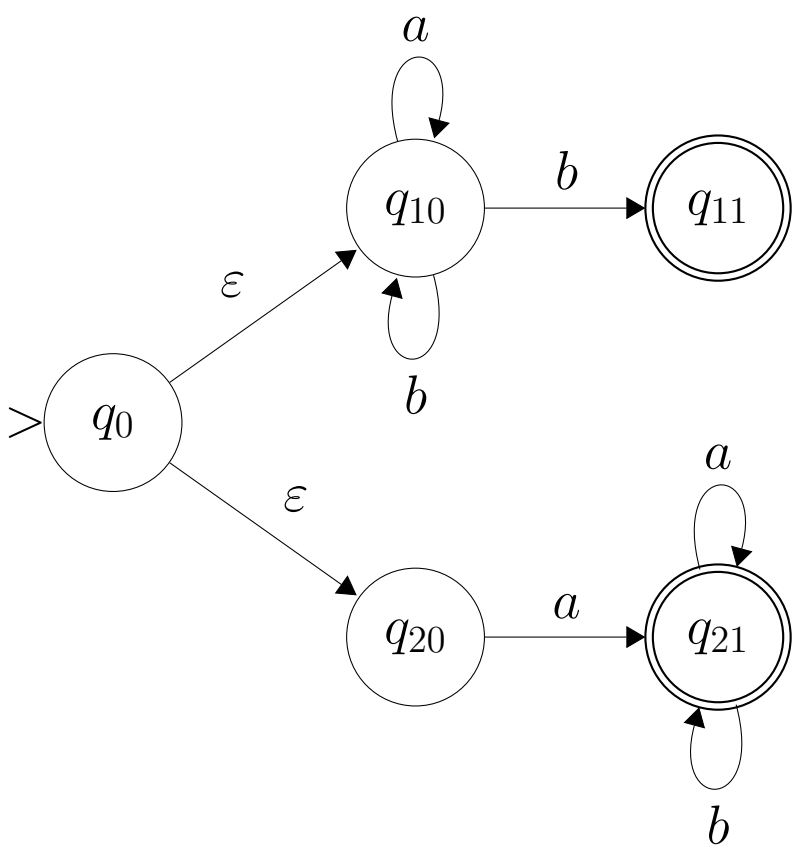


we have that $L(M_2) = a(a + b)^*$

To obtain $M \quad NFA \quad | \quad L(M) = L(M_1) \cup L(M_2)$ we apply the algorithm this way:

We build $M = (K, \Sigma, \Delta, s, F)$ with:

$$K = \{q_0, q_{10}, q_{11}, q_{20}, q_{21}\}$$
$$\Sigma = \{a, b\}$$
$$\Delta = \{(q_0, \varepsilon, q_{10}), (q_0, \varepsilon, q_{20}), (q_{10}, a, q_{10}), (q_{10}, b, q_{10}), (q_{10}, b, q_{11}), (q_{20}, a, q_{21}), (q_{21}, a, q_{21}), (q_{21}, b, q_{21})\}$$
$$s = q_0$$
$$F = \{q_{11}, q_{21}\}$$



resulting in $L(M) = L(M_1) \cup L(M_2) = (a+b)^*b + a(a+b)^*$

Example .3.2. Concatenation

Be M_1 and M_2 the NFA from the previous example.

To obtain $M \quad NFA \quad | \quad L(M) = L(M_1) \cdot L(M_2)$ we apply the algorithm this way:

We build $M = (K, \Sigma, \Delta, s, F)$ with:

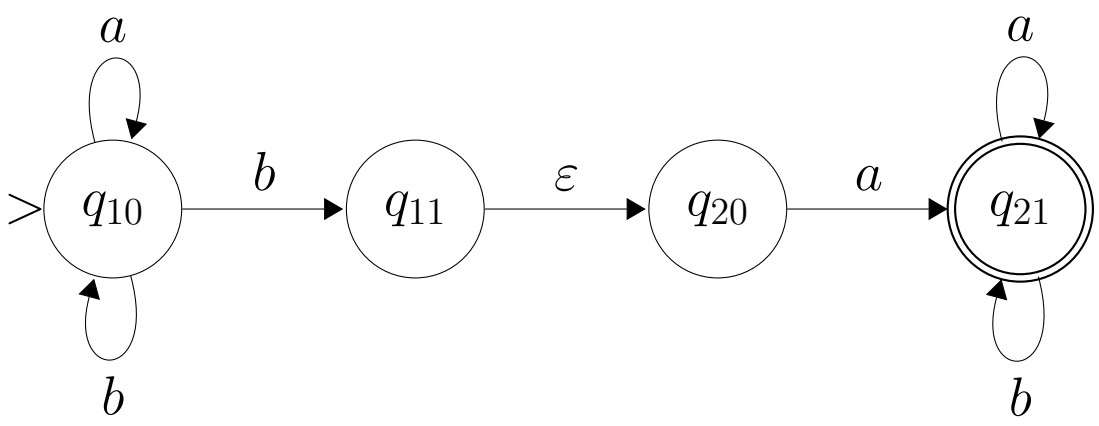
$K = \{q_{10}, q_{11}, q_{20}, q_{21}\}$

$\Sigma = \{a, b\}$

$\Delta = \{(q_{10}, a, q_{10}), (q_{10}, b, q_{10}), (q_{10}, b, q_{11}), (q_{11}, \varepsilon, q_{20}), (q_{20}, a, q_{21}), (q_{21}, a, q_{21}), (q_{21}, b, q_{21})\}$

$s = q_{10}$

$F = \{q_{21}\}$



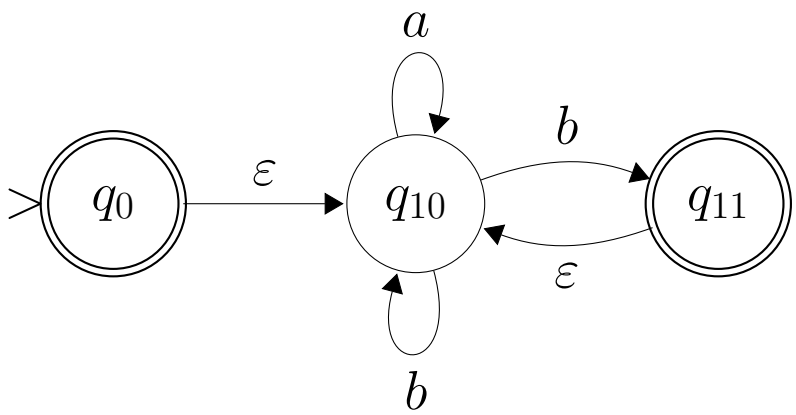
resulting in $L(M) = L(M_1) \cdot L(M_2) = (a + b)^* ba(a + b)^*$

Example .3.3. Kleene closure

Be M_1 the NFA from the first example.

To obtain $M \quad NFA \quad | \quad L(M) = L(M_1)^*$ we apply the algorithm this way:

We build $M = (K, \Sigma, \Delta, s, F)$ with:
$K = \{q_0, q_{10}, q_{11}\}$
$\Sigma = \{a, b\}$
$\Delta = \{(q_0, \varepsilon, q_{10}), (q_{10}, a, q_{10}), (q_{10}, b, q_{10}), (q_{10}, b, q_{11}), (q_{11}, \varepsilon, q_{10})\}$
$s = q_0$
$F = \{q_0, q_{11}\}$



resulting in $L(M) = L(M_1)^* = ((a + b)^*b)^* = (a + b)^*b + \varepsilon$

Example .3.4. Complement

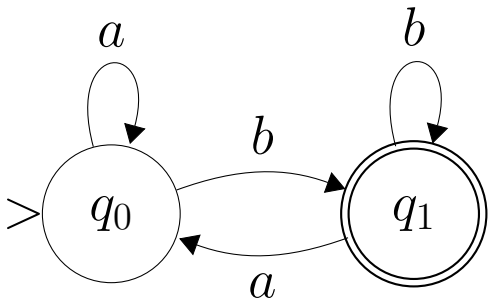
Be M_1 the NFA from the first example.

To obtain the complement of M_1, first we need to get a $DFA \quad M \quad | \quad M \equiv M_1$, that we will obtain applying an algorithm, getting the following DFA:

Be $M = (K, \Sigma, \delta, s, F)$ with:
$K = \{q_0, q_1\}$
$\Sigma = \{a, b\}$

$\delta(q, \sigma)$	a	b
q_0	q_0	q_1
q_1	q_0	q_1

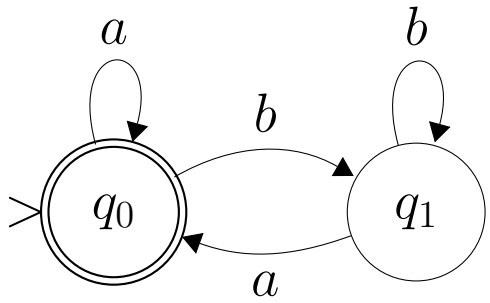
$s = q_0$
$F = \{q_1\}$



resulting in $M \equiv M_1$.

After this, we apply another algorithm to M to obtain $\overline{M} \quad DFA \quad | \quad L(\overline{M}) = \Sigma^* - L(M)$, this way:

We build $\overline{M} = (K, \Sigma, \delta, s, K - F)$



resulting in $L(\overline{M}) = \Sigma^* - L(M) = \Sigma^* - L(M_1) = (a + b)^* a + \varepsilon$

Example .3.5. Intersection

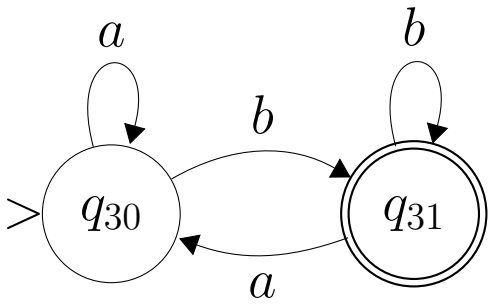
Be M_1 and M_2 the NFA from the first example.

To obtain $M \quad DFA \quad | \quad L(M) = L(M_1) \cap L(M_2)$, we apply the algorithm this way:

1º/ We apply the conversion algorithm from NFA to DFA to M_1 and M_2, getting:
$M_3 = (K_3, \Sigma_3, \delta_3, s_3, F_3)$ with:
$K_3 = \{q_{30}, q_{31}\}$
$\Sigma_3 = \{a, b\}$

$\delta_3(q, \sigma)$	a	b
q_{30}	q_{30}	q_{31}
q_{31}	q_{30}	q_{31}

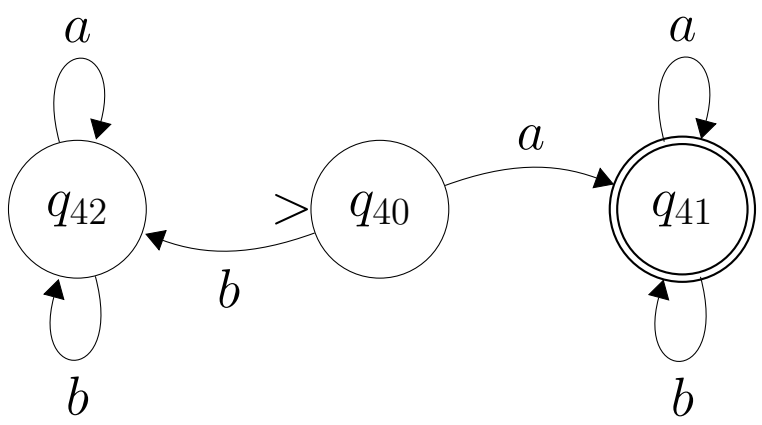
$s_3 = q_{30}$
$F_3 = \{q_{31}\}$



and
$M_4 = (K_4, \Sigma_4, \delta_4, s_4, F_4)$ with:
$K_4 = \{q_{40}, q_{41}, q_{42}\}$
$\Sigma_4 = \{a, b\}$

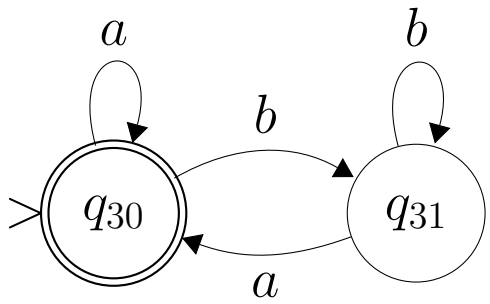
$\delta_4(q, \sigma)$	a	b
q_{40}	q_{41}	q_{42}
q_{41}	q_{41}	q_{41}
q_{42}	q_{42}	q_{42}

$s_4 = q_{40}$
$F_4 = \{q_{41}\}$

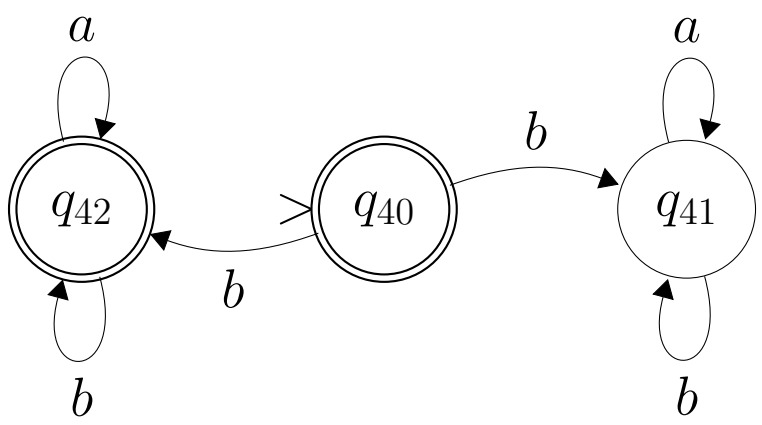


resulting in $M_3 \equiv M_1$ and $M_4 \equiv M_2$.

2º/ We apply the algorithm to M_3 and M_4, to obtain their complements, M_5 and M_6 respectively, this way:
$M_5 = (K_3, \Sigma_3, \delta_3, s_3, K_3 - F_3)$

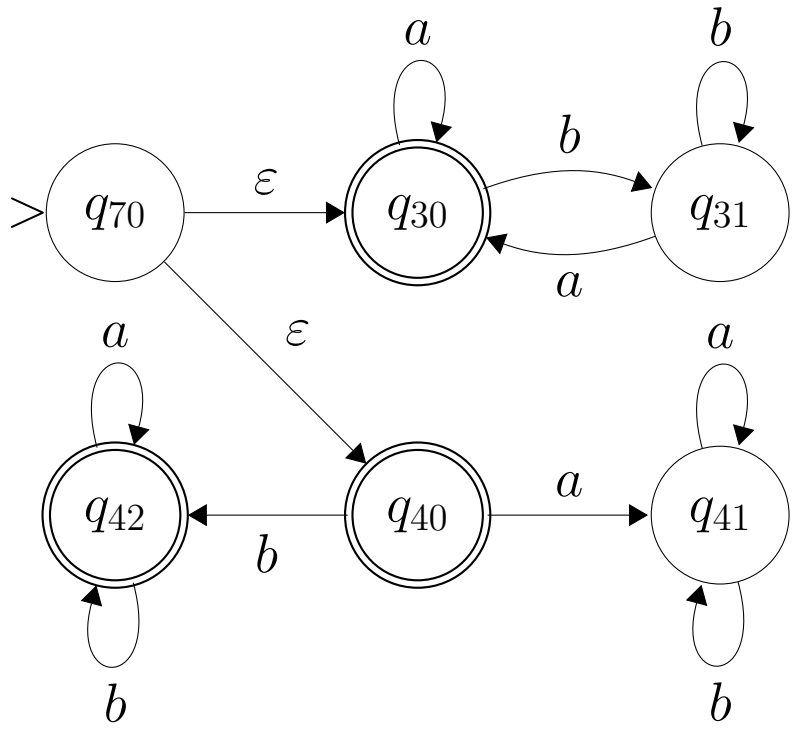


and
$M_6 = (K_4, \Sigma_4, \delta_4, s_4, K_4 - F_4)$



resulting in
$L(M_5) = \Sigma^* - L(M_3) = \Sigma^* - L(M_1) = (a+b)^*a + \varepsilon$
and in $L(M_6) = \Sigma^* - L(M_4) = \Sigma^* - L(M_2) = b(a+b)^* + \varepsilon$

3º/ We apply another algorithm to M_5 and M_6, to obtain M_7 NFA such that $L(M_7) = L(M_5) \cup L(M_6)$, thus:
$M_7 = (K_7, \Sigma_7, \Delta_7, s_7, F_7)$ with:
$K_7 = \{q_{30}, q_{31}, q_{40}, q_{41}, q_{42}, q_{70}\}$
$\Sigma_7 = \{a, b\}$
$\Delta_7 = \{(q_{30}, a, q_{30}), (q_{30}, b, q_{31}), (q_{31}, a, q_{30}), (q_{31}, b, q_{31}), (q_{40}, a, q_{41}), (q_{40}, b, q_{42}),$
$(q_{41}, a, q_{41}), (q_{41}, b, q_{41}), (q_{42}, a, q_{42}), (q_{42}, b, q_{42}), (q_{70}, \varepsilon, q_{30}), (q_{70}, \varepsilon, q_{40})\}$
$s_7 = q_{70}$
$F_7 = \{q_{30}, q_{40}, q_{42}\}$



resulting in $L(M_7) = L(M_5) \cup L(M_6) = (a+b)^*a + b(a+b)^* + \varepsilon$

4º/ We apply the algorithm to convert NFA to DFA to M_7 to obtain $M_8 | M_8 \equiv M_7$, thus:

$M_8 = (K_8, \Sigma_8, \delta_8, s_8, F_8)$ with:

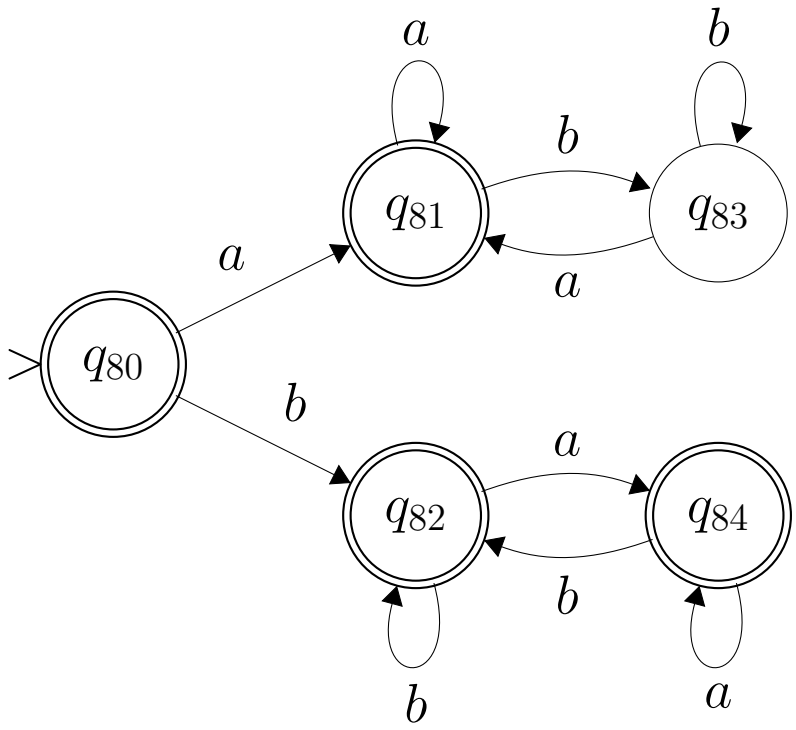
$K_8 = \{q_{80}, q_{81}, q_{82}, q_{83}, q_{84}\}$

$\Sigma_8 = \{a, b\}$

$s_8 = q_{80}$

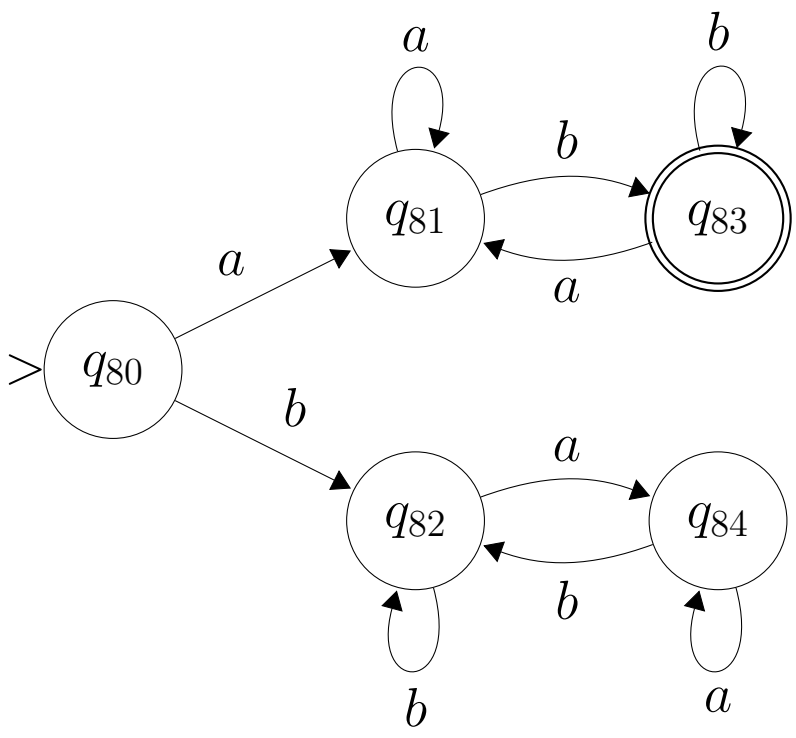
$F_8 = \{q_{80}, q_{81}, q_{82}, q_{84}\}$

$\delta_8(q,\sigma)$	a	b
q_{80}	q_{81}	q_{82}
q_{81}	q_{81}	q_{83}
q_{82}	q_{84}	q_{82}
q_{83}	q_{81}	q_{83}
q_{84}	q_{84}	q_{82}



resulting in $M_8 \equiv M_7$.

5º/ Finally we apply another algorithm to M_8 to obtain its complement, thus:
$M = (K_8, \Sigma_8, \delta_8, s_8, K_8 - F_8)$



resulting in $L(M) = \Sigma^* - L(M_8) = L(M_1) \cap L(M_2) = a(a+b)^*b$

.4 $L \in \mathscr{L}.2 \Rightarrow L \in \mathscr{L}(\mathbf{NPA})$

This is one of the two implications to proof 6.6.1 (i.e. $\mathscr{L}.2 = \mathscr{L}(\text{NPA})$), namely, that $\mathscr{L}(\text{NPA}) \subseteq \mathscr{L}.2$.
This is proofed by a conclusive algorithm that, starting from a CFG G, it obtains an NPA wich accepts the language $\mathcal{L}(G)$.

.5 F(WHILE) to REC proof

Definition .5.1. Piecewise-defined function (*Función definida por casos*)
Let $k \in \mathbb{N}, k > 0$.
Let $g_1, \ldots g_q$ be functions of k arguments.
Let $P_1, \ldots, P_q$ be predicates of k arguments such that:
$\forall \underline{n} \in \mathbb{N}^k \; \exists! \; i \in \{1, \ldots, q\} \mid P_i(\underline{n})$
If the function $f : \mathbb{N}^k \to \mathbb{N}$ is

$$f(\underline{n}) = g_i(\underline{n}) \text{ if } P_i(\underline{n})$$

then f is a piecewise-defined function from $g_1, \ldots, g_q$ and $P_1, \ldots, P_q$.

Proposition .5.1.
If f is the piecewise-defined function from $g_1, \ldots, g_q$ and $P_1, \ldots, P_q$,

$g_1, \ldots, g_q \in \mathrm{REC},$
$P_1, \ldots, P_q \in \mathrm{PRED(TREC)}$
$\Rightarrow f \in \mathrm{REC}$

Proof.
As $P_i \in \text{PRED(TREC)}\ \forall i$, then $\exists\ p_1, \ldots, p_q \in \text{TREC}$ such that $P_{p_i} = P_i\ \forall i$
Let define, given $i \in \{1, \ldots, q\}$, the function

$$h_i = \left\langle \theta^k \mid g_i(\pi_1^{k+2}, \ldots, \pi_k^{k+2}) \right\rangle (\pi_1^k, \ldots, \pi_k^k, p_i)$$

where θ^k is the constant zero function of k arguments, that can be defined as:

$$\theta^k = \left\langle \theta \mid \pi_2^2 \right\rangle (\pi_1^k)$$

Then, $f = add^q(h_1, \ldots, h_q)$, where add^q is the function that gives the sum of q arguments.
Finally, we have to see that $add^q \in \text{REC}$.
Indeed, given $q > 2$, we can define:

$$add^q = add(\pi_1^q, add^{q-1}(\pi_2^q, \ldots, \pi_q^q))$$

and $add^2 = add \in \text{REC}$. □

Note that, except for g_i and h_i functions, all the functions used in the proof are in TREC. Then, if $g_1, \ldots, g_q \in \text{TREC}$, we have that $h_1, \ldots, h_q \in \text{TREC}$ and then $f \in \text{TREC}$.

Definition .5.2. Extract function (*Función extraer*)
Let $Q = (n, s) \in \text{WHILE}$,
$p = max\,\{m \in \Sigma_d^+ \,:\, \text{Xm} \prec s\}$ (also in further definitions).
$extr_Q : \mathbb{N}^2 \to \mathbb{N}$

$$extr_Q(z, i) = \begin{cases} \sigma^1_{p+1,i}(z) & \text{if } 1 \leq i \leq p+1 \\ 0 & \text{otherwise} \end{cases}$$

Proposition .5.2. $extr_Q \in \text{REC}$

Proof. Given $Q = (n, s) \in \text{WHILE}$, we have that $extr_Q$ is a piecewise-defined function from functions $\sigma^1_{p+1,1}(\pi^2_1), \ldots, \sigma^1_{p+1,p+1}(\pi^2_1), \theta^2$ and predicates $P_1, \ldots, P_{p+1}, P_{distinct}$, where given $i \in \{1, \ldots, p+1\}$, P_i is the predicate of two arguments that determines if the second argument is equal to i, and $P_{distinct} = \neg(P_1 \vee \cdots \vee P_{p+1})$.
As this functions are recursive and the predicates are decidable, $extr_Q$ is also recursive.

□

Definition .5.3. Replace function (*Función reemplazar*)
Let $Q = (n, s) \in$ WHILE.
$replace_Q : \mathbb{N}^3 \to \mathbb{N}$

$$replace_Q(z, k, x) = \begin{cases} \sigma_1^{p+1}(extr_Q(z, 1), \ldots, extr_Q(z, k-1), x, \\ \qquad extr_Q(z, k+1), \ldots, extr_Q(z, p+1)) & \text{if } 1 \leq k \leq p+1 \\ 0 & \text{otherwise} \end{cases}$$

Proposition .5.3. $replace_Q \in$ REC

Proof. This function is a piecewise-defined function from recursive functions and decidable predicates, so it is also a recursive function. □

Definition .5.4. Update function (*Función actualizar*)
Let $Q = (n, s) \in \text{WHILE}$.
$update_Q : \mathbb{N}^3 \to \mathbb{N}$

$$update_Q(z, k, x) = \begin{cases} replace_Q(z, 1, x) & \text{if } k = 1 \\ \sigma_1^{p+1}(extr_Q(z, 1) + 1, extr_Q(z, 2), \ldots, \\ \qquad extr_Q(z, k-1), x, extr_Q(z, k+1), \ldots, \\ \qquad extr_Q(z, p+1)) & \text{if } 2 \leq k \leq p+1 \\ 0 & \text{otherwise} \end{cases}$$

Proposition .5.4. $update_Q \in \text{REC}$

Proof. This function is a piecewise-defined function from recursive functions and decidable predicates, so it is also a recursive function. □

Definition .5.5. Next function (*Función siguiente*)
Let $Q = (n, s) \in$ WHILE
$Next_Q : \mathbb{N} \to \mathbb{N}$
$Next_Q(z) = \sigma_1^{p+1}(next_Q(extr_Q(z, 1), \ldots, extr_Q(z, p + 1)))$
where $next_Q$ is the next configuration function already defined for WHILE programs.

Proposition .5.5. $Next_Q \in$ REC

Proof.
Given $Q = (n, s) \in$ WHILE and $z \in \mathbb{N}$,

$$Next_Q(z) = \begin{cases} next_k(z) & \text{if } extr_Q(z, 1) = k, 1 \leq k \leq size(Q) \\ z & otherwise \end{cases}$$

where each $next_k$ function is defined depending on the specific Q WHILE program in the following way:
$line(s, k) = X_i := 0 \Rightarrow next_k(z) = update_Q(z, i + 1, 0)$
$line(s, k) = X_i := X_j \Rightarrow next_k(z) = update_Q(z, i + 1, extr_Q(z, j + 1))$
$line(s, k) = X_i := X_j + 1 \Rightarrow next_k(z) = update_Q(z, i + 1, extr_Q(z, j + 1) + 1)$
$line(s, k) = X_i := X_j - 1 \Rightarrow next_k(z) = update_Q(z, i + 1, extr_Q(z, j + 1) - 1)$
$line(s, k) = while\ X_i \neq 0\ do \Rightarrow$

$$next_k(z) = \begin{cases} update_Q(z, 1, k + 1) & \text{if } extr_Q(z, i + 1) \neq 0 \\ update_Q(z, 1, go(s, k)) & \text{if } extr_Q(z, i + 1) = 0 \end{cases}$$

$line(s, k) = od \Rightarrow next_k(z) = update_Q(z, 1, go(s, k))$

Note that $size$, go and $line$ are functions already defined. Also, for each code s and line k, $go(s,k)$ is a constant that does not depend on z, so it doesn't affect the recursivity of $next_k$.
As $extr_Q$ is defined by functions that are also in TREC, we have that $extr_Q$ is total recursive. Then, $next_k$ functions are recursive. Finally, as $Next_Q$ is a piecewise-defined function from recursive functions and decidable predicates, it is recursive.

□

Definition .5.6. Calculation function (*Función cálculo*)
Let $Q = (n,s) \in$ WHILE
$Cal_Q : \mathbb{N}^{n+1} \to \mathbb{N}$
$Cal_Q(\underline{x}, i) = \sigma_1^{p+1}(cal_Q(\underline{x}, i))$
where cal_Q is the calculation function already defined for WHILE programs.

Proposition .5.6. $Cal_Q \in$ REC

Proof. Given $Q = (n,s) \in$ WHILE, $\underline{x} \in \mathbb{N}^n$ and $i \in \mathbb{N}$, we see that

$$Cal_Q(\underline{x}, i) = \begin{cases} \sigma_1^{p+1}(1, \underline{x}, \underline{0}) & \text{if } i = 0 \\ Next_Q(Cal_Q(\underline{x}, i-1)) & \text{if } i > 0 \end{cases}$$

Then,

$$Cal_Q = \left\langle \sigma_1^{p+1}(C_1^n, \pi_1^n, \ldots, \pi_n^n, \theta^n, \ldots, \theta^n) \mid Next_Q(\pi_{n+2}^{n+2}) \right\rangle$$

where C_1^n is the constant one function of n arguments, which is clearly recursive.

As all the functions used are recursive, we have that $Cal_Q \in \text{REC}$

$\square$

Proposition .5.7. $T_Q \in \text{REC}$

Proof. Given $Q = (n, s) \in \text{WHILE}$ and $\underline{x} \in \mathbb{N}$,
$T_Q(\underline{x}) = minimum(\{t \in \mathbb{N} : extr_Q(Cal_Q(\underline{x}, t), 1) = size(Q) + 1\})$
Then,
$T_Q = \mu\left[subtraction(C_1^{n+1}, equal(C_{size(Q)+1}^{n+1}, extr_Q(Cal_Q, C_1^{n+1})))\right]$
where $equal = X_{V_{P_{equal}}}$, with P_{equal} the predicate of two arguments that gives if two numbers are equal or not. We can give the next expression for *equal* function:

$$equal = \left\langle \sigma(\theta) \mid \theta^2 \right\rangle (add(subtraction, subtraction(\pi_2^2, \pi_1^2)))$$

As *equal* function and constant functions are recursive, then T_Q is recursive.

$\square$

Proposition .5.8. $F_Q \in \text{REC}$

Proof. Given $Q = (n, s) \in \text{WHILE}$ and $\underline{x} \in \mathbb{N}$,
$F_Q(\underline{x}) = extr_Q(Cal_Q(\underline{x}, T_Q(\underline{x})), 2)$
Then,
$F_Q = extr_Q(Cal_Q(\pi_1^n, \ldots, \pi_n^n, T_Q), C_2^n)$
As constant functions are recursive, $C_2^n \in \text{REC}$, and then $F_Q \in \text{REC}$. □

Proposition .5.9. F(WHILE) $\subseteq$ REC

Proof. Let $f \in \text{F(WHILE)}$ be a WHILE-computable function.
Then $\exists Q \in \text{WHILE} : f = F_Q$
As $F_Q \in \text{REC} \Rightarrow f \in \text{REC}$ □

.6 Functioning of the Universal While program

Example .6.1. Execution of Universal Program Example Given the following while program that the Universal Program will have to simulate and that receives only one parameter:

```
Q = (1, s)
s :
   while X1 != 0 do
      X1 := X1-1
   od
```

The first thing to do, is to encode the given while program into a Gödel number using Gödel's encoding method:

$$
\begin{gathered}
code2N(s) = \Gamma\big(sent2N(s)\big) - 1 \\
sent2N(s) = 5\sigma_1^2\big(1-1, code2N(X1 := X1-1)\big) + 4 \\
code2N(X1 := X1-1) = \Gamma\big(sent2N(X1 := X1-1)\big) - 1 \\
sent2N(X1 := X1-1) = 5\sigma_1^2(0,0) + 3 = 3 \\
code2N(X1 := X1-1) = \Gamma(3) - 1 = 10 - 1 = 9 \\
sent2N(s) = 5\sigma_1^2(0,9) + 4 = 5 * 54 + 4 = 274 \\
code2N(s) = \Gamma(274) - 1 = 37950 - 1 = 37949
\end{gathered}
$$

The Universal Program will receive 2 paramaters: the 37949 that will store the code of the program that has to simulate and an imput of 1 (X1 = 37949, X2 = 1). Now, we are going to see how the universal program will execute instruction by instruction with the imputs described:

```
Xk+2 := godelk(X2,...,Xk+1); (* X3 := godelk(X2) -> X3 := 3 *)
```

Here, we just encoded the arguments of the program to simulate into a Gödel number.

```
Xk+2 := simulate(X1,Xk+2); (* X3 := simulate(X1, X3) *)
```

With this, we will store in X3 the result of the auxiliar function "simulate" that we will see now.

The merge function is used to handle the instruction's stack. Its implementation is located in the anexe. Basically, it receives two parameteres: the first one is the code of a while program and the second one is another piece of code that we want to put on top of the code of the first one. In this case, we have:

```
while X1 != 0 do
    X1 := X1-1
od
```

And since what we want to include on the top, are all the instructions inside the loop (in this case only one), what the function will return, is a Gödel number that encodes this program:

```
X1 := X1-1;
while X1 != 0 do
    X1 := X1-1
od
```

Since the implementation is already in the anexe, I will not show step by step how is done becouse it would only be an explanation on how we can modify the instruction's stack. What we would obtain is:

```
X1 := merge(X1, X4) (* X1 := 751924808 *)
```

Now, we can continue with the execution of the universal program with the next iteration.

Now, we will execute the second main auxiliar function, in charge of modifying the variable's stack:

As with the merge function, the implementation of replace is located in the anexe. In this case, instead of handling the instruction's stack, it handles the variable's stack. I

will not show its execution as with the merge function. In this case, it will write the value stored in X5 in the variable that X4 is pointing to and the function will return the Gödel number that encodes all the variables (in this case only X1 = 0):

```
X1 := replace(X2,X4,X5) (* X1 := 1 *)
```

After this, we return this result to the simulate function such that X2 (that stores all the variables of the program) is finally modified:

```
X2 := execute(X3,X2); (* X2 := 1 *)
X1 := pop(X1)
```

This the last auxiliar function located in the anexe, and the simplest to understand. What it does is to remove from the instruction's stack the instruction located at the top of it and return the Gödel number that encodes the program. In this case, it will remove the X1 := X1 - 1 that we placed previously outside of the loop:

```
X1 := pop(X1) (* X1 := 37950 *)
```

After this, we will start the third loop iteration.

After this we will jump again to the loop's head.
And after this, we return to the main function the modified variable's stack.

```
Xk+2 := simulate(X1,Xk+2); (* X3 := 1 *)
X1 := degodel(Xk+2,1) (* X1 := degodel(X3,1) -> X1 := 0 *)
```

So, what we obtain as an output, is the value 0 (that is in fact what the function given as parameter, would return with the imput of 1.

References

Chomsky, N. (1956). Three models for the description of language. *IRE Transactions on Information Theory*, *2*(2), 113–124.

Chomsky, N. (1959). On certain formal properties of grammars. *Information and Control*, *2*(2), 137–167. Retrieved from https://www.sciencedirect.com/science/article/pii/S0019995859903626

Chomsky, N., & Miller, G. (1958). Finite-state languages. *Information and Control*, *1*, 91–112.

Drobot, V. (1989). *Formal languages and automata theory*. Rockville (MD): Computer Science Press.

Floyd, R. W., & Beigel, R. (1994). *The language of machines: an introduction to computability and formal languages*. New York: Computer Science Press.

Gurari, E. (1989). *An introduction to the theory of computation*. Rockville, MD: Computer Science Press.

Harel, D. (1992). *Algorithmics: the spirit of computing*. Reading, MA: Addison-Wesley.

Harrison, M. (1978). *Introduction to formal language theory*. Reading, MA: Addison-Wesley.

Hopcroft, J., & Ullman, J. (1979). *Introduction to automata theory, languages and computation*. Reading, MA: Addison-Wesley.

Kleene, S. (1952). *Introduction to metamathematics*. Walters-Noordhoff & North-Holland.

Ramos, G., & Morales, R. (2011). *Teoría de autómatas y lenguajes formales* (G. Ramos, Ed.).

Turing, A. M. (1936). On computable numbers, with an application to the entscheidungsproblem. *Proceedings of the London Mathematical Society*, *42*(1), 230–265.

$simulate = (2, s)$

s :

```
(* X1 = sentence, X2 = variables *)
X1 := X1 + 1;
while X1 ≠ 0 do
    (* X1 != 0, so we do enter the loop *)
    X3 := degodel(X1,1);
    (* check if assignment or loop *)
    if senttype(X3) = 4 then
        (* It is equal to 4, so we do enter the if *)
        (* value of control variable *)
        X4 := lhs(X3);
        if degodel(X2,X4) ≠ 0 then
            (* It is equal to one, so we do enter the if *)
            X4 := rhs(X3);
            (* we store in X4 the number that encodes all the instructions inside the loop  *)
            X1 := merge(X1, X4)
od
```

$simulate = (2, s)$

s :

```
while X1 ≠ 0 do
    (* X1 != 0, so we do enter the loop *)
    X3 := degodel(X1,1);
    (* check if assignment or loop *)
    if senttype(X3) ≤ 3 then
        (* It is equal to 3, so we do enter the if *)
        X2 := execute(X3,X2);
od
```

$execute = (2, s)$

s :

```
(* X1 = sentences, X2 = variables *)
X3 := senttype(X1);
(* index of target variable *)
X4 := lhs(X1);
if X3 != 0 then
    (* X3 = 3 so we do enter the if *)
    (* value of source variable *)
    X5 := rhs(X1);
    X5 := degodel(X2,X5);
    if X3 = 3 then
        X5 := X5 - 1;
        (* X5 = 0 *)
        X1 := replace(X2,X4,X5)
    fi
```

$simulate = (2, s)$

s :

```
while X1 ≠ 0 do
    (* X1 != 0, so we do enter the loop *)
    X3 := degodel(X1,1);
    (* check if assignment or loop *)
    if senttype(X3) = 4 then
        (* It is equal to 4, so we do enter the if *)
        (* value of control variable *)
        X4 := lhs(X3);
        if degodel(X2,X4) = 0 then
            (* It is equal to 0, so we do enter the if *)
            X1 := pop(X1, X4)
od
```

$simulate = (2, s)$

s :

```
while X1 ≠ 0 do
    (* X1 = 0, so we do not enter the loop *)
od
X1 := X2 (* we return the variable's stack *)
```